



School of
Engineering

Projektarbeit (Informatik)

Classification of radio galaxies with Deep Learning

Autoren

Hannah Schlemßer

Hauptbetreuung

Schilling Frank-Peter

Nebenbetreuung

Denzel Philipp

Datum

05.06.2025

Erklärung betreffend das selbstständige Verfassen einer Bachelorarbeit an der School of Engineering

Erklärung betreffend das selbstständige Verfassen einer Projektarbeit an der School of Engineering

Mit der Abgabe dieser Projektarbeit versichert der/die Studierende, dass er/sie die Arbeit selbständig und ohne fremde Hilfe verfasst hat. (Bei Gruppenarbeiten gelten die Leistungen der übrigen Gruppenmitglieder nicht als fremde Hilfe.)

Der/die unterzeichnende Studierende erklärt, dass alle zitierten Quellen (auch Internetseiten) im Text oder Anhang korrekt nachgewiesen sind, d.h. dass die Bachelorarbeit keine Plagiate enthält, also keine Teile, die teilweise oder vollständig aus einem fremden Text oder einer fremden Arbeit unter Vorgabe der eigenen Urheberschaft bzw. ohne Quellenangabe übernommen worden sind.

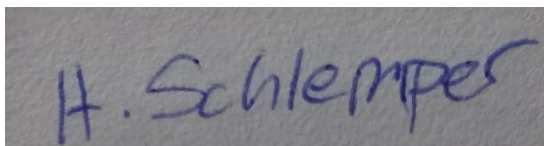
Bei Verfehlungen aller Art treten die Paragraphen 39 und 40 (Unredlichkeit und Verfahren bei Unredlichkeit) der ZHAW Prüfungsordnung sowie die Bestimmungen der Disziplinar massnahmen der Hochschulordnung in Kraft.

Ort, Datum:

Name Studierende:

Dietikon 06.06.2025

Hannnah Schlemper



Bemerkung zur Nutzung künstlicher Intelligenz in dieser Arbeit

Hier von Projektarbeit (als Referenz)

In dieser Arbeit wurde an verschiedenen Stellen künstliche Intelligenz verwendet. Hauptsächlich handelte es sich dabei um Sprachmodelle wie ChatGPT, Übersetzungsleistungen und Bildergenerierung. Die Plattform deepl.com wurde verwendet, um einzelne Begriffe oder Phrasen zwischen Deutsch und Englisch zu übersetzen.

ChatGPT wurde zur Verbesserung von Grammatik und Rechtschreibung eingesetzt hatte aber keinerlei Einfluss auf den Inhalt dieser Arbeit oder dieses Textes

Zusammenfassung

In dieser Arbeit wurden etablierte Bildklassifizierungsarchitekturen wie ResNet, EfficientNet und Swin-Transformer auf Radioteleskopbildern trainiert, um ihre Leistungsfähigkeit im direkten Vergleich zu untersuchen. Ein besonderer Fokus lag dabei auf dem Preprocessing der Bilddaten. Verschiedene Normalisierungsmethoden kamen zum Einsatz, um deren Einfluss auf die Modellleistung zu analysieren. Mit EfficientNet konnte dabei eine Validierungsgenauigkeit von über 88 % erzielt werden.

Abstract

In this work, established image classification architectures such as ResNet, EfficientNet, and Swin Transformer were trained on radio telescope images. The aim was to compare the architectures. Special attention was given to the preprocessing of the image data. Various normalization methods were tested to determine whether they could improve the models' performance. Using EfficientNet, a validation accuracy of over 88% was achieved.

Vorwort

Diese Bachelorarbeit wurde in Zusammenarbeit mit Frau Moritz Gehring (ZHAW, IT) erarbeitet. Dabei verhält es sich, gemäss der hier festgehaltener Vereinbarung mit den beiden Betreuenden Dozenten so, dass es in unseren beiden Arbeiten (Hannah Schlemper: Projektarbeit; Moritz Gehrig: Bachelorarbeit) bewusst zu Überschneidungen kommen kann und auch soll. Teile der Arbeit und auch dieses (hier) Berichts können also ähnliche oder sogar gleiche Inhalte enthalten. Zudem sei hier bemerkt, dass die Code-Basis für die gesamte Arbeit in Teamarbeit entwickelt wurde.

Ich bedanke mich herzlich bei Moritz für die Unterstützung und gemeinsamen Coding-Sessions. Auch bedanken möchte ich mich bei den Betreuenden Dozenten Frank-Peter Schilling und Philipp Denzel. Sie haben uns mit viel Material zum Nachdenken und mit wertvollen Tipps versorgt und auch unsere doch recht häufig auftauchenden Fragen immer zeitnah beantwortet.

Inhaltsverzeichnis

1 Einleitung	8
1.1 Ausgangslage	8
1.2 Stand der Technik	9
1.2.1 Datensätze	10
1.3 Zielsetzung	11
2 Theoretische Grundlagen	11
2.1 Neuronale Netze	11
2.1.1 Gradient-Descent und Backpropagation	13
2.2 Bildklassifikation	14
2.2.2 Relevante Architekturen	15
2.2.2.1 Convolutional Neural Networks	15
2.2.2.2 Vision-Transformer	19
2.2.2.3 Residual Neuronal Network	21
2.2.2.4 EfficientNet	23
2.2.2.5 Swin-Transformer	24
3 Vorgehen und Methode	26
3.1 Komponenten und Hyperparameter	27
3.1.1 Inputdaten und Preprocessing	27
3.1.2 Hyperparameter und veränderbare Modelkomponenten	33
3.1.3 Sweeps/ Logging	38
3.2 Vorbereitung	38
3.2.1 Erstellen der Trainingspipeline	38
3.2.2 Vorbereiten des Datasets	40
3.3 Modelle	42
3.3.1 ResNet	43
3.3.2 EfficientNet	46
3.3.3 Swin-Transformer	51
4 Resultate	54
4.1 ResNet	54
4.1.1 Erstes ResNet Experiment (Kapitel 3.3.1.1)	54
4.1.2 Zweites ResNet Experiment (Kapitel 3.3.1.2)	59

4.2 EfficientNet	65
4.2.1 Erstes EfficientNet Experiment (Kapitel 3.3.2.1)	65
4.2.2 Test der Hep-Normalisierung (Kapitel 3.3.2.2).....	67
4.2.3 Zweites EfficientNet Experiment (Kapitel 3.3.2.2)	68
4.3 Swin-Transformer	71
4.3.1 Erstes Swin-Transformer Experiment (3.3.3.1)	71
4.3.2 Zweites Swin Experiment (3.3.3.2).....	73
5 Diskussion und Ausblick	76
6 Verzeichnisse.....	78
6.1 Quellenverzeichnis	78
6.2 Tabellenverzeichnis	80
6.3 Abbildungsverzeichnis	81
7 Anhang	83
7.1 Aufgabenstellung aus Complexis	83
7.2 Weitere Anhänge.....	83

1 Einleitung

Die Faszination des Nachthimmels begleitet den Menschen seit jeher. Mit diversen Arten von Teleskopen wird er erforscht und kartographiert. Dabei kommen unter anderem Radioteleskope zum Einsatz, welche enorme Mengen an Aufnahmen liefern – sowohl von intragalaktischen Quellen, Planeten, Nebeln, Schwarzen Löchern als auch von extragalaktischen Quellen, fremden Galaxien.

In der Vergangenheit wurden die Bilder jener Teleskope oft von Freiwilligen analysiert. Jedoch wird durch die Entwicklung immer modernerer und besserer Teleskope auch der Rahmen des Möglichen immer grösser. Es werden immer besser auflösende, grössere Datensätze mit vielen Galaxien generiert. Zum Beispiel wird erwartet, durch das Radioteleskop Ska2 bis zu einer Milliarden Galaxien zu entdecken[1], ein Überschuss an Daten, welcher unmöglich von Menschen abzuarbeiten ist.

Dieser Überschuss verlangt nach einer effizienteren Methode der Klassifizierung. Künstliche Intelligenz bietet sich als Lösung an.

Bisher wurden schon einige Pilotprojekte durchgeführt, die versuchen, Radioteleskopbilder durch maschinelles Lernen zu klassifizieren. Diese Arbeit möchte sich jenen Projekten anschliessen und im Speziellen testen, wie gängige Bildklassifizierungsarchitekturen auf einem Datensatz von Radioteleskopaufnahmen von Galaxien abschneiden. Ziel ist es, die Performance der Architekturen zu vergleichen und das am besten geeignete Modell zu finden.

Dabei wird auch ein spezielles Augenmerk auf das Preprocessing der Daten gelegt, da sich von Radioteleskopen erzeugte Bilder in ihrer Form von natürlichen Farbbildern unterscheiden.

1.1 Ausgangslage

Sowohl in der Bildklassifizierung als auch bei der Objekterkennung wurden bereits mehrere Ansätze im Zusammenhang mit Radioteleskopbildern verfolgt.

In CLARAN – A Deep Learning Classifier for Radio Morphologies haben Wu et al. im Rahmen eines „Proof of Concept“ ein Objekterkennungsmodell für Radioteleskopbilder von Galaxien entwickelt.[2]

Dabei wurden vielversprechende Erfolge erzielt. Wu et al. haben ein Faster R-CNN trainiert und damit auf dem Radio-Galaxy-Zoo-OD-Datensatz eine Mean Average Precision von 83,6 % sowie eine empirische Genauigkeit von über 90 % erreicht. Ein Faster R-CNN ist ein Modell, das für die Objekterkennung eingesetzt wird. Aufgaben der Objekterkennung bestehen typischerweise aus zwei Komponenten: der Klassifikation und der Lokalisierung von Objekten.

In der Literaturrecherche wurden zwei Arbeiten identifiziert, die sich ausschließlich mit der Klassifikation von Galaxien befassen:

Die Erste ist „The FIRST Classifier: Compact and Extended Radio Galaxy Classification using Deep Convolutional Neural Networks“ von Alhassan et al.[3]

In dieser Studie wurde ein Convolutional Neural Network (CNN) auf einem Datensatz des FIRST-Radioteleskops trainiert. Dabei wurde eine Genauigkeit (Accuracy) von 97 % erreicht. Diese Arbeit weist zwar Ähnlichkeiten zur vorliegenden Untersuchung auf, unterscheidet sich jedoch in zwei zentralen Aspekten:

Zum einen ist der in dieser Arbeit verwendete Datensatz granularer, da er sechs Klassen unterscheidet, während Alhassan et al. nur vier Klassen verwenden. Zum anderen werden hier mehrere bereits etablierte Modellarchitekturen miteinander verglichen, anstatt ein einzelnes CNN zu trainieren.

Die Zweite relevante Arbeit ist „CNN Architecture Comparison for Radio Galaxy Classification“ von Becker et al.[4]. In dieser Studie werden verschiedene CNN-Architekturen hinsichtlich ihrer Performance und Rechenanforderungen verglichen. Ein besonderes Augenmerk liegt dabei auf der Generalisierung auf neue, unbekannte Daten.

Im Gegensatz dazu liegt der Fokus der vorliegenden Arbeit stärker auf der gezielten Optimierung einzelner Architekturen durch systematische Hyperparameter-Suchen, mit dem Ziel, für jede Architektur die bestmögliche Leistung zu erzielen

1.2 Stand der Technik

Eine gängige Benchmark für Bildklassifizierungsmodelle ist die ImageNet-1k Benchmark. ImageNet-1k ist ein grosser Datensatz mit 1.000 Klassen und über 1,2 Millionen Trainingsbildern an denen Modelle trainiert und getestet werden. Die Resultate vieler Modelle sind zum Beispiel auf der paperswithcode Website zusammengetragen und einsehbar[5].

Zwischen 2010 und 2017 fand auf Basis dieses Datensatzes jährlich die ImageNet Large Scale Visual Recognition Challenge (ILSVRC) statt. Ziel war es, das Modell mit der höchsten Klassifikationsgenauigkeit einzureichen, gemessen am Top-1- und Top-5-Fehler.

In 2012 wurde mit Alex-Net von Krizhevsky et al. das erste tiefe Convolutional Neural Network (CNN) publiziert[6]. Alex-Net gewann die ILSVRC deutlich, mit einem Top-5-Fehler von 15,3 %, während das zweitbeste Modell dieser Zeit bei etwa 26 % lag[7]. Mit diesem Erfolg leitete es eine neue Ära des Deep Learning ein und motivierte den rasanten Fortschritt von CNN-Architekturen in den Folgejahren.

In der Bildklassifikation haben sich zum heutigen Stande zwei Typen von Architekturen etabliert: CNNs wie ResNet und Vision Transformer. In der vorliegenden Arbeit werden

drei Architekturtypen untersucht: ResNet und EfficientNet (beide CNN-basiert) sowie der Swin Transformer.

Die Leistungsfähigkeit dieser Architekturen hängt stark von der konkreten Implementierung ab, beispielsweise von der Modellgrösse und der gewählten Trainingsstrategie. Alle drei Architekturen sind gut für die Klassifikation natürlicher Bilder geeignet und werden in dieser Arbeit auf ihre Anwendbarkeit bei Radioteleskopbildern getestet.

1.2.1 Datensätze

Alle Modelle werden auf einem Datensatz trainiert, der auf einem Subset des Radio Galaxy Zoo Data Release 1 (DR1) basiert. [8] Dabei handelt es sich um ein Citizen-Science-Projekt, in dessen Rahmen Freiwillige Radiogalaxien in astronomischen Bilddaten identifizierten und klassifizierten.

Der Radio Galaxy Zoo (RGZ) umfasst 100 185 Klassifizierungen von insgesamt 99 146 Radioquellen aus dem Faint Images of the Radio Sky at Twenty Centimeters (FIRST) Survey sowie 583 Quellen aus dem Australia Telescope Large Area Survey (ATLAS). Die durchschnittliche Zuverlässigkeit der Klassifikationen beträgt 83 % und basiert auf gewichteten Konsensbewertungen, die mit Expertenurteilen abgeglichen wurden. Es gibt mehr Klassifizierungen als Quellen, da manche Quellen in mehreren klassifizierten Bildern auftauchen.

Radiogalaxien bestehen typischerweise aus strukturellen Komponenten wie Loben, Jets und einem zentralen Kern. Den Teilnehmenden wurden Radiobilder gezeigt, die mit Infrarotaufnahmen überlagert waren. Auf diesen sollten sie markieren, welche Komponenten vermutlich zusammengehören und welche der überlagerten Infrarotgalaxien mit hoher Wahrscheinlichkeit die Quelle der jeweiligen Radioemission ist.

Beim Erstellen des Datensatzes wurde sich zu Nutze gemacht, dass ein kleiner erfahrener Teilnehmeranteil einen Grossteil der Klassifizierungen durchführte. Die Kompetenz der Teilnehmer wurde anhand eines sogenannten „Gold Samples“ ermittelt, das aus 20 von Experten klassifizierten Bildern besteht, welche wiederholt in den Klassifizierungsaufgaben auftauchten. Abhängig von der Übereinstimmung der individuellen Klassifizierungen mit den Expertenbewertungen wurden die Beiträge der Teilnehmenden unterschiedlich, mit einem Faktor zwischen eins und fünf, gewichtet.

Um die Verlässlichkeit der Radioklassifikationen im DR1-Datensatz zu bewerten, wurden 1000 Quellen mit mittlerem gewichtetem Nutzerkonsens (0,6–0,8) durch Experten erneut klassifiziert. Dabei stimmten Experten und Nutzer in 85 % der Fälle überein, wenn der Nutzerkonsens über 0,65 lag. Unterhalb dieses Schwellenwerts nahm die Übereinstimmung deutlich ab. Daher wurde ein Konsens-Grenzwert von 0,65 als Mindestanforderung für zuverlässige Klassifikationen im RGZ DR1 festgelegt.

1.3 Zielsetzung

In dieser Arbeit werden verschiedene Modelle unterschiedlicher Architekturen auf dem RGZ-Datensatz trainiert und durch Hyperparameter-Suchen sowie den möglichst optimal gewählten Komponenten so weit optimiert, dass sie ihre bestmögliche Genauigkeit erreichen. Einer dieser Hyperparameter betrifft die Anwendung verschiedener Preprocessing-Methoden zur Bildvorverarbeitung.

Ziel ist die Beantwortung folgender Fragen:

1. Welches Modell erzielt die beste Leistung?
2. Lässt sich eine Methode der Bildnormalisierung identifizieren, die modellübergreifend zu überdurchschnittlichen Ergebnissen führt – also eine Normalisierungsstrategie für Radioteleskopbilder, die unabhängig vom eingesetzten Modell besonders effektiv ist?

2 Theoretische Grundlagen

2.1 Neuronale Netze

Ein einfaches neuronales Netz besteht aus Schichten von Neuronen, der Input-Schicht, einer oder mehreren versteckten Schichten (Hidden Layers) und der Output-Schicht. Jedes Neuron ist mit den Neuronen der nächsten Schicht durch Gewichte (Weights) verbunden. Diese bestimmen, wie stark der Wert eines Neurons in einer Schicht L den Wert eines Neurons der nächsten Schicht $L+1$ beeinflusst. Ein Neuron berechnet eine

gewichtete Summe seiner Eingaben, addiert dazu einen Bias-Term und wendet darauf eine Aktivierungsfunktion an, um den Wert nichtlinear zu transformieren.

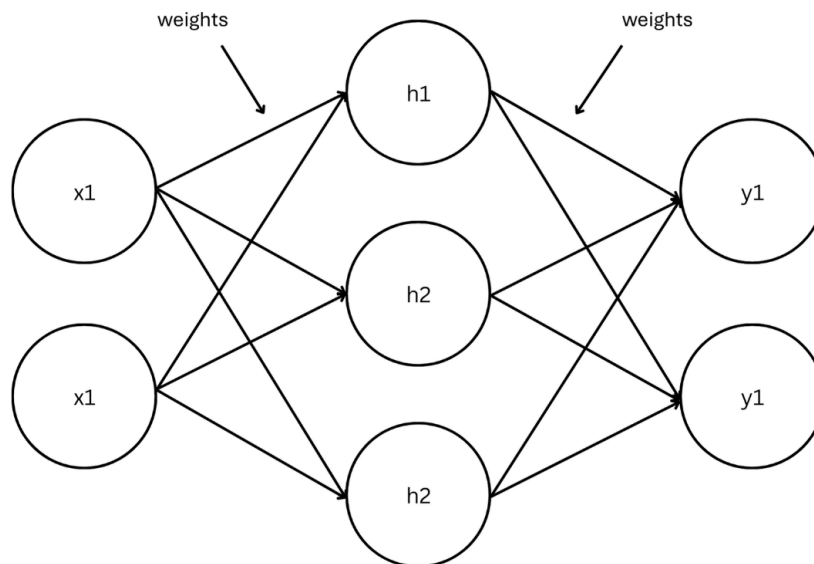


Abbildung 1: Schematische Darstellung eines seichten Neuronalen Netzes. Darstellungen aus [9] nachempfunden

Um ein Beispiel anhand Abbildung 1 zu machen:

In dem dritten Neuron des Hidden-Layers h_3 wird die Aktivierung a_3^2 folgendermassen berechnet:

$$a_3^2 = a(z_3^2), \quad \text{mit} \quad z_3^2 := b_3^2 + \sum_{j=1}^2 \omega_{3j} \cdot a_j^1$$

Dabei ist $a(\cdot)$ die Aktivierungsfunktion, in der Regel ein rectified linear Unit oder ReLU, a_j^1 ist der Output des j -ten Neurons aus dem vorherigen Layer, in diesem Fall des Input Layers.

$$ReLU(x) = \max(0, x)$$

ω_{31} ist hierbei das Gewicht (Weight) des Ersten Inputs, oder genereller des ersten Vorgängerneurons, auf das Neuron, ω_{32} das Gewicht des zweiten. ω_{3j} gibt also an wieviel Einfluss ein Vorgängerneuron auf ein Folgeuron hat. b_3^2 ist der Bias-Term von Neuron 3.

Diese Berechnung wird für jedes Neuron in jeder versteckten Schicht durchgeführt:

$$a_i^L = a \left(b_i^L + \sum_{j=1}^{n_{L-1}} \omega_{ij} \cdot a_j^{L-1} \right)$$

Alle Gewichte ω_{ij} und Biases b_i^L sind trainierbare Parameter.

Der finale Output-Vektor besteht aus reellen Zahlen, der gewichteten Summe der vorhergehenden Hidden-Layers, die als Wahrscheinlichkeiten für die jeweiligen Klassen interpretiert werden können.

Das Training eines solchen Netzwerkes geschieht in der Regel in vier Schritten:

1. Forward Pass: Eine Eingabe wird durch das Netzwerk geschickt, und eine Vorhersage wird gemacht.
2. Loss-Berechnung: Der Fehler zwischen Vorhersage und tatsächlichem Label wird mit einer Loss-Funktion berechnet.
3. Backward Pass (Backpropagation): Mit dem Gradientenabstieg (Gradient-Descent) wird berechnet, wie stark jeder Parameter zum Fehler beigetragen hat.
4. Optimizer-Schritt: Der Optimizer nutzt die Gradienteninformationen, um die Parameter des Modells in Richtung eines kleineren Fehlers anzupassen.

Deep Learning bezeichnet den Einsatz neuronaler Netze mit mehreren Hidden-Layern.

2.1.1 Gradient-Descent und Backpropagation

Nachdem eine Klassifizierung durchgeführt wurde, kann der Fehler (Loss) zwischen Vorhersage und tatsächlichem Label berechnet werden. Dazu gibt es verschiedene Formeln und Ansätze. Diese Arbeit verwendet den Cross-Entropy Loss, grundsätzlich lässt sich der Loss jedoch als Funktion der trainierbaren Parameter (Weights und Biases) formulieren:

$$L(\theta)$$

Ziel des Trainings ist es, diese Funktion zu minimieren. Nach einem Trainingsschritt werden dazu die Ableitungen der Funktion nach allen trainierbaren Parametern berechnet:

$$\frac{dL(\theta)}{d\theta} = \begin{bmatrix} \frac{dL}{d\omega_1} \\ \frac{dL}{d\omega_2} \\ \vdots \\ \frac{dL}{d\beta_1} \\ \vdots \\ \frac{dL}{d\beta_n} \end{bmatrix}$$

Diese Ableitungen, auch Gradienten genannt, geben an, wie stark und in welche Richtung der Loss durch eine Veränderung eines bestimmten Parameters beeinflusst wird.

Die Berechnung aller dieser Gradienten ist in der Praxis sehr aufwändig. Daher wird der sogenannte Backpropagation-Algorithmus eingesetzt. Dabei wird der Fehler (Loss) vom Ausgang des Netzwerks aus rückwärts durch die Schichten propagiert. Schritt für Schritt wird mithilfe der Kettenregel die Ableitung des Fehlers zunächst nach den Ausgaben der Neuronen, dann nach ihren Eingaben und schliesslich nach den dazugehörigen Gewichten und Biases berechnet.

Für jeden Layer wird also auf Basis der Gradienten der folgenden Schicht berechnet, wie sich die Weights und Biases der aktuellen Schicht anpassen sollten, um den Fehler zu verringern. Zusätzlich werden dabei auch die Gradienten der Neuronenausgaben des vorherigen Layers berechnet, um die Kette im nächsten Schritt fortzusetzen.

Auch wenn die Aktivierung vorheriger Neuronen nicht direkt geändert werden kann, werden sie durch die Anpassung der Weights und Biases der ihnen vorgelagerten Schichten indirekt beeinflusst.

Die berechneten Gradienten werden an den Optimizer übergeben. Dieser passt die Parameter des Modells auf Basis dieser Informationen an. Verschiedene Optimizer verwenden unterschiedliche Strategien zur Parameteranpassung, wie adaptive Lernraten.

Grundsätzlich erfolgt die Aktualisierung nach dem Prinzip:

$$\theta \leftarrow \theta - \alpha \frac{dL(\theta)}{d\theta}$$

Dabei ist α die Lernrate, ein sogenannter Hyperparameter, der vor dem Training festgelegt wird und bestimmt, wie gross der Schritt in Richtung des Gradienten ausfällt.

Gradient Descent bildet die Basis des Lernens. In der Praxis werden komplexere Optimierungsmethoden, Stochastic Gradient Descent oder Adam angewandt, deren Implementationen in der Methodik beschrieben werden.

2.2 Bildklassifikation

In dem Buch „Understanding Deep Learning“ beschreibt S. J. D. Prince drei grundlegende Eigenschaften von Bildern, die bei deren Klassifizierung berücksichtigt werden sollten [9]:

1. Bilder sind hochdimensional. Ein kleines Bild mit einer Auflösung von 224×224 Pixeln und drei Farbkanälen umfasst bereits 150.528 Eingabewerte.

2. Zwischen benachbarten Pixeln bestehen statistische Abhängigkeiten. Strukturen und Kanten ergeben sich aus geordneten Übergängen von Pixelwerten, was sich nutzen lässt.
3. Der Bildinhalt bleibt unter geometrischen Transformationen weitgehend invariant. Ein Bild eines Baumes bleibt ein solches, auch wenn es rotiert oder um einige Pixel verschoben wird.

Obwohl die in dieser Arbeit bearbeiteten Bilder von Radioteleskopen sich in einigen Aspekten von natürlichen Bildern unterscheiden, weisen auch sie die genannten Eigenschaften auf. Ein Bild einer bestimmten Galaxienart bleibt ein solches, auch wenn es rotiert oder um einige Pixel verschoben wird. Zudem bestehen auch in diesen Bilddaten statistische Zusammenhänge zwischen benachbarten Pixeln. Zwar verfügen Radioteleskopbilder nur über einen Kanal, im Vergleich zu natürlichen Bildern die drei Farbkanäle besitzen, dennoch wird angenommen, dass Architekturen, die sich im Allgemeinen für die Klassifikation natürlicher Bilder bewährt haben, auch für die Klassifikation von Radiobildern geeignet sind.

In dieser Arbeit werden verschiedene Modelle zweier Typen solcher Architekturen getestet: Convolutional Neural Networks (CNN) und Vision Transformer.

CNNs beschreiben eine Architektur, die unter der Annahme entworfen wurde, Bilder oder bildähnliche Daten zu klassifizieren. Dabei wird davon ausgegangen, dass die zu verarbeitenden Daten unter anderem die oben genannten Eigenschaften aufweisen. Solche vorab getroffenen Annahmen über die Struktur von Daten werden als *inductive biases* bezeichnet. Durch die Berücksichtigung dieser *inductive biases* sind CNNs vergleichsweise schlank und sehr effizient in der Lösung ihrer spezifischen Aufgabe.

Vision Transformer verfügen über keine ausgeprägten *inductive biases* und sind somit nicht explizit auf die Bildklassifikation ausgerichtet. Dennoch haben sie in diesem Aufgabenbereich bemerkenswerte Ergebnisse erzielt – insbesondere dann, wenn sie auf grossen Datensätzen trainiert wurden. [10]

Im Folgenden Kapitel werden alle verwendeten Architekturen genauer erklärt.

2.2.2 Relevante Architekturen

2.2.2.1 Convolutional Neural Networks

Bei den Verwendeten Architekturen ResNet und EfficientNet, handelt es sich um Versionen von Convolutional Neural Networks.

„Convolution“ bedeutet „Faltung“ und bezeichnet in diesem Kontext die gewichtete Summe von Eingabewerten. CNNs verbinden, wie die meisten neuronalen Netze, die Werte des Eingabevektors über trainierbare Weights mit Hidden Units. In diesen werden, wie auch bei einem gewöhnlichen Neuronalen Netz, die gewichteten und aufsummierten

Eingabewerte um einen ebenfalls trainierbaren Bias ergänzt und anschliessend durch eine Aktivierungsfunktion nichtlinear transformiert.

Da in Bildern statistische Zusammenhänge zwischen benachbarten Pixeln bestehen, verarbeiten CNNs Eingabedaten lokal. Dies geschieht mithilfe sogenannter *Filter* oder *Kerne* (*Kernels*), die in *Convolutional Layers* realisiert sind. Ein solcher Filter besteht aus einer festen Anzahl von Weights, die über die Eingabedaten verschoben und an jeder Position identisch angewendet werden.

Im eindimensionalen Fall, wie in Abbildung 2 unter der a) und b) dargestellt, transformiert ein Convolutional Layer einen Eingabevektor

$$x = (x_1, x_2, \dots, x_n)$$

in einen Ausgabevektor

$$z = (z_1, z_2, \dots, z_m)$$

durch Anwendung eines Filters mit Gewichtungen

$$w = (w_1, w_2, \dots, w_k)$$

wobei k die *Kernelgrösse* (*kernel size*) bezeichnet und in der Regel ungerade ist. Die Ausgabe an Position i ergibt sich dann durch die gewichtete Summe der benachbarten Eingabewerte:

$$z_i = \sum_{j=-k}^k w_j \cdot x_{i+j}$$

Dabei werden die gleichen Gewichtungen w_j an jeder Position des Eingabevektors verwendet. Diese lokale Gewichtung benachbarter Werte stellt einen der zentralen inductive biases von CNNs dar und nennt sich Weight Sharing.

Am Ausgang des Convolutional Layers wird der berechnete Wert z_i um einen trainierbaren Bias-Term b ergänzt und anschliessend durch eine Aktivierungsfunktion $a(\cdot)$ nichtlinear transformiert, typischerweise durch die ReLU-Funktion. Diese bilden den Inhalt der hidden units:

$$h_i = a(z_i + b)$$

Die Anzahl der Eingabewerte, die in die Berechnung jedes einzelnen Ausgabewerts einfließen, wird durch die Grösse des Filters bestimmt. Die Ausgabevektoren der Filter werden auch Feature Maps genannt.

Filter besitzen nicht nur eine bestimmte Grösse, sondern auch eine Schrittweite (Stride) sowie eine Ausdehnung (Dilation). Während die Schrittweite bestimmt, in welchen

Abständen der Filter über das Eingabebild verschoben wird, ermöglicht die Ausdehnung, mit vergleichsweise wenigen Gewichtungen eine grössere Fläche des Bildes abzudecken, indem Abstände zwischen den benachbarten Filterelementen eingefügt werden.

Es gibt zwei gängige Ansätze, um mit dem Problem umzugehen, dass an den Rändern eines Eingabevektors nicht genügend Werte für die Faltung zur Verfügung stehen. Eine Möglichkeit besteht darin, den Eingabevektor künstlich zu erweitern, entweder durch *Zero Padding*, bei dem Nullen an den Rändern hinzugefügt werden, oder durch eine zyklische Fortsetzung des Inputs. Der alternative Ansatz, die *Valid Convolution*, berechnet nur jene Outputs, an welchen der Filter vollständig innerhalb des Eingabebereichs liegt.

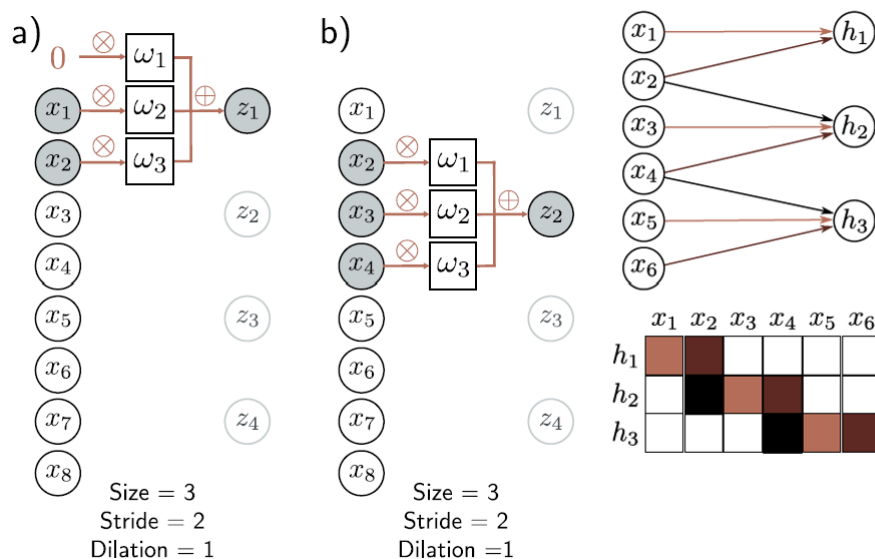


Abbildung 2: Übernommen aus [9]. Gezeigt wird ein 1D-Convolutional-Layer mit Kernelgröße 3, Stride 2 und Dilation 1. In (a) und (b) wird dargestellt, wie ein Filter mit drei trainierbaren Gewichten $\omega_1, \omega_2, \omega_3$ über die Eingabe $\vec{x}$ gleitet. Zero-Padding wird links angewendet, um auch den ersten Eingabebereich abzudecken. Die dritte Darstellung rechts zeigt den vollständigen Filter und dessen Gewichtsmatrix, wobei identische Gewichte farblich gleich dargestellt sind. Nicht verwendete Gewichtseinträge (Nullen) sind weiss. Bias-Terme sind nicht gezeigt.

Ein Convolutional-Layer verfügt in der Regel über mehrere Filter, wobei jeder Filter ein eigenes Merkmal extrahiert. Die Ausgaben der Filter werden nicht miteinander kombiniert, sondern als separate Kanäle gestapelt. Sie bilden gemeinsam den mehrkanaligen Ausgabebild, ein multidimensionales Array, des Layers. Dieser Tensor wird in seiner Gesamtheit an den nächsten Layer weitergegeben. Dort kann wiederum jeder Filter, nun mit einer Tiefe entsprechend der Anzahl der Eingangskanäle, kanalübergreifend auf sämtliche Informationen zugreifen.

CNNs verfügen in der Regel über mehr als einen Convolutional-Layer. Am Ausgang des ersten Layers, also nach Anwendung der Aktivierungsfunktion, kann ein neuer Satz von

Filtern angesetzt und trainiert werden. Jede nachfolgende Schicht verarbeitet somit die Ausgabedaten der vorherigen Schicht.

In der Praxis werden zur Bildklassifizierung in der Regel zweidimensionale CNNs verwendet.

$$X \in \mathbb{R}^{H \times W}$$

mit Höhe H und Breite W dargestellt werden. Der zweidimensionale Faltungsfilter ist ebenfalls eine Matrix, typischerweise

$$h_{ij} = a \left(\beta + \sum_{m=1}^M \sum_{n=1}^M \omega_{mn} \cdot x_{i+m-\lfloor M/2 \rfloor, j+n-\lfloor M/2 \rfloor} \right), \lfloor M/2 \rfloor \text{ wird aufgerundet}$$

wobei die Filtergrösse M bestimmt, wie viele benachbarte Pixel in der Berechnung berücksichtigt werden. Der Filter wird sowohl horizontal als auch vertikal über das zweidimensionale Eingabebild verschoben, um an jeder Position einen Ausgabewert zu erzeugen. Abbildung 3 zeigt ein visuelles Beispiel für einen 2D-Filter.

Normalerweise besitzt das Bild mehrere Kanäle (meist RGB mit drei Kanälen). Darum wird der Filter um eine Dimension auf $3 \times M \times M$ erweitert, sodass er über alle Kanäle hinweg wirkt. Die Faltung erfolgt kanalweise mit anschliessender Summation. Der Ausgabekanal besteht aus einer gewichteten Summe über $3 \times M \times M$ Werte plus einem Bias-Term und Aktivierungsfunktion.

Dieser Vorgang kann mit unterschiedlichen Filtern wiederholt werden, um unterschiedliche Kanten zu lernen. Die einzelnen Ergebnisse werden anschliessend zu einem dreidimensionalen Tensor zusammengefügt.

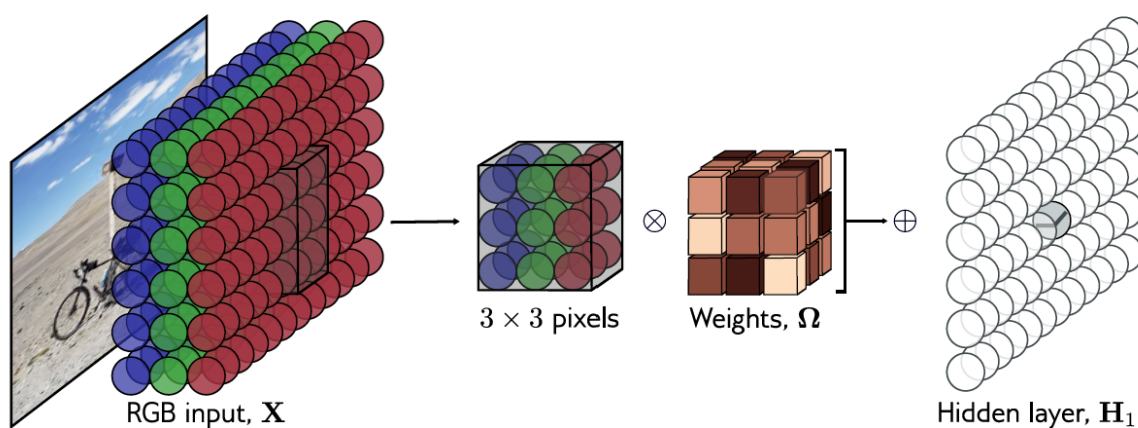


Abbildung 3: Übernommen aus [9], Die Abbildung zeigt eine 2D-Convolution auf ein Bild angewandt. Dabei wird das Bild als zweidimensionaler Input mit drei Kanälen interpretiert, die den Rot-, Grün- und Blau-Komponenten entsprechen. Bei Verwendung eines 3×3 -Kernels wird jede Pre-Activation z_{ij} des Hidden-Layers als punktweise Multiplikation der 3×3 Weights Matrix mit dem entsprechenden 3×3 -RGB-Ausschnitt des Bildes berechnet, aufsummiert und um einen Bias ergänzt.

Auch im Zweidimensionalen gib es Schrittweiten, Ausdehnung und Padding, die nun jeweils eine Komponente hinzugewinnen.

2.2.2.2 Vision-Transformer

Transformer wie in „Understanding Deep Learning“ beschrieben, wurden ursprünglich als Architekturen eingeführt, die besonders für die Verarbeitung natürlicher Sprache geeignet sind, und bringen eigene *inductive biases* mit[9].

Die grundlegende Idee hinter Transformern ist die Self-Attention. Dabei werden die Eingabedaten, beispielsweise Wörter oder Wortsegmente eines Textes, in einem Embedding-Layer in Vektorrepräsentationen überführt. Anschließend verarbeitet das Modell diese Repräsentationen mithilfe von Self-Attention.

Self-Attention im Zusammenhang mit Sprache beschreibt, wie stark ein Wort mit den anderen Wörtern in einer Sequenz zusammenhängt. Ein stark vereinfachtes Beispiel, das lediglich dem intuitiven Verständnis des Konzepts dient, wäre:

In dem Satz „Der Affe isst eine Banane.“ würde das Wort „Der“ stärker mit „Affe“ in Beziehung stehen als beispielsweise mit „eine“. Die Attention zwischen „Der“ und „Affe“ wäre in diesem Fall also größer.

Ein grundlegender Self-Attention-Block $sa[\cdot]$ erhält eine Sequenz von N $x_1 \dots x_N$ Tokens, wobei jeder Token ein Vektor der Größe $D \times 1$ ist. Als Ausgabe produziert der Block wiederum eine Sequenz von N Vektoren derselben Dimension.

Für jedes Token wird berechnet, wie stark es auf alle anderen Tokens „achtet“:

$$sa_n[x_1, \dots, x_N] = \sum_{m=1}^N a[k_m, q_n] \cdot v_m$$

Dabei ist q_n die sogenannte Query, also das Token, das Aufmerksamkeit verteilt und jedes k_m ist ein Key, also ein Token, auf das geachtet wird.

In jedem Schritt betrachtet eine Query alle Keys der Sequenz. Jeder Input-Token wird dabei genau einmal als Query verwendet, sodass für jede Position ein eigener Ausgabewert entsteht.

Die Attentions $a[k_m, q_n]$ ergeben sich aus dem skalierten Skalarprodukt zwischen Query- und Key-Vektoren, gefolgt von einer Softmax-Normalisierung.

Man kann diese Operation auch effizient als Matrixmultiplikation zwischen allen Query- und Key-Vektoren auffassen.

Am Ende wird die Attention noch mit Values v_m gewichtet.

Queries, Keys und Values werden folgendermassen berechnet:

$$V[X] = \beta_v 1^T + \Omega_v X$$

$$K[X] = \beta_k \mathbf{1}^T + \Omega_k X$$

$$Q[X] = \beta_q \mathbf{1}^T + \Omega_q X$$

Und die Attention wird neu dargestellt als:

$$\text{Sa}[X] = V[X] \cdot \text{Softmax}(K[X]^T Q[X])$$

Dabei sind Ω_v, Ω_k und Ω_q die trainierbaren Weigts und β_v, β_k und β_q trainierbare Biases.

Damit die Self-Attention effektiv funktioniert, scheinen mehrere Attention-Blöcke auch Heads genannt notwendig zu sein. Es wird vermutet, dass sie das Self-Attention-Netzwerk robuster gegenüber ungünstigen Initialisierungen machen.[9]

Dabei werden die Eingabevektoren, wie in Abbildung 4 dargestellt, auf mehrere Heads aufgeteilt, deren Ausgaben anschließend konkateniert werden.

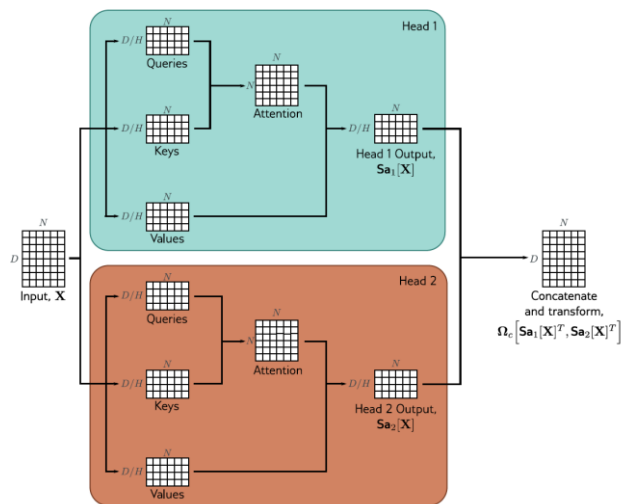


Abbildung 4: Multi-Head Self-Attention. Übernommen aus[9]

In ihrem Paper „An Image Is Worth 16×16 Words“ beschreiben Dosovitskiy et al. eine Strategie, Transformer zur Klassifikation von Bildern einzusetzen. [10]

Dabei werden Bilder in 16 x 16 Pixel Patches unterteilt. Diese Patches werden anschließend geflattet und embedded, das heißt in Token-Vektoren umgewandelt. Anschließend wird ein Positional Encoding hinzugefügt, ein Embedding, das angibt, das wievielte Patch in der Sequenz jeweils vorliegt.

Diese Embeddings werden gemeinsam mit einem sogenannten Classification Token, einem initial leerem Embedding, wie in Abbildung 5 dargestellt, einem Encoder übergeben. Ein Encoder ist ein Transformer-Block, der die Kontextinformationen der Eingabesequenz erfassen soll.

In diesem Fall wird der Encoder jedoch zur Klassifikation eingesetzt: Der Classification Token dient dabei als Träger der zusammengefassten Information über die gesamte

Sequenz. Während der Verarbeitung durch die Encoderschichten sammelt dieser Token Attention von allen anderen Positionen ein.

Aus den im Classification Token gesammelten Informationen wird am Ende mithilfe eines MLP-Heads, einem kleinen Feedforward-neuronalen Netzwerkblock, der als Klassifikator dient, eine Vorhersage erzeugt.

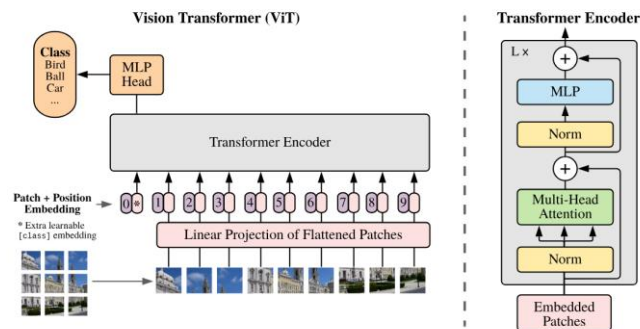


Figure 1: Model overview. We split an image into fixed-size patches, linearly embed each of them, add position embeddings, and feed the resulting sequence of vectors to a standard Transformer encoder. In order to perform classification, we use the standard approach of adding an extra learnable "classification token" to the sequence. The illustration of the Transformer encoder was inspired by Vaswani et al. (2017).

Abbildung 5: Aufbau des Vision Transformer. Übernommen aus [10]

Vision Transformer übertreffen CNNs in Klassifikationsaufgaben insbesondere dann, wenn sie auf sehr großen Datensätzen trainiert wurden.

2.2.2.3 Residual Neuronal Network

Das ResNet ist eine der drei in dieser Arbeit verwendeten Architekturen. Es handelt sich dabei um eine spezielle Form eines CNNs, das sogenannte Skip-Connections implementiert.

Nachdem in 2012 CNNs mit AlexNet die ImageNet Challenge mit bahnbrechenden Ergebnissen abgeschlossen hatten, entwickelte sich der Trend, immer tiefere Netzwerke zu entwerfen und diese erneut an der Challenge zu erproben.

In 2015 stellten He et al. bei diesem Wettkampf in ihrer Publikation „Deep Residual Learning for Image Recognition“ fest, dass der Trainingsfehler von CNNs ab einer bestimmten Modelltiefe zunimmt.[11] Dabei handelt es sich nicht um Overfitting, bei dem der Trainingsfehler zwar sinkt, der Fehler auf den Testdaten jedoch stagniert oder zunimmt, sondern vielmehr um das Problem, dass sich nicht alle Systeme beziehungsweise Modelle gleich gut optimieren lassen.

Sie stellen folgendes Gedankenexperiment vor: es existieren zwei Modelle. Ein kleines Modell, das eine zufriedenstellende Klassifizierungsfunktion trainiert hat und ein zweites, tieferes Modell, das von der Architektur her aus den gleichen Schichten wie das

erste Modell sowie weiteren extra Schichten besteht. Das tiefere Modell müsste in der Lage sein mindestens gleich gut zu klassifizieren wie das kleinere, denn theoretisch könnte der „vordere Teil“ die Parameter des kleineren Modells kopieren, und die zusätzlichen Schichten müssten dann, um gleich gute Ergebnisse zu erzielen die Identitätsfunktion abbilden.

Das grössere Modell sollte also gleichgut oder besser performen als das kleinere. Das dies nicht der Fall ist zeigt auf, dass manche Modelle schwieriger zu optimieren sind.

Beim Design von ResNets wurde davon ausgegangen, dass in den meisten Fällen die Identitätsfunktion, mit einigen kleineren Abweichungen, dem gewünschten Output bereits nahekommt und dass es daher einfacher ist, nur diese Abweichungen zu lernen, als sämtliche Merkmale von Grund auf zu erfassen. Also, anstatt wie bei einem klassischen Layer $\tilde{x} = f(x)$ zu lernen, wird gelernt, welche Veränderungen zur Identität hinzugefügt werden müssen, also Layer $\tilde{x} = x + f(x)$.

Abbildung 6 zeigt den grundlegenden Aufbau eines Residual-Learning-Blocks, der Veränderungen lernt, die der Identität hinzugefügt werden sollen. Abbildung 7 zeigt den schematischen Aufbau eines ResNets.

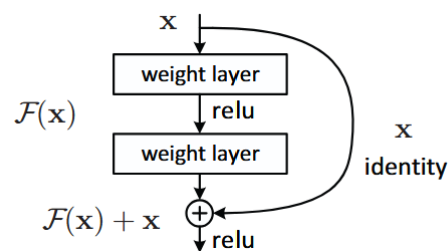


Abbildung 6: Residual-Learningblock. Übernommen aus[11]

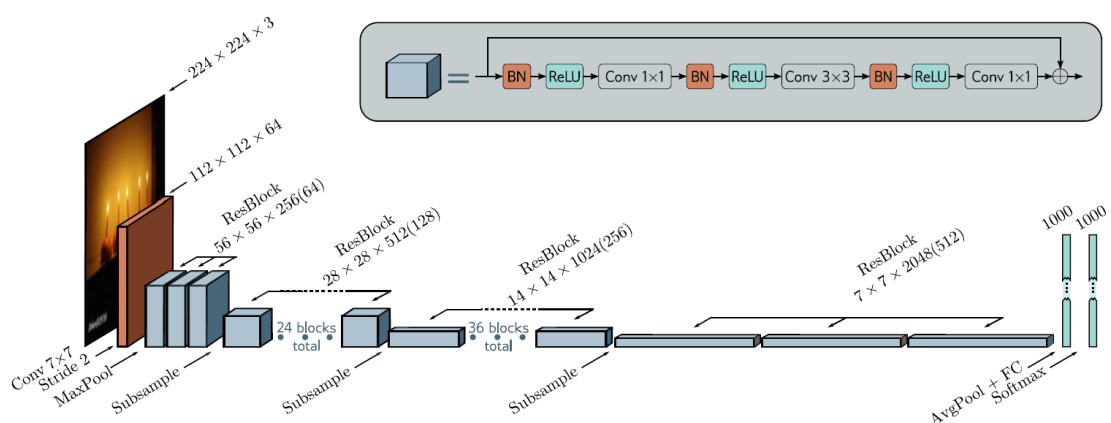


Abbildung 7: Abbildung eines ResNet, mit 24 Residual-Block mit Batchnorm, ReLU, Convolutional-Layers und Skip-Connections

Dieser Ansatz war sehr erfolgreich: ResNet übertraf andere populäre CNN-Architekturen wie beispielsweise VGG auf dem ImageNet-Benchmark deutlich.

2.2.2.4 EfficientNet

Eine weitere in dieser Arbeit verwendete Architektur ist EfficientNet, vorgestellt von Tan et al. im Jahr 2020 [12]. Also eine wesentlich neuere Architektur.

EfficientNet ist kein ResNet, sondern ein eigenständiges CNN, das auf eine effiziente Skalierung von Modellen abzielt.

Vor der Einführung von EfficientNet wurden Modelle, die vergrößert werden sollten, üblicherweise in einer einzigen Dimension skaliert. Entweder in der Tiefe, seltener in der Eingaberesolution oder der Breite.

Tan et al. schlugen hingegen vor, Modelle gleichzeitig in allen drei Dimensionen zu vergrößern: Tiefe, Breite und Auflösung, und zwar anhand fester Skalierungskoeffizienten.

Dabei wird ein vom Nutzer definierter Skalierungsfaktor ϕ verwendet, der angibt, wie viel zusätzliche Rechenleistung zur Verfügung steht. Die drei Modellparameter sollen gemäß den folgenden Formeln skaliert werden:

$$\text{Tiefe} = \alpha^\phi, \text{Breite} = \beta^\phi, \text{Auflösung} = \gamma^\phi$$

Das Finden geeigneter Werte für α , β und γ wird als Optimierungsproblem behandelt. In ihrer Arbeit bestimmten Tan et al. die Parameter mittels Grid Search als:

$$\alpha = 1.2, \quad \beta = 1.1, \quad \gamma = 1.15$$

Dabei wurde die Nebenbedingung eingeführt, dass gilt:

$$\alpha \cdot \beta^2 \cdot \gamma^2 \approx 2$$

Diese Einschränkung sorgt dafür, dass sich die FLOPs (Floating Point Operations) bei jeder Erhöhung von ϕ um den Faktor 2^ϕ anwachsen.

In einem ersten Schritt wurde auf Basis eines kleinen Modells, EfficientNet-B0 eine geeignete Kombination der Parameter α , β und γ ermittelt. Anschließend wurde dieses Basismodell nach dem beschriebenen Prinzip hochskaliert, um die Modelle EfficientNet-B1 bis B7 zu erzeugen.

Auch dieser Ansatz war sehr erfolgreich: Abbildung 8 zeigt, wie EfficientNet mit vergleichsweise wenigen Parametern sehr hohe Genauigkeiten erreicht.

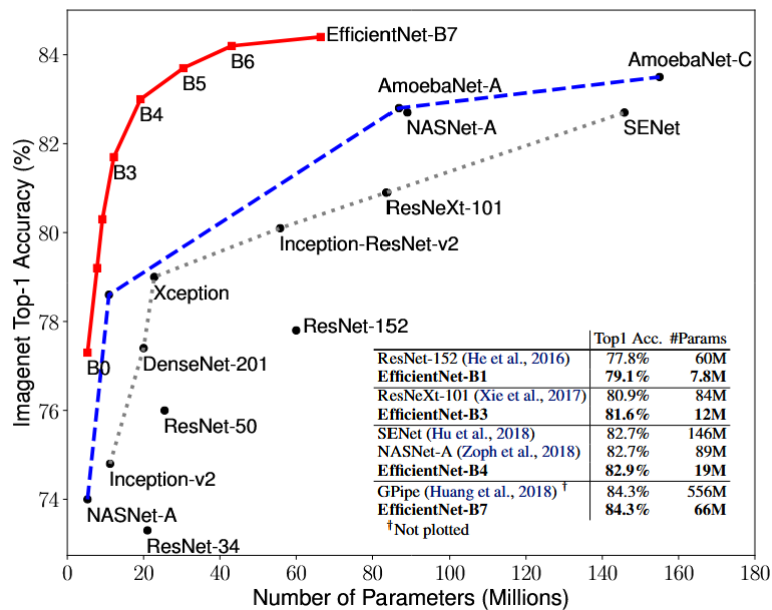


Abbildung 8: Performance-Vergleich von EfficientNet mit anderen Bildklassifikatoren auf ImageNet. Übernommen aus[12]

2.2.2.5 Swin-Transformer

Swin steht für Shifted Window. Swin-Transformer wurden 2021 von Liu et al. vorgestellt und gehören zur Klasse der Vision Transformer[13]. Sie führen mehrere Konzepte ein, die an CNNs erinnern, die Verwendung von lokalen Attention-Window, die funktional an Kernels erinnern, sowie eine hierarchische Struktur, die dem Konzept des Receptive Fields ähnelt.

Das Receptive Field beschreibt den Bereich des Eingabebildes, der Einfluss auf ein Neuron in einem tieferen Layer hat. Dieses Feld wächst mit zunehmender Tiefe im Netzwerk.

Beispielsweise hat bei einem 1D-Kernel der Größe 2 ein Neuron im ersten Layer Zugriff auf zwei Eingabewerte. Ein Neuron im zweiten Layer aggregiert wiederum zwei Neuronen aus dem ersten Layer, die jeweils auf zwei Eingabewerte zugreifen. Bei einem Stride und einer Dilation von jeweils 1 ergibt sich ein Receptive Field von 3.

Swin-Transformer nutzen eine hierarchische Struktur ähnlich wie CNNs, wodurch Repräsentationen mit zunehmendem „Receptive Field“ entstehen. Diese Struktur erlaubt es, mit geringem Rechenaufwand auch hochauflösende Bilder zu verarbeiten, da Attention nicht global, sondern lokal innerhalb verschiebbarer Fenster, den Attention-Windows berechnet wird.

Abbildung 9 zeigt den Aufbau eines Swin-Transformers, in dem mehrere Transformer-Blöcke zwischen sogenannten Patch-Merging-Blöcken angeordnet sind. Diese erzeugen die hierarchische Repräsentation.

Die Eingabebilder werden zunächst in 4×4 -Patches unterteilt, welche, wie im Standard-ViT linearisiert und eingebettet werden. Die resultierenden Token-Vektoren werden von einem Swin-Transformer-Block verarbeitet, der eine modifizierte Version der Eingabetoken zurückgibt, wobei Anzahl und Länge der Token-Vektoren erhalten bleiben.

Anschließend werden jeweils vier benachbarte Patches (2×2) durch Konkatination ihrer Token-Vektoren zusammengeführt und in einem „Dense Layer“ auf die Hälfte der Gesamtlänge komprimiert. Damit wird die Länge der Token also verdoppelt.

Die Länge des Token-Vektors wird in Abbildung 9 als C angegeben. Durch das Mergen wurden 4 Patches der Grösse C auf einen Patch der Grösse $2C$ verdichtet.

Die Anzahl Patches hat sich nach einem solchen Schritt also verkleinert aber, jeder Patch greift, ähnlich wie ein Neuron beim Receptive Field nun auf einen grösseren Bereich des Bildes zu

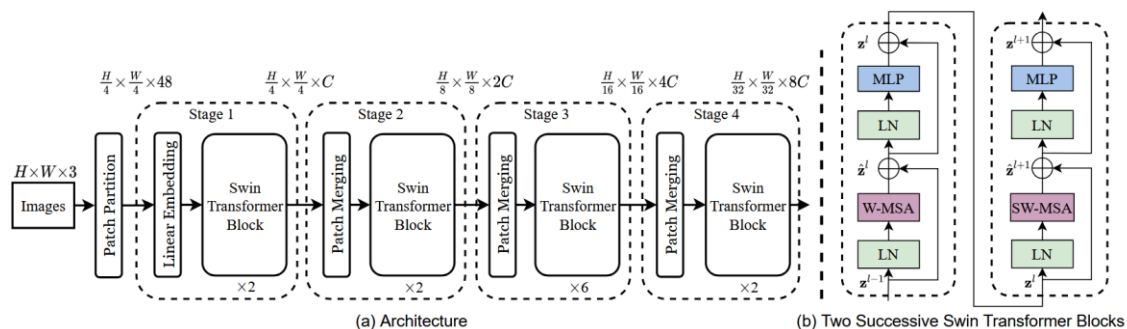


Abbildung 9: Architektur eines Swin-Transformers und Aufbau zweier aufeinanderfolgender Transformerblocks mit Window-Shift. Übernommen aus[13]

Die zweite Abweichung vom Standard-ViT betrifft den Aufbau des Swin-Transformer-Blocks selbst. Dieser verwendet ein sogenanntes Attention Window.

Im Standard Vision Transformer wird Self-Attention global berechnet: Jede Query wird mit allen Keys verglichen. Das bedeutet, jeder Patch interagiert mit allen anderen Patches im Bild.

Im Gegensatz dazu berechnet der Swin Transformer die Self-Attention nur lokal innerhalb eines festen Fensters, dem Attention Window. Das bedeutet, dass eine Query (bzw. ein Patch-Token) nur mit den benachbarten Tokens innerhalb desselben Fensters interagiert. Die Größe dieses Fensters ist durch die Window-Größe definiert.

Dadurch bleibt der Rechenaufwand linear in Bezug auf die Anzahl der Patches, multipliziert mit der Window-Größe, im Gegensatz zum quadratischen Aufwand im globalen Self-Attention-Mechanismus.

Nach jedem Swin-Transformer-Block wird das Fenster zyklisch verschoben – wie in Abbildung 10 dargestellt, sodass Patches benachbarte Patches, die in einem Layer L in unterschiedlichen Fenstern lagen und nicht miteinander interagieren konnten, im nächsten Layer in denselben Fenstern liegen und kommunizieren können

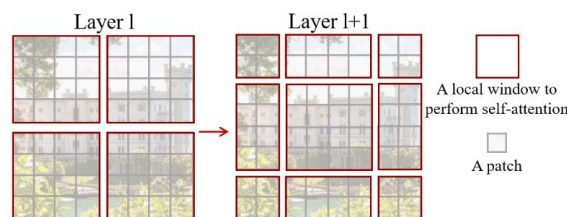


Abbildung 10: Im Layer l (links) wird eine reguläre Fensteraufteilung verwendet, und die Self-Attention wird innerhalb jedes Fensters berechnet. Im nächsten Layer $l + 1$ (rechts) wird die Fensteraufteilung **verschoben**, wodurch neue Fenster entstehen. Die Self-Attention-Berechnung in diesen neuen Fenstern überschreitet die Grenzen der vorherigen Fenster aus Layer l und ermöglicht dadurch Verbindungen zwischen den Fenstern. Übernommen aus [13]

Zusammenfassend: Swin-Transformer reduzieren Rechenaufwand mit Attention Windows die sich, in jedem transformerblock verschieben und bauen eine Art Zugriffshierarchie, in der Tokens in einem tieferen Layer Zugriff auf einen grösseren Ausschnitt des Bildes haben, als Tokens in einem der vorderen Layer.

3 Vorgehen und Methode

Für den Aufbau des Experiments wird folgende Vorgehensweise angewendet: Für die zu Testenden Image-Classification Architekturen werden verschiedene Trainingskomponenten wie Optimizer, Lernraten-Scheduler und Methoden zur Datenvorverarbeitung systematisch erprobt.

Zusätzlich erfolgt eine Hyperparametersuche, um die bestmögliche Modellleistung zu erzielen.

Getestet werden die Modelle ResNet, EfficientNet und der Swin-Transformer.

Die variablen Komponenten sind der Optimizer, der Lernraten-Scheduler, verschiedene Normalisierungsmethoden, die Modellgrösse sowie Strategien zur Vermeidung von Overfitting, spezifisch L1- und L2-Regularisierung sowie die Dropout-Rate.

Auf den Aufbau der einzelnen Experimente sowie die zu optimierenden Hyperparameter wird in Unterkapitel 3.3 näher eingegangen.

3.1 Komponenten und Hyperparameter

3.1.1 Inputdaten und Preprocessing

3.1.1.1 Radio Galaxy Zoo und verwendeter Datensatz

Die Grundlage dieser Arbeit ist der das RGZ-DR1 Subset RGZ-OD (Radio-Galaxy-Zoo-Object-Detection) von ClaRAN[2]. Abbildung 11 zeigt einige Beispielbilder desselben.

Es besteht aus Klassifizierungen, welche über ein Konsenslevel grösser als 0.6 und weniger als 4 Komponenten oder Peaks verfügen. Dies liegt an der Verteilung des Datensatzes. Es gibt nur sehr wenige Beispiele von Galaxien mit vielen Komponenten oder Peaks. Siehe Anhang 1 zum Vergleich.

Die Quellen werden nach dem folgenden Schema benannt: C_P, wobei C die Anzahl Komponenten und P die Anzahl Peaks aufzählt. Die Notation von C_P werden im folgenden auch als Zahl_Zahl geführt.

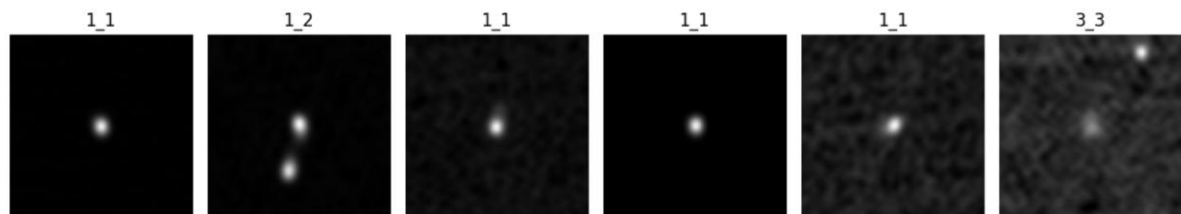


Abbildung 11: Beispiele von verschiedenen Radioquellen

Mit diesen Auswahlkriterien bleibt ein Datensatz von 10.744 RGZ-Objekten.

Die meisten Quellen mit einer Komponente verfügen aufgrund ihrer relativen Einfachheit über hohe Konsenslevel. Zum Beispiel haben 1133 von 1412 1C_3P-Quellen einen Konsenslevel von 1,0. 1C_2P hat einen etwas niedrigeren Mediankonsenslevel (0,98) als 1C_1P oder 1C_3P. Multikomponenten-/Peak-Quellen hingegen haben allgemein deutlich niedrigere Konsenslevel (Consensus Level CL) Werte. So liegen die meisten Konsenslevel von 2C_2P und 2C_3P zwischen 0,69 und 0,85, mit einem Median von 0,76. Die CL-Werte von 3C_3P-Quellen haben einen ähnlichen Median von 0,73 und eine Verteilung zwischen 0,66 und 0,81. Es wird grundsätzlich schwieriger, einen Konsens zu erreichen, je komplexer die Struktur bei Mehrkomponentenquellen wird.

Die Verteilung des RGZ-OD Datensatz verhält sich wie in Abbildung 12 und Tabelle 1 dargestellt.

1C_1P	1C_2P	1C_3P	2C_2P	2C_3P	3C_3P
5300	1331	1412	1251	1208	1334

Tabelle 1: Verteilung der RGZ-OD Datensatzes

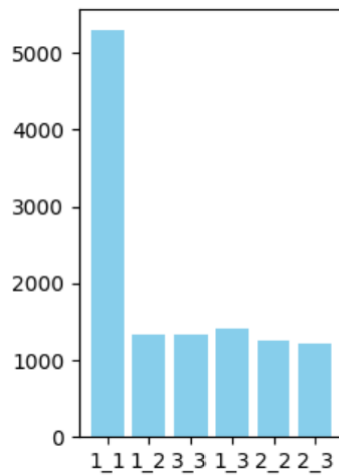


Abbildung 12: Verteilung des RGZ-OD Datensatz

Der RGZ-OD Datensatz wurde für diese Arbeit auf zwei Arten verändert.

Er verfügt über einige wenige Bilder, die mehrere Quellen beinhalten. Diese Bilder sind zwar für Object Detection geeignet, jedoch nicht für die Klassifizierung, die das Ziel dieser Arbeit ist. Diese Bilder wurden aus dem Datensatz entfernt.

Die Anzahl Bilder aller Klassen ausser 1_1 wurden durch eine um 90 Grad rotierte Kopie jedes Bildes verdoppelt. Die Klasse 1_2, die nach dem Entfernen der Bilder mit mehreren Quellen sehr klein war, wurde zusätzlich durch Spiegeln, und das erneute Rotieren des gespiegelten Bildes vervierfacht. Die neue Verteilung des Datensatzes ist, wie in Abbildung 13 dargestellt, gleichmässiger.

1C_1P	1C_2P	1C_3P	2C_2P	2C_3P	3C_3P
3845	3176	2722	2648	2398	2340

Tabelle 2: Verteilung des Augmentierten Datensatzes

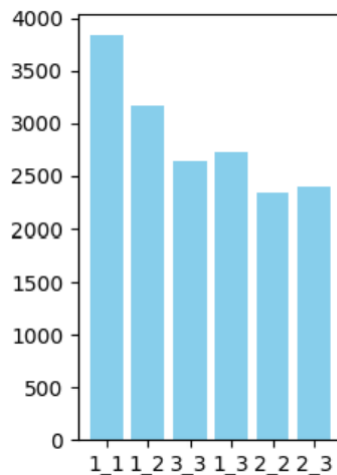


Abbildung 13: Verteilung des verwendeten Datensatzes.

Vor der Data Augmentation bestand der Datensatz aus FITS-Files sowie XML-Files, in denen die Annotationen im Pascal VOC-Format gespeichert waren.

FITS (Flexible Image Transport System) ist ein gängiges Dateiformat zur Speicherung astronomischer Bilddaten, während Pascal VOC ein etabliertes Format für Object Detection Annotationen ist.

Für die Weiterverarbeitung mit PyTorch-basierten Tools und Libraries ist jedoch das COCO-Format deutlich verbreiteter und unterstützt unter anderem komplexere Annotationen wie segmentierte Objekte.

Deshalb wurden die Annotationen beim Aufbereiten des Datensatzes in das COCO-Format konvertiert und in einem einzigen JSON-File zusammengefasst.

3.1.1.2 Normalisierung der Bilder

Die Normalisierung ist ein wesentlicher Schritt bei der Vorverarbeitung von Daten für Machine-Learning und eine Technik zur Merkmalsskalierung (Feature Scaling). Dabei werden die Input-Daten in der Regel auf einen gemeinsamen Wertebereich skaliert. Dies sorgt für eine bessere Vergleichbarkeit einzelner Inputs und führt zu stabilerer Konvergenz des Gradient-Descent-Verfahrens, da sich die Kostenfunktion in einem symmetrischeren Raum bewegt und die Gradienten dadurch gleichmässiger und effizienter berechnet werden können. Ohne Normalisierung kann es passieren, dass Merkmale mit grösseren Wertebereichen die Optimierung dominieren, was zu langsamem oder fehlerhaftem Lernen führt.

Zusätzlich ist für diese Arbeit relevant, dass mit vortrainierten Modellen gearbeitet wurde. Da diese mit normalisierten Daten trainiert wurden wird angenommen, dass sie mit ähnlich normalisierten Daten die besten Ergebnisse erzielen.

Die verwendeten Modelle, ResNet, EfficientNet und Swin sind jeweils auf IMAGENET1K_V1 vortrainiert. Darum wird folgendes Preprocessing als optimal erwartet:

Die Bilder werden erst auf einen Wertebereich zwischen 0 und 1 skaliert. Dann werden optional mittels Z-Score Normalisierung die einzelnen Farbkanäle erneut auf die Norm der Bilder angepasst mit welchem das Model trainiert wurde.

Die Z-Score Normalisierung funktioniert nach folgender Formel:

$x = \frac{x - \mu}{\sigma}$, wobei μ der Mittelwert und σ die Standardabweichung sind:

	Mittelwert	Standardabweichung
Red	0.485	0.229
Green	0.456	0.224
Blue	0.406	0.225

Tabelle 3: Mittelwert und Standardabweichung der Z-Score-Normalisierung

Die Werte werden, falls der ursprüngliche Wertebereich zwischen 0 und 1 liegt, so neu auf einen Wertebereich von etwa -2.25 bis 2.25 projiziert. Dies ist von der Form der Daten des Pretrainings abhängig und für alle Modelle gleich. Diese Z-score Normalisierung wurde nur in manchen Experimenten angewandt.

Eine erste Normalisierung, meistens auf den Wertebereich 0 und 1, wurde in allen Experimenten durchgeführt. Dabei wurden verschiedene Normalisierungen angewandt, deren Vorteile im Folgenden diskutiert werden.

In der Regel sind die nicht normalisierten Bilder in einer Form, dass die meisten Pixelwerte um einen negativen Minimalwert herum liegen, während an manchen wenigen Stellen, den Galaxien, helle Peaks auftauchen.

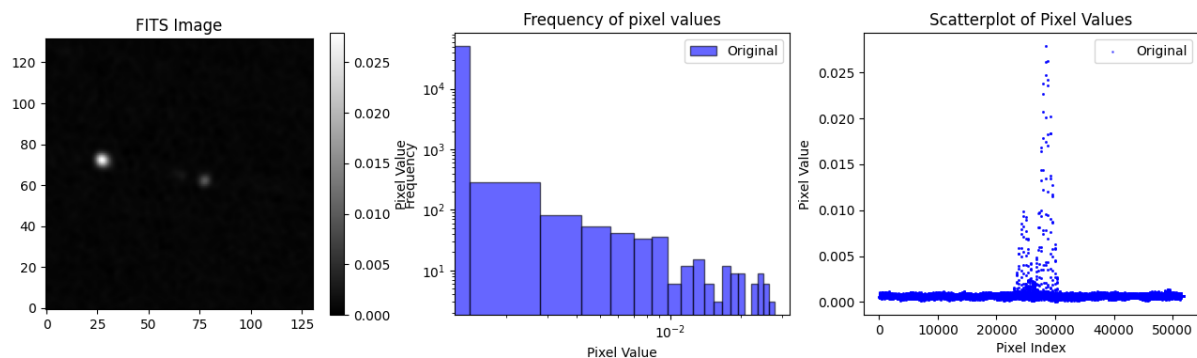


Abbildung 14: Beispiel eines nicht normalisierten Bildes. Die meisten Werte liegen um das Minimum mit zwei helleren Stellen

Die Differenzen zwischen Maximal- und Minimalwerten sind dabei sehr klein. In Abbildung 14 liegen alle Pixelwerte zwischen -0.0006571476 und 0.02722745. Diese geringe Differenz aus Minimal- und Maximalwerten gilt für die meisten Bilder aus dem Datensatz, allerdings nicht für alle. Während die Maximalwerte der meisten Bilder zwischen 0.01 und 0.05 liegen, beträgt das globale Maximum 6.864. Dasselbe gilt für minimal Werte. Die meisten Minima liegen zwischen -0.003 und -0.01, doch gibt es einige wenige Minima die zwischen -0.01 und dem globalen Minimum von -0.0375 liegen. Die

globalen Maximal- und Minimalwerte sind weit von den anderen Werten und den resultierenden Mittelwerten entfernt.

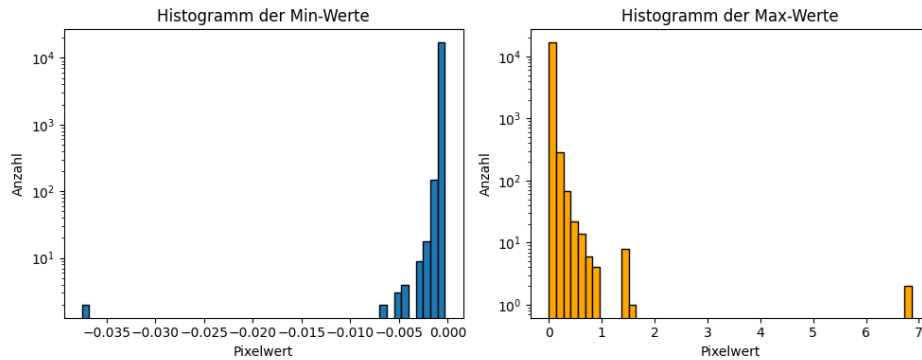


Abbildung 15: Verteilung der maximalen und minimalen Werte in allen Bildern.

Abbildung 15 zeigt die Verteilung der höchsten und niedrigsten Pixelwerte über den augmentierten Datensatz.

Min-Max

Die Min-Max-Normalisierung skaliert die Werte nach der Formel

$$x_{\text{norm}} = \frac{x - x_{\min}}{x_{\max} - x_{\min}},$$

wobei $x_{\min}$ und $x_{\max}$ jeweils den minimalen und maximalen Pixelwert innerhalb eines Bildes annehmen. Nach der Normalisierung liegt der ursprüngliche Maximalwert bei 1 und der Minimalwert bei 0.

Diese Methode zählt zu den gängigsten Verfahren der Merkmalsskalierung und wurde in dieser Arbeit insbesondere beim Aufbau der Trainingspipeline und zur Erzeugung eines Baseline-Modells verwendet. Abbildung 16 zeigt Histogramm sowie Scatterplot der Pixelwerte vor und nach der Normalisierung.

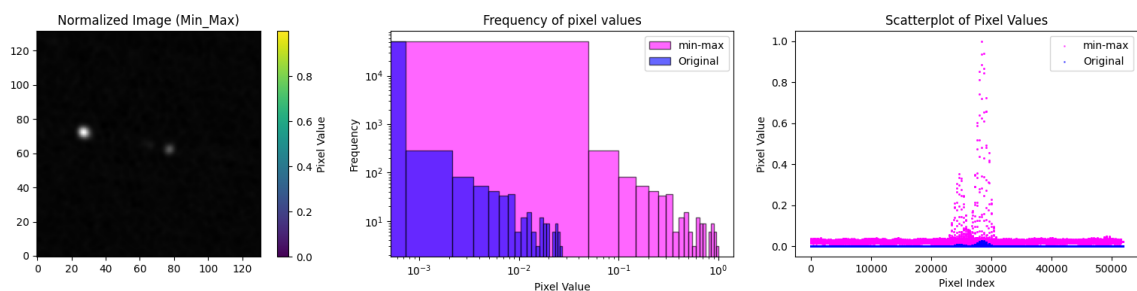


Abbildung 16: Mit Min-Max Normalisiertes Bild. Die Form der Verteilung bleibt erhalten ist aber auf einen Grösseren Wertebereich skaliert

Z-Scale-Intervall

Der Z-Scale-Algorithmus wurde ursprünglich vom National Optical Astronomy Observatory entwickelt und ist im IRAF-Framework implementiert [14]. Er wurde entworfen, um Intensitätswerte in der Nähe des Medianwertes des Bildes darzustellen. Dabei werden Randwerte ermittelt, die dann in einer Min-Max Normalisierung verwendet werden können. Dies ist besonders wertvoll für astronomische Aufnahmen, da diese typischerweise einen ausgeprägten Licht Peak, im Vergleich zum Hintergrundhimmel aufweisen.

Abbildung 17 zeigt Histogramm sowie Scatterplot der Pixelwerte vor und nach der Normalisierung.

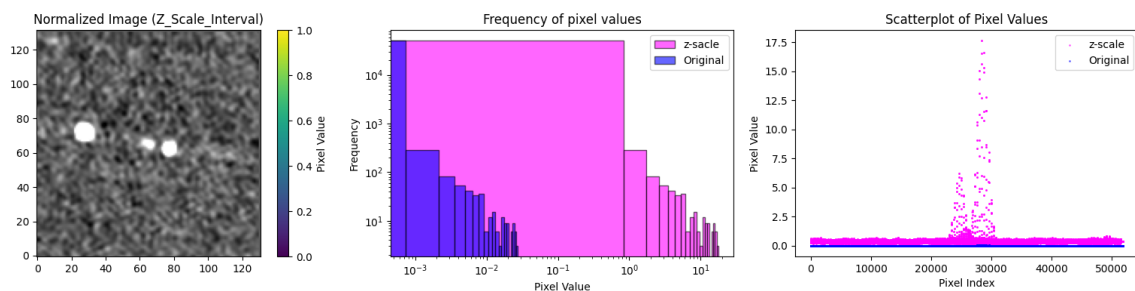


Abbildung 17: Mit dem Z-Scale-Intervall Normalisiertes Bild.

High-Energy-Physics (HEP)

Diese Normalisierung wurde von M. Drozdova et al. zur Normalisierung von Radiogalaxiebildern vorgeschlagen [15], und ist von der Analyse von Teilchenkollisionen inspiriert. Die Formel $x_{\text{norm}} = \left(\frac{x}{c}\right)^{1/\gamma}$ eignet sich gut für Signale geringer Dichte, signifikanter Bildinhalte und niedriger Signalstärke. Dabei wird γ als Hyperparameter verstanden und c der globale Maximalwert, also der höchste Wert über alle Bilder. Im zitierten Paper wird davon ausgegangen, dass die verwendeten Bilder keine negativen Werte aufweisen. Dies ist bei diesem Experiment nicht der Fall. Um negative Werte unter der Wurzel zu vermeiden, wurde ein Shift vorgenommen.

Dabei können 2 Ansätze gewählt werden. Entweder ein Shift um das lokale Minimum, so dass der dunkelste Wert neu gerade bei 0 liegt, oder ein Shift um das globale Minimum. Beide Ansätze wurden in dieser Arbeit ausprobiert. Die verwendete Formel ist also:

$x_{\text{norm}} = \left(\frac{x - x_{\min}}{c - x_{\min}} \right)^{1/\gamma}$ wobei $x_{\min}$ entweder der dunkelste Pixel im individuellen Bild oder der dunkelste Pixel im gesamten Datensatz ist.

Abbildung 18 zeigt Histogramm sowie Scatterplot der Pixelwerte vor und nach der Normalisierung.

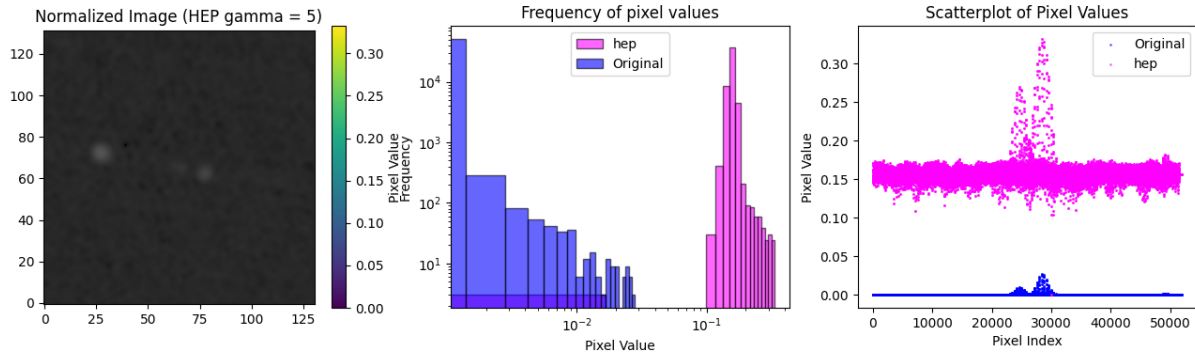


Abbildung 18: Mit HEP normalisiertes Bild. $\gamma = 5$. Es wurde ein Shift um das Lokale Minimum vorgenommen

3.1.2 Hyperparameter und veränderbare Modelkomponenten

3.1.2.1 Verlustfunktion / Loss-Funktion

Für alle Experimente wurde Cross-Entropy-Loss, eine gängige Verlustfunktion für Klassifikationsprobleme in neuronalen Netzen verwendet. Sie misst den Unterschied zwischen der vorhergesagten Wahrscheinlichkeitsverteilung des Modells und der tatsächlichen Verteilung der Klassenlabels.

Konkret wurde die Implementierung von PyTorch verwendet [16]. Die verwendete Loss-Klasse (`torch.nn.CrossEntropyLoss`) erwartet als Eingabe den direkten Output des Modells – also ein Array (Tensor) mit unnormalisierten Logits für jedes Bild im Trainingsbatch. Jeder Logit-Vektor enthält einen Wert pro Klasse, der nicht direkt eine Wahrscheinlichkeit ist, sondern erst durch Anwendung der Softmax-Funktion in Wahrscheinlichkeiten umgewandelt werden kann:

$$\text{Softmax}(x_{\text{groundTruth}}) = \frac{\exp(x_{\text{groundTruth}})}{\sum_{j=1}^C \exp(x_j)}, C = \text{Anzahl Klassen}$$

Diese Funktion transformiert die Logits in Wahrscheinlichkeitswerte, sodass sie in der Summe 1 ergeben. Anschliessend wird der Negative Log-Likelihood (NLL) der Wahrscheinlichkeit der Klasse berechnet, die der Ground Truth entspricht:

$$\mathcal{L}(x_{gT}) = -\log(\text{Softmax}(x_{gT}))$$

Das entspricht der Cross-Entropy zwischen dem Softmax-Ausgang und dem Target-Label. Der durchschnittliche Fehler (Loss) über den gesamten Batch wird dann als Mittelwert aller Einzelloss-Werte berechnet.

3.1.2.2 Optimierungsalgorithmen – Optimizer

Für EfficientNet und ResNet wurden sowohl der SGD-Optimizer als auch der Adam-Optimizer verwendet; beim Swin Transformer kam zusätzlich der AdamW-Optimizer zum Einsatz.

SGD-Optimizer basieren auf dem Prinzip des Stochastic Gradient Descent. Die grundlegende Idee besteht darin, die Gradienten nicht nach jedem einzelnen Bild, sondern über den durchschnittlichen Verlust eines Mini-Batches hinweg zu berechnen. Dadurch werden die Modellparameter seltener angepasst. Das direkte Berechnen und Anwenden der Gradienten nach jedem Bild ist zum einen rechnerisch sehr aufwendig und birgt zum anderen das Risiko, in ein lokales Minimum zu konvergieren. Das kann abhängig von den Anfangswerten, für das Training suboptimal sein kann. Das Mitteln des Losses über mehrere Bilder innerhalb eines Batches erzeugt eine gewisse Varianz, die diesem Problem entgegenwirken soll.

Konkret wurde in dieser Arbeit der SGD-Optimizer aus der PyTorch-Bibliothek verwendet, er ist für das Aktualisieren der Modellparameter zuständig.

Bei diesem Update-Prozess berücksichtigt die PyTorch-Implementierung zusätzlich die Parameter Weight Decay und Momentum. Das Aktualisieren der Parameter erfolgt dabei folgendermaßen:

Ein Parameter wird grundsätzlich wie in Formel X beschrieben aktualisiert. Dabei wird der Gradient g_t aus dem Batch-Loss berechnet.

Gibt man dem Optimizer zusätzlich einen Wert für das sogenannte Weight Decay (λ) mit, so wird der berechnete Gradient vor dem eigentlichen Parameter-Update um einen zusätzlichen Strafterm erweitert:

$$g_t \leftarrow g_t + \lambda \cdot \theta_{t-1}$$

Dabei ist g_t der Gradient und θ_{t-1} der aktuelle Wert des Parameters. Durch diesen Zusatzterm wird ein großer Parameterwert stärker bestraft, was einer L2-Regularisierung entspricht. Ziel dieser Regularisierung ist es, die Modellkomplexität zu kontrollieren und Overfitting zu vermeiden.

Beim Momentum-Verfahren geht es darum, nicht nur den aktuellen Gradienten zu berücksichtigen, sondern auch eine Art „Bewegungsrichtung“ (ein Momentum) aufzubauen. Dadurch kann das Optimierungsverfahren Beschleunigung in „gute“ Richtungen aufbauen und Schwankungen (z. B. in schmalen Tälern) reduzieren.

Es gilt: μ ist der Momentum-Faktor, b_t die momentane Richtung, τ ein Faktor, der den Einfluss des berechneten Gradienten dämpft, g_t der berechnete Gradient (inklusive Weight-Decay) und α die Lernrate. Für den ersten Schritt gilt $b_t = g_t$ da es noch keinen alten Impuls gibt. Danach gilt

$$b_t = \mu \cdot b_{t-1} + (1 - \tau) \cdot g_t \text{ und } \theta_t = \theta_{t-1} - \gamma \cdot b_t$$

Die Richtung wird durch den Gradienten aktualisiert, der Parameter anschließend durch die Richtung angepasst.

Die Grundidee von Adam ist, dass eine konstante Schrittweite das Konvergieren erschweren kann. Große Gradienten führen zu großen Schritten, was dazu führen kann, dass das Minimum ständig übersprungen wird. Kleine Gradienten hingegen können den Lernprozess unnötig verlangsamen. Um dieses Problem zu lösen, normalisiert Adam die Gradienten und führt dabei zwei separate Momentumschätzungen ein: eines für den Gradienten selbst und eines für den quadratischen Verlauf des Gradienten, der den Gradienten skaliert. Dadurch passt Adam die Schrittweite dynamisch und richtungsabhängig an.

Die Pytorch Implementierung verwendet α als Lernrate, g_t als Gradient, β_1, β_2 als Momentum-Faktoren, m_t als erstes und v_t als zweites Momentum. m_t und v_t werden als 0 initialisiert und für jeden Parameter t upgedated.

Zuerst wird der Weight-Decay angewandt, dann werden m_t und v_t berechnet:

$$m_t \leftarrow \beta_1 \cdot m_{t-1} + (1 - \beta_1) \cdot g_t$$

$$v_t \leftarrow \beta_2 \cdot v_{t-1} + (1 - \beta_2) \cdot g_t^2$$

Als nächstes wird eine Bias Korrektur durchgeführt:

$$\hat{m}_t \leftarrow \frac{m_t}{1 - \beta_1^t}$$

$$\hat{v}_t \leftarrow \frac{v_t}{1 - \beta_2^t}$$

Zuletzt werden die Parameter aktualisiert (ϵ ist ein kleiner Term, der die Division durch 0 verhindern soll):

$$\theta_t \leftarrow \theta_{t-1} - \alpha \cdot \hat{m}_t / (\sqrt{\hat{v}_t} + \epsilon)$$

ADAMW unterscheidet sich von ADAM darin, dass die Weight-Decay-Regularisierung nicht auf den Gradienten, sondern direkt auf den Parameter angewandt wird. Also vor dem Updaten wie in Formel X wird der Parameter auf folgende Weise aktualisiert:

$$\theta_t \leftarrow \theta_{t-1} - \alpha \lambda \theta_{t-1}$$

Auch hier wurde die Pytorch Implementierung verwendet.

3.1.2.3 Learning-Rate-Scheduler

ResNet und *EfficientNet* verwenden den ReduceLROnPlateau-Lernraten-Scheduler aus der PyTorch-Bibliothek. Dieser überwacht den Verlauf einer Metrik – in diesem Fall den Validierungsverlust – und reduziert die Lernrate um einen festgelegten Faktor (hier: 10),

sofern sich die Metrik über eine definierte *Patience*-Periode (hier: 10 Epochen) nicht signifikant verbessert.

Eine signifikante Verbesserung ist definiert als:

$$\text{akuteler Wert} < \text{bester Wert} * (1 - 0.0001)$$

Beim Training des Swin-Transformers wird der Cosine Annealing-Lernraten-Scheduler der Pytorch-Bibliothek verwendet. Dabei sinkt die Lernrate über die gesamte Trainingsdauer hinweg kontinuierlich entlang einer Kosinuskurve, von einer initialen Lernrate $a_{\max}$ bis zu einer definierten minimalen Lernrate $a_{\min}$.

Die Lernrate wird für jeden Schritt t gemäss folgender Formel berechnet:

$$a_t = a_{\min} + \frac{1}{2}(a_{\max} - a_{\min}) \left(1 + \cos\left(\frac{T_{\text{cur}}}{T_{\max}}\pi\right)\right)$$

Dabei ist T_{cur} die momentane Epoche und $T_{\max}$ die Anzahl Epochen.

3.1.2.4 L1-Regularisierung und Dropout

Verschiedene Strategien zur Verminderung von Overfitting wurden angewandt. Weight-Decay/L2-Regularisierung wurde schon im Unterkapitel 3.1.2.2 erklärt. L1-Regularisierung und Dropout werden in diesem Kapitel näher beschrieben.

Die L1-Regularisierung summiert die absoluten Werte aller Gewichte und fügt diesen, multipliziert mit der Regularisierungsstärke λ , der Verlustfunktion hinzu. Diese Strafkomponeute soll bewirken, dass bestimmte Gewichte auf null gesetzt werden, wodurch irrelevante Features unterdrückt und wichtige Features bevorzugt berücksichtigt werden sollen.[17]

Die L1-Regularisierung wurde selbst programmiert:

```
def l1_penalty(loss, model, l1_lambda=0.01):
    if l1_lambda == 0.0:
        return loss
    l1_norm = 0.0
    for name, param in model.named_parameters():
        if 'weight' in name and param.requires_grad:
            l1_norm += param.abs().sum()
    return loss + l1_lambda * l1_norm
```

Abbildung 19: Implementation der L1-Regularisierung

Wie in Abbildung 19 gezeigt, wird, sollte die Regularisierungsstärke null sein, zur Einsparung von Rechenaufwand der Loss direkt zurückgegeben. Andernfalls wird der Loss um die oben beschriebene Strafkomponeute ergänzt.

Beim Dropout werden in einem Layer mit einer Wahrscheinlichkeit p , der Dropout rate, die Neuronen einer Schicht „Deaktiviert“. Das heisst, dass die Weights von diesen Neuronen auf 0 gesetzt werden. Die verbleibenden aktiven Neuronen werden mit dem

Faktor $\frac{1}{1-p}$ multipliziert, damit die Gesamtsumme der Neuronenwerte ungefähr konstant bleibt.

3.1.2.5 Evaluations Metriken

Geloggt wurden die Metriken Accuracy, Precision, Recall und F1-Score. Dies erfolgte sowohl auf dem Trainings- als auch auf dem Test-Set. Die Berechnung der Kennzahlen wurde mit Funktionen aus dem Modul `sklearn.metrics` durchgeführt[18].

Im Folgenden werden die verwendeten Metriken kurz erläutert:

Accuracy bezeichnet den Anteil korrekt klassifizierter Beispiele an allen Beispielen im Datensatz.

Precision misst die Genauigkeit der positiven Vorhersagen: Sie gibt an, welcher Anteil der Instanzen, die einer bestimmten Klasse zugeordnet wurden, tatsächlich zu dieser Klasse gehört.

Recall beschreibt, welcher Anteil der tatsächlich zu einer Klasse gehörenden Instanzen korrekt erkannt wurde.

Der F1-Score ist das harmonische Mittel aus Precision und Recall. Er bietet ein ausgewogenes Maß für die Klassifikationsleistung, insbesondere wenn ein Ungleichgewicht zwischen Klassen vorliegt oder sowohl falsch positive als auch falsch negative Vorhersagen relevant sind.

3.1.2.6 Model-Heads

Für ResNet und EfficientNet wurden eigene Heads implementiert, um Batch Normalization und Dropout anwenden zu können. Der Head des ResNet-Modells besteht aus drei vollständig verbundenen Schichten (Dense Layers), wobei zwischen der innersten und der mittleren sowie zwischen der mittleren und der Ausgabeschicht jeweils Batch Normalization und Dropout eingesetzt werden. Das bedeutet, dass in der innersten und der mittleren Schicht, abhängig von der gewählten Dropout-Rate, zufällig ausgewählte Neuronen während des Trainings auf null gesetzt werden. Die Anzahl der Eingabeneuronen (`in_features`) wird dabei zwei Mal halbiert und anschließend auf sechs reduziert.

Der *EfficientNet*-Head besteht aus zwei vollständig verbundenen Schichten (*Dense Layers*) mit *Dropout*. Dabei wird die Anzahl der Eingabeneuronen zunächst halbiert und anschließend auf sechs reduziert.

Für den *Swin Transformer* wurde kein eigener Head implementiert, sondern der bereits integrierte Klassifikationskopf mit der korrekten Anzahl an Klassen verwendet.

Für diese Arbeit wurden ausschließlich Modelle verwendet, die auf dem ImageNet-Datensatz vortrainiert wurden. Dadurch kann auf bereits gelernte allgemeine

Bildmerkmale zurückgegriffen werden, die für viele visuelle Aufgaben nützlich sind. Mittels Transfer Learning werden diese Modelle anschließend auf die Klassifikation von Radiogalaxien angepasst[19].

Hierzu wird der sogenannte Backbone des Modells – also der Teil, der die Bildmerkmale extrahiert – zunächst eingefroren. Das bedeutet, dass dessen Parameter während des ersten Trainings nicht verändert werden.

Ziel ist es, die bereits gelernten Repräsentationen nicht direkt zu überschreiben und damit zu zerstören. Stattdessen wird am Ende des Modells ein neuer, auf die Zielaufgabe zugeschnittener Klassifikationskopf (Head) hinzugefügt, der trainiert wird auf Basis der vom Backbone extrahierten Feature Maps die Bildklasse vorherzusagen.

Da dieser Head in der Regel relativ einfach aufgebaut ist, neigt er weniger stark zu Overfitting als ein vollständig trainiertes Modell.

Nachdem der Head trainiert wurde, wird anschließend doch noch das gesamte Modell trainiert (finetuning). Da der Head bereits angepasst ist, erhofft man sich kleinere Gradienten und dadurch ein stabileres sowie kontrollierteres Lernen, das das Vergessen nützlicher Merkmale vermeidet.

3.1.3 Sweeps/ Logging

Für das Logging wurde die Plattform und Bibliothek Weights & Biases (wandb) verwendet. Die variablen Parameter für das Training und die Experimente wurden in einer Datei „config.yaml“ gespeichert und mit der Bibliothek OmegaConf eingebunden.

Dabei kamen stets zwei Konfigurationsdateien zum Einsatz: „config.yaml“ und „override.yaml“. Mit OmegaConf konnten diese Dateien zusammengeführt werden, sodass eine Basis-Konfiguration stets vorhanden war und durch die aktuellen Parameter überschrieben werden konnte.

3.2 Vorbereitung

3.2.1 Erstellen der Trainingspipeline

Zu Beginn der Arbeit wurde Code aus einem Classifier-Tutorial von PyTorch sowie aus einem ResNet-Tutorial von Alireza Samar verwendet, um einen minimalen Trainingsablauf zu implementieren.

Das Programm lädt das ResNet18-Modell von PyTorch mit auf ImageNet1k-v1 vortrainierten Gewichten sowie den CIFAR-10-Datensatz, einen Standarddatensatz mit zehn Klassen. Zudem wird eine *Transform*-Klasse verwendet, die die Bilder auf die Grösse der ImageNet-Bilder skaliert, in Tensoren überführt und eine Z-Score-Transformation der Pixelwerte durchführt. Der Datensatz wird in ein Trainings- und ein Validierungsset unterteilt und den *Data Loadern* übergeben, die während des Trainings

die Daten in Mini-Batches einer vorgewählten Grösse, beispielsweise 32 Bilder pro Batch, zur Verfügung stellen.

Als Nächstes wird das Verlustkriterium auf Cross Entropy Loss gesetzt.

Anschliessend werden alle Layer des Backbones des Modells eingefroren, sodass nur der Head, in diesem Fall die letzte Schicht des Netzwerks, trainiert wird. Als letzter Schritt vor dem Trainingsbeginn wird ein SGD-Optimizer mit einer gewählten Lernrate, Weight Decay sowie den nicht eingefrorenen Modellparametern sowie ein On-Plateau Lernraten-Scheduler initialisiert.

Das Modell, die *Data Loader* der Trainings- und Validierungssets, das Verlustkriterium, der Optimizer sowie die gewählte Anzahl an Epochen, für die trainiert werden soll, werden der Trainingsfunktion übergeben.

In der Trainingsfunktion werden, weitgehend wie in Kapitel 2.1 beschrieben, die einzelnen Trainingsschritte ausgeführt.

Das Modell wird in den Trainingsmodus versetzt. Zu Beginn jeder Trainingsepoche werden Epochen-Loss und Epochen-Accuracy mit dem Wert 0 initialisiert. Für jeden Mini-Batch löscht der Optimizer zu Beginn die alten Gradienten, falls bereits welche berechnet wurden.

Im Forward-Pass werden alle Bilder des Batches klassifiziert, und der Loss wird berechnet. Im anschliessenden Backward-Pass werden die Gradienten bestimmt, woraufhin der Optimizer einen Schritt durchführt, bei dem die Modellparameter aktualisiert werden.

Da in diesem Fall mit SGD optimiert wird, werden sowohl der Loss als auch die Gradienten nicht für einzelne Bilder, sondern über den gesamten Batch hinweg berechnet. Loss und Accuracy werden nach jedem Mini-Batch aktualisiert und am Ende jeder Epoche aggregiert und ausgegeben.

Nachdem über die angegebene Anzahl an Epochen trainiert wurde, werden die Parameter der zuvor eingefrorenen Schichten des Modells freigegeben (unfrozen). Anschliessend wird ein neuer Optimizer für alle Modellparameter erstellt. In einer zweiten Trainingsrunde werden sämtliche Parameter der Trainingsfunktion übergeben, woraufhin, nach demselben Ablauf wie zuvor, das gesamte Modell trainiert wird.

Dieser grundlegende Aufbau, also das Laden der Daten, das Laden eines Modells, das Einfrieren des Backbones, das Instanzieren von Optimizer und Lernraten-Scheduler, das Trainieren des Heads, das Freigeben des Backbones, das Instanzieren eines neuen Optimizers und Lernraten-Schedulers sowie das Finetuning des gesamten Modells – soll innerhalb der Trainingspipeline beibehalten werden.

In der Pipeline werden alle Komponenten und Hyperparameter in einer Config-Datei definiert (beschrieben in Kapitel 3.1.3, ein Beispiel befindet sich im Anhang). Diese Datei kann von Weights & Biases (WandB) Sweeps verwendet werden, um systematisch verschiedene Kombinationen der Hyperparameter zu testen und automatisch die besten Konfigurationen für das Training zu ermitteln.

Die Werte der Hyperparameter werden aus der Konfigurationsdatei geladen, und das Laden der jeweiligen Komponenten erfolgt über if-Anweisungen. Dadurch lässt sich ein neues Experiment sehr einfach und effizient allein durch das Überschreiben der Config-Datei aufsetzen.

```
#region Model
# set model, and put it to device (default = cuda)
if self.config.model_architecture == "EfficientNet":
    model = EfficientNet.from_pretrained(self.config.model_name)
    model = RGZ_SmallHead(model, in_features=self.config.in_features, num_classes=6, dropout_rate=self.config.dropout)
elif self.config.model_architecture == "ResNet":
    if self.config.model_name == "ResNet34":
        model = models.resnet34(weights=ResNet34_Weights.IMAGENET1K_V1)
        model = RGZ_ResNet34Head(model, in_features=512, num_classes=6, dropout_rate=0.5)
    if self.config.model_name == "ResNet50":
        model = models.resnet50(weights=ResNet50_Weights.IMAGENET1K_V1)
        model = RGZ_ResNet50Head(model, in_features=2048, num_classes=6, dropout_rate=self.config.dropout)
elif self.config.model_architecture == "Swin":
    if self.config.model_name == "tiny":
        model = models.swin_t(weights=models.Swin_T_Weights.IMAGENET1K_V1)
    elif self.config.model_name == "small":
        model = models.swin_s(weights=models.Swin_S_Weights.IMAGENET1K_V1)
    else:
        raise ValueError(f"Unknown Swin model name: {self.config.model_name}")
else:
    print("No Model specified")
self.model = model.to(self.device)
#endregion
```

Abbildung 20: Code-Snippet das aufzeigt, wie ein Model geladen wird.

Abbildung 20 zeigt, wie basierend auf den Angaben im Config-File ein Modell für das Training geladen wird. Der Parameter architecture legt fest, welche Art von Architektur verwendet werden soll (z. B. EfficientNet, ResNet oder Swin Transformer), während der Parameter model_name bestimmt, welche Variante bzw. Tiefe des Modells ausgewählt wird (z. B. ResNet34 oder Swin-Tiny). Je nach Auswahl werden passende vortrainierte Gewichte geladen und ein entsprechender Head für die Klassifikation mit sechs Klassen ergänzt.

3.2.2 Vorbereiten des Datasets

Die Experimente werden natürlich nicht mit dem CIFAR-10-Datensatz durchgeführt, sondern mit dem in Kapitel 3.1.1.1 beschriebenen augmentierten RGZ-OD-Datensatz. Dieser muss zunächst geladen und geeigneten Vorverarbeitungsschritten unterzogen werden. Dazu wird die PyTorch-Klasse torch.utils.data.Dataset überschrieben (vererbt), um ein eigenes Dataset-Objekt zu definieren, das sowohl die Bilddaten aus FITS-Dateien lädt als auch die zugehörigen COCO-kompatiblen Annotationen interpretieren kann.

Beim Initialisieren des Datasets wird die COCO-Annotationsdatei geladen. Daraus wird eine Zuordnung von Bild-IDs zu den Dateinamen erstellt. Die Annotationen werden

anschliessend iterativ durchlaufen, wobei für jedes Bild die zugehörige Kategorie-ID, welche die Galaxienklasse angibt, sowie der Pfad zur FITS-Datei bestimmt werden. Die FITS-Dateien werden mit Hilfe der Bibliothek `astropy.io.fits` geöffnet und die enthaltenen Bilddaten extrahiert. Dabei werden potenzielle NaN- oder unendliche Werte mit festen Werten ersetzt, um die Robustheit der Verarbeitung zu gewährleisten (mithilfe von `np.nan_to_num`). Gleichzeitig werden während des Einlesens die minimalen und maximalen Pixelwerte jedes Bildes erfasst. Diese sollen später bei Bedarf für die HEP-Normalisierung zur Verfügung stehen.

Die Annotationen (bestehend aus Bild-ID und Kategorie-ID) werden in einer Liste gespeichert, und zusätzlich wird ein Mapping aufgebaut, das die numerischen Kategorie-IDs den jeweiligen Klassenbezeichnungen zuordnet.

Die Funktion `__getitem__`, die später vom DataLoader aufgerufen wird, gibt ein Bild sowie dessen Label, also dessen Kategorie, zurück. Bevor das Bild als PIL-Bild zurückgegeben wird, wird die Normalisierung durchgeführt.

Welche Normalisierung angewendet wird, sowie der γ -Wert für die HEP-Transformation (falls nötig, siehe Formel X), ist im Config-File festgelegt. Die konkrete Umsetzung erfolgt über verschachtelte if-Statements.

1) Normierung vor der Erweiterung auf drei Kanäle:

Die Bilder werden zunächst normalisiert und anschliessend, durch Duplikation des normierten Einzelkanals, auf drei Kanäle erweitert. In diesem Fall kann eine der folgenden Normalisierungsmethoden angewendet werden:

- HEP-Normalisierung (mit festgelegtem γ -Wert)
- Min-Max-Normalisierung (wurde nur beim Erstellen der Pipeline verwendet und in ersten Trainings runs verwendet)
- Z-Scale-Intervall-Normalisierung

2) Erweiterung auf drei Kanäle vor der Normalisierung:

Die Bilder werden zunächst auf drei Kanäle erweitert. Anschliessend wird jeder Kanal separat mit einer eigenen Normalisierungsmethode bearbeitet.

Dabei sind folgende Kombinationen möglich:

- HEP-Normalisierung auf allen Kanälen, jeweils mit unterschiedlichen γ -Werten
- Kombination aus Z-Scale-Intervall-Normalisierung auf einem Kanal und HEP-Normalisierung auf den anderen beiden (mit gleichen oder unterschiedlichen γ -Werten)

- Z-Intervall-Normalisierung auf zwei Kanälen und HEP-Normalisierung auf dem dritten Kanal
- Eine Kombination aus HEP-Normalisierung auf zwei Kanälen und Min-Max auf einem
- Eine Kombination aus HEP-, Z-Scale-Intervall- und Min-Max-Normalisierung, auf je einem Kanal.

3.3 Modelle

Bei allen Modellen wurden zunächst geeignete Hyperparameter und Komponenten ermittelt. Anschließend wurden mit diesen Einstellungen sämtliche Preprocessing-Methoden als Hyperparameter getestet. Die einzelnen Setups der Experimente sind in Tabellen dokumentiert.

Aufgrund der begrenzten Datensatzgrösse wurde auf den Einsatz großer Modelle verzichtet. Stattdessen kamen bewusst kleine bis mittlere Architekturen zum Einsatz, da diese eine geringere Tendenz zum Overfitting aufweisen.

Die folgenden Konstanten und Hyperparameter blieben über alle Experimente hinweg unverändert, da die Suche nach geeigneten Hyperparametern und Komponenten bereits sehr umfangreich war und von einer weiteren Optimierung dieser im Vergleich zu anderen Hyperparametern nur geringe Verbesserungen zu erwarten waren.

Parameter	ϵ	Batch-Size	Momentum	Betas
Beschreibung	Eine kleine Konstante im Optimizer, die beim Aktualisieren der Modellparameter eine Division durch 0 verhindert.	Die Größe der Mini-Batches, über die der Loss pro Trainingsschritt berechnet wurde. Ein Wert von 32 ist ein gängiger Standard in vielen Papern und Framework-Defaults wie z. B. PyTorch.	Falls der SGD-Optimizer verwendet wurde, kam durchgängig ein Default-Wert für das Momentum zum Einsatz. Es wurde sich an Pytorch-Tutorials orientiert.	Für die Optimizer Adam und AdamW wurden wie beim SGD durchgehend die gleichen Beta-Werte verwendet. Auch hier wurde sich an Pytorch-Tutorials orientiert.
Wert	1e-9	32	0.91	$\beta_1 = 0.9$ $\beta_2 = 0.999$

Tabelle 4: Fixe Konstanten und Hyperparameter

Für das Training wurde in allen Fällen – mit Ausnahme von EfficientNet-B3 – eine Bildauflösung von 224×224 Pixeln verwendet, was der Standardauflösung von ImageNet-

1K entspricht. EfficientNet-B3 hingegen wurde mit einer Auflösung von 300×300 Pixeln trainiert, gemäß dem Compound Scaling-Prinzip dieser Architektur.

3.3.1 ResNet

Alle ResNet Modelle die verwendet wurden, stammen aus der Pytorch-Bibliothek[16]. Zur Erstellung der Pipeline wurde zunächst ein ResNet18-Modell trainiert. Mit diesem Modell wurden jedoch nur geringe Validierungsgenauigkeiten von 66-76% erreicht. Da auch Overfitting bereits als Problem zu erkennen war, wurde die Entscheidung getroffen mit zwei größeren Modellen fortzufahren.

Dementsprechend wurden Sweeps für ResNet50 und ResNet34 durchgeführt, jeweils mit entweder L1-Regularisierung, Weight Decay oder einer Kombination aus Weight Decay und Dropout.

In der initialen Experimentierphase erfolgten einzelne Arbeitsschritte ohne durchgängig standardisierte Methodik, was sich unter anderem in der gleichzeitigen Anpassung mehrerer Hyperparameter und Modellkomponenten äußerte. Zudem wurde ein Implementationsfehler in der HEP-Normalisierung erst zu einem späteren Zeitpunkt identifiziert. Aus diesen Gründen wurden die Experimente mit ResNet34 und ResNet50 erneut durchgeführt. Dabei konnten bereits gewonnene Erkenntnisse aus dem Training mit EfficientNet berücksichtigt werden.

Aus diesem Grund wurde die Suche nach dem besten Modell nicht wie bei EfficientNet mit der Z-Scale-Intervall-Normalisierung durchgeführt, sondern mit der HEP-Normalisierung mit $\gamma = 5$ sowie einem Shift um das lokale Minimum, also den kleinsten Wert des Bildes. Diese Vorgehensweise hatte sich beim Training mit EfficientNet als robuste Prescaling-Methode erwiesen.

3.3.1.1 Erstes ResNet Experiment

In einem ersten Experiment wurden vier Hyperparameter-Sweeps mit ResNet34 und elf Sweeps mit ResNet50 durchgeführt. Da sich ResNet50 bereits nach den ersten vier Sweeps eindeutig als das besser geeignete Modell herausstellte, wurde auf weitere Versuche mit ResNet34 verzichtet.

Jeder Sweep bestand aus sechs Trainingsläufen, wobei mittels Bayesian-Optimisation geeignete Hyperparameter identifiziert wurden.

Untersucht wurden dabei die folgenden Parameter und Komponenten:

Für die Start-Lernrate wurde in jedem Sweep, unabhängig von den übrigen Parametern – stets im Intervall zwischen 0.005 und 0.0001 gesucht. Ein Intervall, der sich während des Setups als sinnvoll ergeben hatte. Als Learning-Rate-Scheduler kam der Reduce-on-Plateau-Scheduler zum Einsatz. Der Modell-Head wurde über 15 Epochen trainiert, das

gesamte Modell anschließend über 35 Epochen feinjustiert. Für alle Sweeps wurde, wie in Kapitel 3.1.1.2 beschrieben ein Z-Score Skalierung durchgeführt.

Modell	Optimizer	Regularisierung	Regularisierungsparameter: (Weight Decay/L1- λ)	Dropout-rate
ResNet34	SGD	Keine	-	0
ResNet34	ADAM	Keine	-	0
ResNet50	SGD	Keine	-	0
ResNet50	ADAM	Keine	-	0
ResNet34	SGD	Weight-Decay	Min:0.01 Max:0.001	0
ResNet34	ADAM	Weight-Decay	Min:0.01 Max:0.001	0
ResNet50	SGD	Weight-Decay	Min:0.01 Max:0.001	0
ResNet50	ADAM	Weight-Decay	Min:0.01 Max:0.001	0

Tabelle 5: Versuchsaufbau mit ResNet34 und ResNet50

Nach diesen vier Experimenten wurde ausschließlich mit ResNet50 weitergearbeitet. Zunächst wurde ein geringerer Weight-Decay-Wertebereich getestet. Anschließend wurde zusätzlich zum Weight Decay ein Dropout eingeführt, und schließlich, anstelle von Weight Decay und Dropout eine L1-Regularisierung erprobt.

Modell	Optimizer	Regularisierung	Regularisierungsparameter: (Weight Decay/L1- λ)	Dropout-rate
ResNet50	SGD	Optimizer	Min:0.005 Max:0.00001	0
ResNet50	ADAM	Weight-Decay	Min:0.005 Max:0.00001	0
ResNet50	SGD	Weight-Decay	Min:0.005 Max:0.00001	0.3 – 0.5
ResNet50	ADAM	Weight-Decay	Min:0.005 Max:0.00001	0.3 – 0.5
ResNet50	SGD	L1	Min:0.001 Max:0.00001	0
ResNet50	ADAM	L1	Min:0.001 Max:0.00001	0
ResNet50	SGD	L1	Min:0.001 Max:0.00001	0.3 – 0.5

Tabelle 6: Versuchsaufbau mit nur ResNet50

3.3.1.2 Zweites ResNet Experiment

Aus den Ergebnissen des ersten Experiments (siehe Kapitel 4.1.1) ergab sich, dass insbesondere der SGD-Scheduler in Kombination mit Weight Decay als Regularisierungsverfahren und einer Start-Lernrate von etwa 0,001 konsistent gute Resultate lieferte. Daher wurde die Suche über die Preprocessing-Methoden mit den folgenden Hyperparametern durchgeführt:

Als Modell wurde ResNet50 genutzt. Trainiert wurde der Head über 15 Epochen und das gesamte Modell über 35 Epochen.

Lernrate	Optimizer	LR-Scheduler	Regularisierung	Regularisierungsparameter:	Dropout-rate
0.001	SGD	On-Plateau	Weight-Decay	0.002	0

Tabelle 7: Model Setup für zweites ResNet-Experiment

Folgende Preprocessing-Methoden wurden getestet:

Die Reihenfolge gibt dabei an, ob zunächst eine Normalisierung und anschließend eine Erweiterung auf drei Kanäle vorgenommen wurde ($N \rightarrow E$) oder ob zuerst eine Erweiterung durchgeführt und anschließend alle Kanäle einzeln normalisiert wurden ($E \rightarrow N$).

Die Angabe „Z-Score“ kennzeichnet, ob die einzelnen Kanäle nach der Normalisierung zusätzlich, wie in Kapitel 3.1.1.2 beschrieben, auf die durch das Pretraining erlernten Wertebereiche skaliert wurden.

In der Tabelle werden die Kanäle mit „Ch“ abgekürzt.

Reihenfolge	Normalisierungsmethode	γ (Falls HEP)	Name im Code	Z-Score
N -> E	Ch1-3: Z-Scale-Intervall	-	z_scale_min_max	Ja
N -> E	Ch1-3: Min-Max	-	min_max	Ja
N -> E	Ch1-3: Hep	$\gamma = [1, 3, 5, 10, 20]$	hep	Ja
E -> N	Ch1: Hep Ch2: Hep Ch3: Hep	$\gamma_1 = [2]$ $\gamma_2 = [2, 3, 5]$ $\gamma_3 = [3, 5, 20, 30]$	hep	Ja
E -> N	Ch1: Hep Ch2: Hep Ch3: Z-Scale-Intervall	$\gamma_1 = [3, 5,]$ $\gamma_2 = [5, 10, 30]$ $\gamma_3 = [0]$	hhz	Ja
E -> N	Ch1: Hep Ch2: Z-Scale-Intervall Ch3: Z-Scale-Intervall	$\gamma_1 = [3, 5, 10, 20, 30]$ $\gamma_2 = [0]$ $\gamma_3 = [0]$	hzz	Ja

E -> N	Ch1: Min-Max Ch2: Hep Ch3: Hep	$\gamma_1 = [0]$ $\gamma_2 = [3, 5, 10]$ $\gamma_3 = [5, 10, 20]$	hep_min_max	Ja
E -> N	Ch1: Min-Max Ch2: Z-Scale-Intervall Ch3: Hep	$\gamma_1 = [0]$ $\gamma_2 = [0]$ $\gamma_3 = [3, 5, 10, 20]$	hep_min_zscale	Ja

Tabelle 8: Zu testende Preprocessingmethoden mit Z-Score

Die letzten beiden Runs wurden mit demselben Setup durchgeführt, jedoch wurde nur das Finetuning auf 25 Epochen beschränkt, da das in Quasi allen Training Runs die Validation Accuracy dort zu stagnieren scheint:

E -> N	Ch1: Hep Ch2: Hep Ch3: Z-Scale-Intervall	$\gamma_1 = [3, 5]$ $\gamma_2 = [5, 10, 30]$ $\gamma_3 = [0]$	Nein
E -> N	Ch1: Min-Max Ch2: Z-Scale-Intervall Ch3: Hep	$\gamma_1 = [0]$ $\gamma_2 = [0]$ $\gamma_3 = [3, 5, 20]$	Nein

Tabelle 9: Zu Testende Preprocessingmethoden mit Z-Score

3.3.2 EfficientNet

Für die EfficientNet-Modelle wurden Implementierungen aus dem Paket `pytorch_efficientNet` verwendet [20].

Getestet wurden die Modellgrößen EfficientNet-B0, das kleinste Modell, und EfficientNet-B3, ein mittelkleines Modell. Dabei stellte sich EfficientNet-B3 frühzeitig als das bessere Modell heraus, weshalb dieses für einen Großteil der Versuche gewählt wurde.

Dabei ist zu beachten, dass sich aufgrund der speziellen Architektur von EfficientNet die erforderliche Eingabebildgröße in Abhängigkeit von der Modellgröße verändert.

EfficientNet-B0 erfordert eine Auflösung von 224 x 224 Pixeln und EfficientNet-B3 eine Auflösung von 300x300. Diese Anforderungen wurden über alle Versuche korrekt eingehalten

Auch in diesem Fall wurde zunächst das am besten geeignete Modell identifiziert und anschließend mit diesem alle Preprocessing-Methoden getestet.

3.3.2.1 Erstes EfficientNet Experiment

In einem ersten Versuch wurde ein Hyperparameter-Sweep mit EfficientNet-B0 und drei Sweeps mit EfficientNet-B3 durchgeführt. Da die ersten beiden Sweeps bereits mit L1-

Regularisierung durchgeführt worden waren, wurden sie der Vollständigkeit halber später nochmals ohne Regularisierung wiederholt.

Wie bei den ResNet-Experimenten wurde auch hier erneut die in Experiment 3.3.2.2 ermittelte HEP-Normalisierung mit $\gamma = 5$ und einem Shift auf das lokale Minimum angewendet.

In beiden Sweep-Durchläufen erzielte EfficientNet-B3 eine signifikant höhere Validierungsgenauigkeit als EfficientNet-B0, weshalb in den weiteren Experimenten ausschließlich mit EfficientNet-B3 gearbeitet wurde. Die Resultate sind im Kapitel 4.2.1 nachzuvollziehen.

Jeder Sweep bestand aus sechs Trainingsläufen, bei denen mittels Bayesian-Optimisation geeignete Hyperparameter bestimmt werden sollten.

Untersucht wurden dabei die folgenden Parameter und Komponenten:

Für die Start-Lernrate wurde in jedem Sweep – unabhängig von anderen Parametern – stets im Bereich zwischen 0,002 und 0,0001 gesucht. Als Learning-Rate-Scheduler kam der Reduce-on-Plateau-Scheduler zum Einsatz, und als Optimizer wurde aus Effizienzgründen von Beginn an Adam festgelegt. Der Modell-Head wurde jeweils über 15 Epochen trainiert. Das gesamte Modell wurde zu Beginn über 35 Epochen, in späteren Experimenten über 25 Epochen feinjustiert.

Als Preprocessing-Methode wurde die Z-Scale-Intervall-Normalisierung verwendet.

Modell	Regularisierung	Weight-Decay	L1 λ	Dropout-Rate	Epochen
EfficientNet-b0	L1 und L2	0.05-0.0001	0.001-0.00005	0.3-0.5	15/35
EfficientNet-b3	L1	0	0.01-0.0001	0.3-0.5	15/35
EfficientNet-b3	L2	0.1-0.00001	0	0.3-0.5	15/35
EfficientNet-b3	L2	0.1-0.00001	0	0	15/35

Tabelle 10: Hyperparameter Set-Up für EfficientNet

Im Nachhinein mit Hep und $\gamma = 5$ wiederholt wurden:

Modell	Regularisierung	Weight-Decay	L1 λ	Dropout-Rate	Epochen
EfficientNet-b0	Keine	0	0	0	15/25
EfficientNet-b3	Keine	0	0	0	15/25

Tabelle 11: Nachträgliche Experimente

3.3.2.2 Test der HEP-Normalisierung

Bis kurz nach den ersten Trainingsversuchen mit EfficientNet wurde die Z-Scale-Intervall-Normalisierung als Standard-Preprocessing-Methode für das Modell-Setup verwendet, da die HEP-Normalisierung bis zu diesem Zeitpunkt tendenziell schlechtere Ergebnisse erzielt hatte.

Zu diesem Zeitpunkt wurde ein Fehler in der Implementierung der HEP-Normalisierung entdeckt, und es stellte sich die Frage, ob ein Shift um das lokale oder das globale Minimum, wie in Kapitel 3.1.1.2 erläutert, zu besseren Ergebnissen führen würde.

Drozdova et al. arbeiten in ihrer Publikation nicht mit negativen Pixelwerten und müssen daher keine Shifts durchführen [15].

Nach genauerer Betrachtung der Daten wurde die Hypothese aufgestellt, dass ein Shift um das globale Minimum die Werte der Pixelübergänge eines durchschnittlichen Bildes stark überschattet.

Eine Analyse der Verteilung der minimalen und maximalen Pixelwerte über alle Bilder im Datensatz zeigt, dass sowohl das globale Minimum als auch das globale Maximum wie in Kapitel 3.1.1.2 gezeigt deutlich vom Mittelwert der lokalen Minima/Maxima abweichen.

Konkret bedeutet das: Das globale Minimum beträgt etwa -0.037 , während das durchschnittliche lokale Minimum bei ca. $-0,0006$ liegt. Das durchschnittliche Maximum beträgt etwa 0.024 .

In einem typischen Bild befinden sich die meisten Pixelwerte in der Nähe des lokalen Minimums, mit einigen wenigen Werten, die im Anstiegsbereich bis zum Maximum liegen (vgl. Abbildung 15). Die Pixel im Bereich dieses Anstiegs bewegen sich also im Wertebereich zwischen ca. 0.00006 und 0.024 . Viele dieser Werte machen dabei nur einen kleinen Bruchteil des globalen Minimums aus und haben, wenn sie gemeinsam mit diesem Wert verschoben werden, nur einen geringen Einfluss auf die Normalisierung.

Die zweite Fragestellung, die im Zusammenhang des Preprocessings aufkam, war, ob das Z-Score-Scaling tatsächlich zu besseren Ergebnissen führen würde.

Daher wurden mit dem folgenden Modell die nachstehenden vier Sweeps durchgeführt:

Modell	Lernrate	Optimizer	LR-Scheduler	Regularisierung	L1 λ	Dropout-Rate
EfficientNet-b3	0.00025	ADAM	On-Plateau	L1	0.0005	0.5

Tabelle 12: Model Set-Up für HEP-Experiment

Shift	Scaling	Normalisierung	Zu testende γ
Globales-Minimum	Z-Score	HEP-Normierung mit gleichem γ auf allen Kanälen	$\gamma = [1, 2, 3, 5, 10, 20]$
Lokales-Minimum	Z-Score	HEP-Normierung mit gleichem γ auf allen Kanälen	$\gamma = [1, 2, 3, 5, 10, 20]$
Globales-Minimum	Kein Z-Score	HEP-Normierung mit gleichem γ auf allen Kanälen	$\gamma = [1, 2, 3, 5, 10, 20]$
Lokales-Minimum	Kein Z-Score	HEP-Normierung mit gleichem γ auf allen Kanälen	$\gamma = [1, 2, 3, 5, 10, 20]$

Tabelle 13: Testaufbau Hep-Experiment

3.3.2.3 Zweites EfficientNet Experiment

Ziel des zweiten EfficientNet-Experiments ist es erneut, die verschiedenen Preprocessing-Methoden als Hyperparameter zu untersuchen.

Aus den Ergebnissen des ersten Experiments (siehe Kapitel 4.2.1) ergab sich, dass die besten Resultate mit EfficientNet-B3, einer niedrigen Lernrate und Weight Decay erzielt wurden. Der Einsatz eines zusätzlichen Dropouts erwies sich dabei als optional.

Aus diesem Grund sollte für die folgenden Versuche ein Modell mit den nachstehenden Parametern verwendet werden:

Modell	Lernrate	Optimizer	LR-Scheduler	Regularisierung	Weight-Decay	Dropout-Rate
EfficientNet-b3	0.00025	ADAM	On-Plateau	L2	0.0001024	0.4

Tabelle 14: Optimales Set-up für Preprocessing Experiment mit EfficientNet

Leider unterlief beim Aufsetzen des Experiments ein Tippfehler: Anstatt mit L2-Regularisierung/Weight Decay zu arbeiten, wurden die Versuche versehentlich mit L1-Regularisierung konfiguriert. Der Fehler entstand dadurch, dass eine Konfiguration aus Experiment X übernommen, jedoch nur teilweise angepasst wurde.

In Experiment X wurde L1-Regularisierung mit einem $L1-\lambda = 0.0005$ verwendet. Zwar war der $L1-\lambda$ Wert in der aktuellen Konfigurationsdatei zur Sicherheit auf 0 gesetzt, was nicht nötig gewesen wäre, wenn der Regularisierungstyp korrekt angepasst worden wäre, jedoch wurde die Änderung des Regularisierungstyps vor dem Schließen der Datei offenbar nicht korrekt gespeichert.

Als der Fehler bemerkt wurde, waren bereits zu viele Versuche durchgeführt worden, als dass diese in vertretbarer Zeit hätten wiederholt werden können. Aus diesem Grund wurden auch die nachfolgenden Sweeps mit dem „falschen“ Setup weitergeführt, da das

Ziel dieses Experiments die Untersuchung verschiedener Preprocessing-Methoden ist und nicht der Vergleich von Regularisierungsarten.

Das tatsächlich verwendete Setup ist somit:

Modell	Lernrate	Optimizer	LR-Scheduler	Regularisierung	L1- λ	Dropout-Rate
EfficientNet-b3	0.00025	ADAM	On-Plateau	L1	0.0	0.4

Tabelle 15: Verwendetes Set-Up

Diese Hyperparameter-Kombination aus geringer Lernrate, Dropout und ohne weitere Regularisierung zeigte insgesamt eine eher schwache Leistung. Sämtliche in diesem Zusammenhang erzielten Ergebnisse lagen unterhalb der Resultate des ersten Experiments sowie des HEP-Experiments.

Da sich im HEP-Experiment gezeigt hatte, dass das Weglassen der Z-Score-Skalierung zu leicht besseren Ergebnissen führte, wurde der Großteil der Versuche ohne Z-Score-Skalierung durchgeführt.

Konkret wurden die folgenden Konfigurationen getestet:

Der Modell-Head wurde jeweils über 15 Epochen trainiert. Das gesamte Modell wurde zunächst über 35 Epochen feinjustiert; im letzten Versuch, dem hhz-Sweep, erfolgte das Fine-Tuning lediglich über 25 Epochen.

Diese Reduktion erfolgte aus Effizienz Gründen und wurde in Kauf genommen, da die Validierungsgenauigkeit bereits nach der 20. Epoche zu stagnieren begann und in der Regel bereits eine hohe Trainingsgenauigkeit ($> 0,99$) erreicht wurde.

Reihenfolge	Normalisierungsmethode	γ (Falls HEP)	Name im Code	Z-Score
N -> E	Ch1-3: Z-Scale-Intervall	-	z_scale_min_max	Nein
N -> E	Ch1-3: Min-Max	-	min_max	Nein
N -> E	Ch1-3: Hep	$\gamma = [3, 5, 20]$	hep	Nein
E -> N	Ch1: Hep Ch2: Hep Ch3: Hep	$\gamma_1 = [5]$ $\gamma_2 = [3, 10]$ $\gamma_3 = [5, 10, 20]$	hep	Ja
E -> N	Ch1: Hep Ch2: Hep Ch3: Hep	$\gamma_1 = [5]$ $\gamma_2 = [3, 10]$ $\gamma_3 = [5, 20, 30]$	hep	Nein
E -> N	Ch1: Hep Ch2: Hep Ch3: Z-Scale-Intervall	$\gamma_1 = [3, 5,]$ $\gamma_2 = [5, 10]$ $\gamma_3 = [0]$	hhz	Nein

E -> N	Ch1: Hep Ch2: Z-Scale-Intervall Ch3: Z-Scale-Intervall	$\gamma_1 = [3, 5, 10, 20, 30]$ $\gamma_2 = [0]$ $\gamma_3 = [0]$	hzz	Nein
E -> N	Ch1: Hep Ch2: Z-Scale-Intervall Ch3: Z-Scale-Intervall	$\gamma_1 = [3, 5, 10, 20, 30]$ $\gamma_2 = [0]$ $\gamma_3 = [0]$	hzz	Ja
E -> N	Ch1: Min-Max Ch2: Hep Ch3: Hep	$\gamma_1 = [0]$ $\gamma_2 = [2, 3, 5]$ $\gamma_3 = [5, 10, 20]$	hep_min_max	Nein
E -> N	Ch1: Min-Max Ch2: Z-Scale-Intervall Ch3: Hep	$\gamma_1 = [0]$ $\gamma_2 = [0]$ $\gamma_3 = [3, 5, 10, 20]$	hep_min_zscale	Nein

Tabelle 16: Untersuchte Preprocessingmethoden

3.3.3 Swin-Transformer

Alle verwendeten Swin-Modelle stammen aus der PyTorch-Bibliothek.[16] Getestet wurden die Varianten Swin-Tiny und Swin-Small. Dabei wurde kein eigener Klassifikations-Head definiert, sondern der im Modell bereits enthaltene Head verwendet. Dies ist vor allem darauf zurückzuführen, dass sich zu diesem Zeitpunkt bereits abzeichnete, dass ein systematischer Vergleich zwischen dem Original-Head und einem selbst hinzugefügten Head den Rahmen dieser Arbeit sprengen würde.

Ein zeitlich besonders aufwendiger Teil der Arbeit war das Inbetriebnehmen der Swin-Modelle. Zunächst wurden Trainingsversuche ausschließlich mit dem kleinsten Modell Swin-Tiny durchgeführt, wobei die zuvor verwendeten Lernraten-Scheduler und Optimizer übernommen wurden.

Dabei zeigte sich jedoch sowohl mit Adam als auch mit SGD stets das folgende Phänomen:

Das Modell trainierte den Klassifikations-Head zunächst erfolgreich, jedoch brachen sämtliche Metriken abrupt ein, sobald der Backbone freigegeben (unfreezed) wurde. Danach stagnierte das Training.

Da die Form der Accuracy-Kurven über die Epochen stark auf eine zu hohe Lernrate hinwiesen, wurden zunächst schrittweise kleinere Lernraten getestet.

Nachdem dies erfolglos blieb wurden sowohl der Optimizer als auch der Lernraten-Scheduler durch AdamW bzw. den Cosine-Annealing-Scheduler ersetzt, da diese auch im Original-Swin-Transformer-Paper zum Einsatz kamen.[13]

Nachdem auch diese Anpassungen nicht zum gewünschten Erfolg führten, wurde nach dem Entfrieren des Backbones eine Konstante implementiert, die im Konfigurationsfile angegeben werden konnte. Diese Konstante ermöglichte es, die Start-Lernrate nach dem Unfreeze des Backbones zu reduzieren, indem sie durch den Wert geteilt wurde.

Diese Maßnahme löste das Problem, was darauf hindeutet, dass beim Fine-Tuning des Backbones eine separate, deutlich kleinere Lernrate erforderlich ist, um bereits gelernte Merkmale nicht zu überschreiben.

Nach dieser Einarbeitungsphase wurde – erneut aus zeitlichen Gründen – entschieden, alle weiteren Tests mit AdamW und dem Cosine-Annealing-Scheduler durchzuführen. Diese Entscheidung stützt sich auf dem oben erwähnten Paper.

Zudem wurden, insbesondere während der Suche nach optimalen Hyperparametern, Trainingsläufe, die offensichtlich schlechtere Ergebnisse lieferten als vorherige, bei zufälliger Entdeckung manuell abgebrochen und verworfen.

Die unten aufgeführten Experimente dienen dem Zweck, die optimalen Hyperparameter- und Preprocessing-Methoden zu identifizieren.

Sweeps, die ausschließlich zu Debugging-Zwecken oder zur Einstellung der Lernrate durchgeführt wurden, sind hier nicht gelistet.

3.3.3.1 Erstes Swin-Transformer Experiment

In diesem Experiment wurden vier Hyperparameter-Sweeps mit Swin-Tiny und fünf Sweeps mit Swin-Small durchgeführt. Getestet wurden verschiedene Lernraten-Divisoren sowie – wie in den vorherigen Experimenten – unterschiedliche Regularisierungsstrategien.

Jeder Sweep bestand aus drei bis sechs Trainingsläufen, da in einigen Fällen vorzeitig abgebrochen wurde.

Untersucht wurden dabei die folgenden Parameter und Komponenten:

Für die Start-Lernrate wurde in jedem Sweep, unabhängig von den übrigen Parametern – stets im Intervall zwischen 0.005 und 0.0001 gesucht. Als Learning-Rate-Scheduler kam der Cosine Annealing-Scheduler zum Einsatz. Der Modell-Head wurde über 15 Epochen trainiert, das gesamte Modell anschließend über 35 oder 25 Epochen feinjustiert. Für alle Sweeps wurde, wie in Kapitel 3.1.1.2 beschrieben ein Z-Score Skalierung durchgeführt.

Modell	Regularisierung	Regularisierungsparameter: (Weight Decay/L1- λ)	Dropout-rate	Lernraten-divisor	Epochen
Tiny	Keine	-	0	0.1	35
Tiny	Keine	-	0	0.01	35
Small	Keine	-	0	0.1	25
Small	Keine	-	0	0.1	35
Small	Keine	-	0	0.01	25
Tiny	Weight-Decay	Min:0.1 Max:0.001	0	0.1	35
Small	Weight-Decay	Min:0.1 Max:0.001	0	0.1	25
Tiny	L1	Min:0.01 Max:0.001	0	0.1	25
Small	L1	Min:0.01 Max:0.001	0	0.1	25

Tabelle 17: Hyperparameter Set-Up für Swin-Transformer

3.3.3.2 Zweites Swin-Transformer Experiment

Aus den Ergebnissen des ersten Experiments (siehe Kapitel 4.3.1) ergab sich, dass Modelle ohne Regularisierung und mit einer Start-Lernrate von etwa 0.003 konsistent gute Resultate lieferten. Daher wurde die Untersuchung der Preprocessing-Methoden mit den Hyperparametern des best performenden Modells durchgeführt.

Modell	Lernrate	Lernraten-Divisor	Regularisierung	Dropout-rate
Swin-Small	0.00279	0.1	keine	0

Tabelle 18: Model Set-Up für Preprocessing-Experiment

Trainiert wurde in jedem Fall der Modell-Head über 15 Epochen, anschließend wurde das gesamte Modell über 25 Epochen feinjustiert.

Es wurden die folgenden Preprocessing-Methoden getestet. Für einen zusätzlichen Versuch mit HEP-, Z-Scale-Intervall- und Min-Max-Normalisierung auf jeweils einem Kanal blieb leider keine Zeit mehr.

Reihenfolge	Normalisierungsmethode	γ (Falls HEP)	Name im Code	Z-Score
N -> E	Ch1-3: Z-Scale-Intervall	-	z_scale_min_max	Ja
N -> E	Ch1-3: Min-Max	-	min_max	Ja
N -> E	Ch1-3: Hep	$\gamma = [2, 10, 20]$	hep	Ja
E -> N	Ch1: Hep Ch2: Hep Ch3: Hep	$\gamma_1 = [2, 3]$ $\gamma_2 = [5]$ $\gamma_3 = [3, 10, 20]$	hep	Ja

E -> N	Ch1: Hep Ch2: Hep Ch3: Z-Scale-Intervall	$\gamma_1 = [2, 5, 10]$ $\gamma_2 = [5, 20]$ $\gamma_3 = [0]$	hhz	Ja
E -> N	Ch1: Hep Ch2: Z-Scale-Intervall Ch3: Z-Scale-Intervall	$\gamma_1 = [2, 5, 10, 20]$ $\gamma_2 = [0]$ $\gamma_3 = [0]$	hzz	Ja
E -> N	Ch1: Min-Max Ch2: Hep Ch3: Hep	$\gamma_1 = [0]$ $\gamma_2 = [3, 5, 10]$ $\gamma_3 = [5, 10, 20]$	hep_min_max	Ja
E -> N	Ch1: Min-Max Ch2: Hep Ch3: Hep	$\gamma_1 = [0]$ $\gamma_2 = [5, 20]$ $\gamma_3 = [2, 5]$	hep_min_max	Nein

Tabelle 19: Zu testende Preprocessingmethoden

4 Resultate

4.1 ResNet

4.1.1 Erstes ResNet Experiment (Kapitel 3.3.1.1)

Zunächst wurden ResNet34 und ResNet50 miteinander verglichen, bevor anschließend das performanteste Modell identifiziert wurde. Die durchgeführten Sweeps werden im Folgenden abgekürzt notiert:

R34/R50 = ResNet34/ResNet50,

S/A = SGD/Adam,

X/L1/L2 = Keine Regularisierung / L1-Regularisierung / L2-Regularisierung (Weight Decay),

D = Dropout.

Verglichen werden hier die Ergebnisse der jeweils besten Modelle eines Sweeps auf dem Validierungsdatensatz:

Sweep	Accuracy	Precision	Recall	F1
R34-S-X	0.83095	0.81879	0.81635	0.81696
R50-S-X	0.84029	0.83248	0.83156	0.83171
R34-A-X	0.82774	0.82316	0.82134	0.8209
R50-A-X	0.85255	0.84481	0.84577	0.84449

R34-S-L2	0.82365	0.81697	0.81344	0.81433
R50-S-L2	0.85109	0.84412	0.84353	0.84359
R34-A-L2	0.82015	0.81379	0.81376	0.81354
R50-A-L2	0.84058	0.83439	0.83511	0.83368

Tabelle 20: Finale Metriken auf dem Validierungsset

ResNet50 übertraf dabei ResNet34 in allen Metriken.

Der Verlauf der Trainings- und Validierungsgenauigkeit (Accuracy) beider Modelle über die Epochen zeigt zudem, dass in beiden Fällen Overfitting ein zentrales Problem darstellt. Dies wird in den Abbildungen 21 und 22 qualitativ dargestellt.

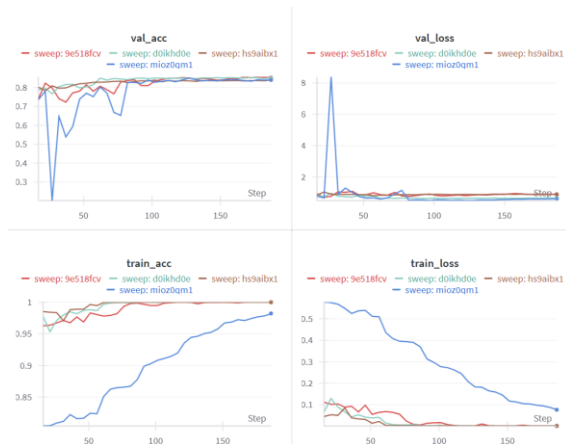


Abbildung 21: Aus WandB - Beste Runs der ResNet50 Sweeps

R50-S-X	hs9aibx1
R50-A-X	9e518fcv
R50-S-L2	d0ikhd0e
R50-A-L2	mioz0qm1

Tabelle 21: Beste Runs der ResNet50 Sweeps

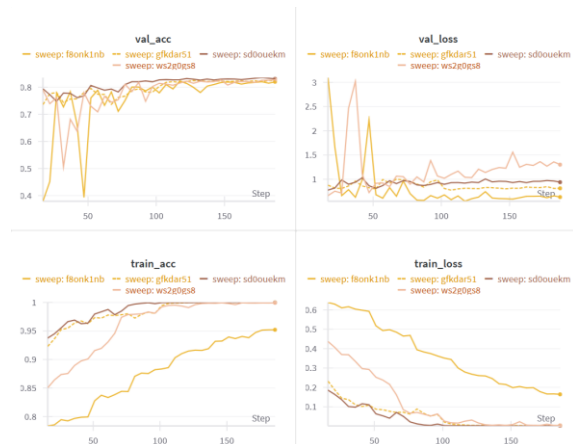


Abbildung 22: Aus WandB - Beste Runs der ResNet34 Sweeps

R34-S-X	sd0ouekm
R34-A-X	ws2g0gs8
R34-S-L2	gfkdar51
R34-A-L2	f8onk1nb

Tabelle 22: Beste Runs der ResNet34 Sweeps

Für die ResNet50-Sweeps zeigen Abbildung 23 und Abbildung 23 Validierungs- und Trainings-Accuracy sowie Precision, Recall und F1-Score, um performante Konfigurationen besser identifizieren zu können.

Da das Ziel die Identifikation der besten Modelle ist, wurden zur Verbesserung der Übersichtlichkeit Validierungs-Accuracies unter 0,75 aus den Plots entfernt. Runs des gleichen Sweeps wurden durch eine gestrichelte Linie verbunden.

Zusätzlich wurde eine Tabelle mit den fünf besten Runs aller Sweeps erstellt.

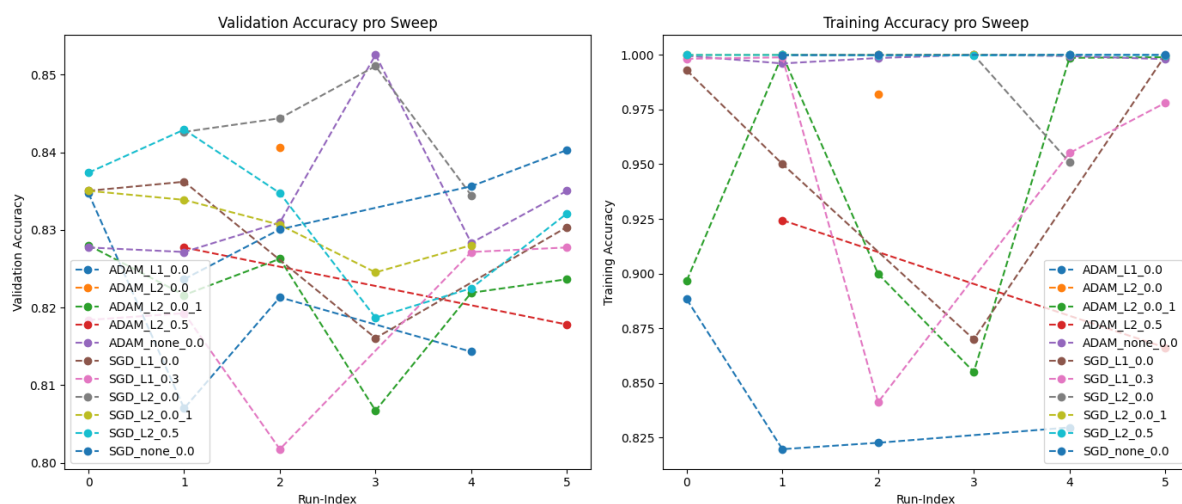


Abbildung 23: Validation und Test Accuracy der ResNet50 Sweeps

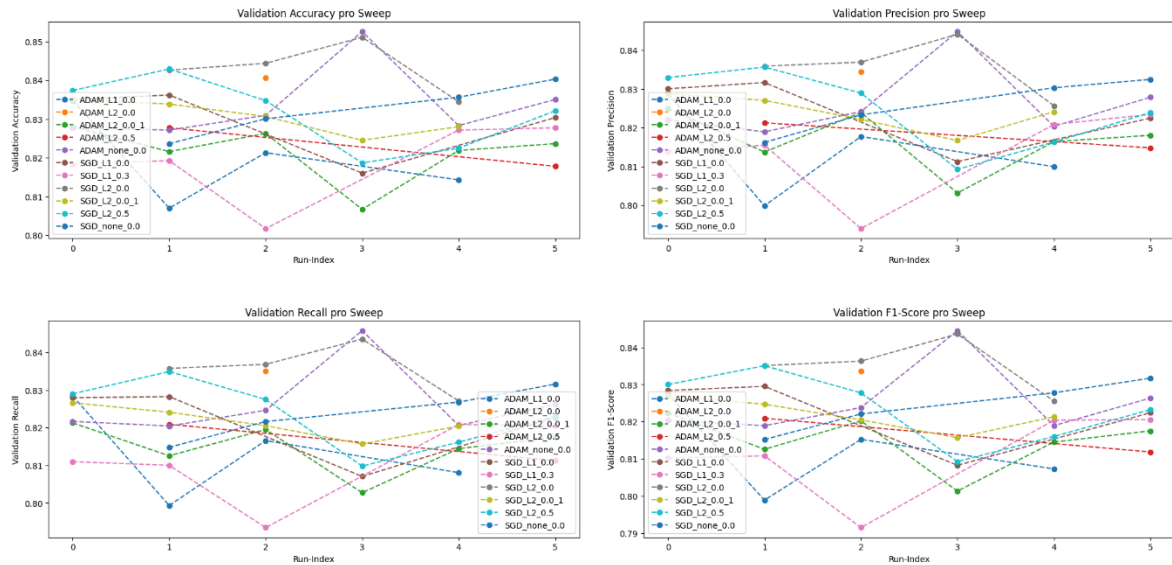


Abbildung 24: Metriken des Validation-Sets

Abbildung 24 zeigt, dass der SGD-Optimizer tendenziell gut mit Weight Decay funktioniert. Die Sweeps SGD-L2-0.5 (R50_S-L2_D), SGD_L2_0.0 (R50-S-L2 mit Suche über einen hohem Weight Decay Wertebereich) und SGD_L2_0.0_1 (R50-S-L2 mit Suche über einen tieferen Weight Decay Wertebereich) erzielten konsistent gute Resultate.

Für den ADAM-Optimizer scheint Weight Decay weniger gut zu funktionieren – bessere Ergebnisse wurden mit L1-Regularisierung oder ohne Regularisierung erzielt. Dies wirft die Frage auf, ob die gewählten Weight-Decay-Werte noch zu hoch sind. Betrachtet man jedoch Runs aus ADAM_L2_0.0_1 mit sehr kleinem Weight Decay (zwischen 0.0034 und 1.03×10^{-5}), so zeigt sich, dass diese durchgehend ähnlich oder schlechter abschneiden als Runs mit ADAM ohne Regularisierung.

Sweep	Name	Accu-racy	F1-Score	Preci-sion	Recall	Loss	Lern-Rate	DR-Rate	l1- λ	Weight-Decay
R50-A-X	flowing-sweep-4	0.85255	0.84449	0.84481	0.84577	0.88441	0.0002	0	0	0
R50-S-L2	vibrant-sweep-4	0.85109	0.84359	0.84412	0.84353	0.64932	0.00099	0	0	0.00144
R50-S-L2	cerulean-sweep-3	0.84438	0.8363	0.83692	0.83682	0.58198	0.00163	0	0	0.00592
R50-S-L2-D	generous-sweep-2	0.84292	0.83505	0.83565	0.83489	0.7511	0.00202	0.3	0	0.00123
R50-S-L2	vital-sweep-2	0.84263	0.8351	0.83589	0.83575	0.62013	0.00331	0	0	0.00248

Tabelle 23: Finale Metriken auf dem Validierungsset, sowie relevante Hyperparameter der besten 5 Runs

Die obige Tabelle zeigt die Metriken der fünf besten Modelle auf dem Validierungsdatensatz. Zwar belegt ein Modell mit dem ADAM-Optimizer den ersten Platz, insgesamt liefert jedoch SGD mit Weight Decay zuverlässiger gute Ergebnisse.

Wie in Abbildung 25 zu erkennen ist, tritt ein Overfitting auf. Diese Beobachtung deckt sich in Teilen mit den Ergebnissen von X, fällt in dieser Arbeit jedoch deutlich schwächer aus. Eine mögliche Erklärung dafür ist, dass die Validierungsdaten den Trainingsdaten dieser Arbeit sich stärker ähneln und ein insgesamt größerer Trainingsdatensatz verwendet wurde.

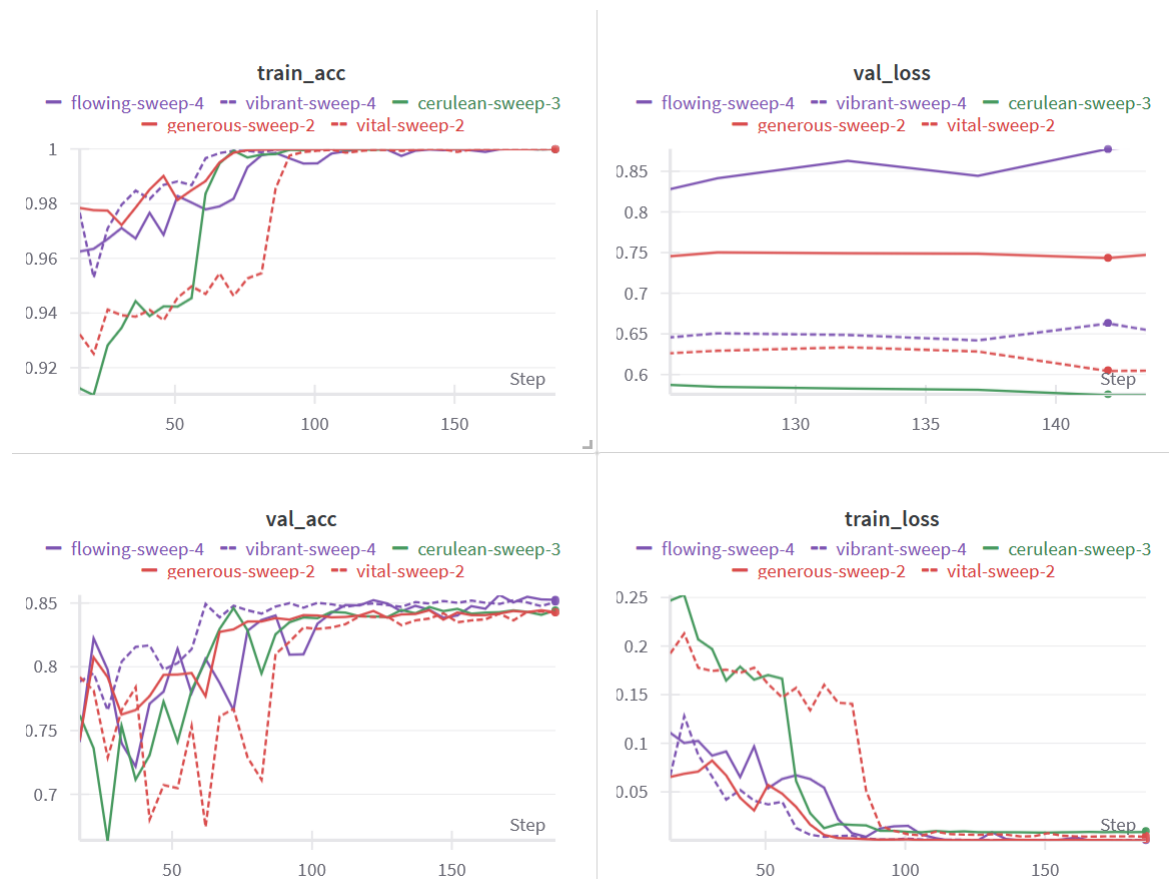


Abbildung 25: Trainings- und Validation-Loss und Accuracy

4.1.2 Zweites ResNet Experiment (Kapitel 3.3.1.2)

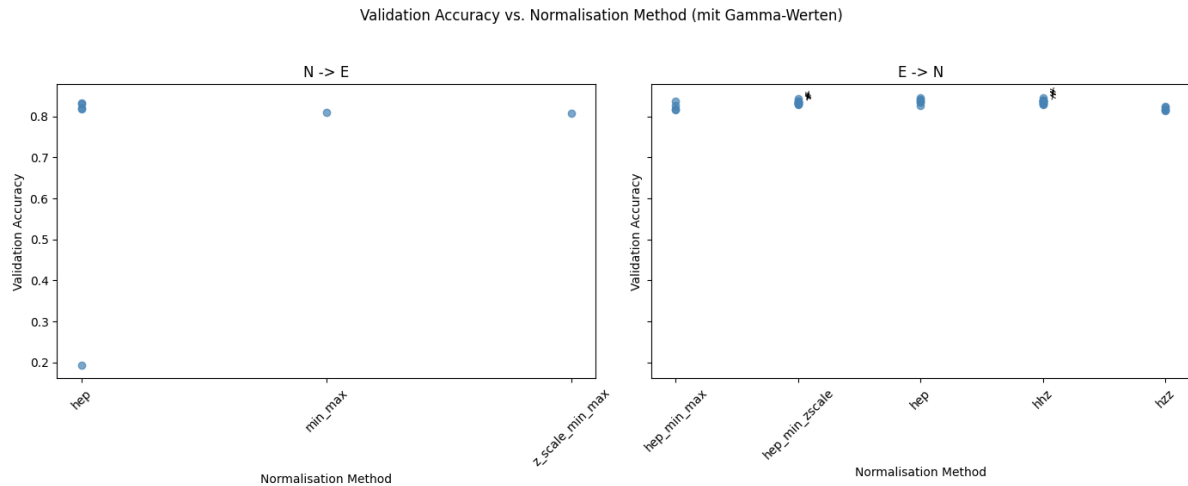


Abbildung 26: Validation Accuracy aller Runs mit allen Preprocess-methoden.

Abbildung 26 zeigt die Validierungsgenauigkeit jedes einzelnen Versuchs. Dabei fällt ein Run auf, der lediglich eine Accuracy von 0.2 erreicht. In diesem Experiment wurde die HEP-Normalisierung mit $\gamma = 1$ angewendet, noch bevor das Bild auf drei Kanäle erweitert wurde. Die entsprechende Formel lautet:

$$x_{\text{norm}} = \left(\frac{x - x_{\min}}{c - x_{\min}} \right)^{1/\gamma=1} = \frac{x - x_{\min}}{c - x_{\min}}$$

Dabei handelt es sich um eine Variante der Min-Max-Normalisierung, bei der jedoch nicht das lokale, sondern das globale Maximum zur Skalierung verwendet wurde.

Dies führt zwar zu Pixelwerten im Bereich von 0 bis 1, allerdings werden Kontraste stark abgeschwächt – die Unterschiede zwischen hellsten und dunkelsten Pixeln werden sehr klein. Wahrscheinlich ist der Run deshalb gescheitert.

Für die weitere Analyse wurde dieser Ausreißer entfernt. Abbildung 27 zeigt, analog zu Abbildung 26, die Validierungsgenauigkeiten der einzelnen Experimente. Versuche, bei denen kein zusätzliches Z-Score-Scaling durchgeführt wurde, sind in beiden Abbildungen mit einem „x“ markiert.

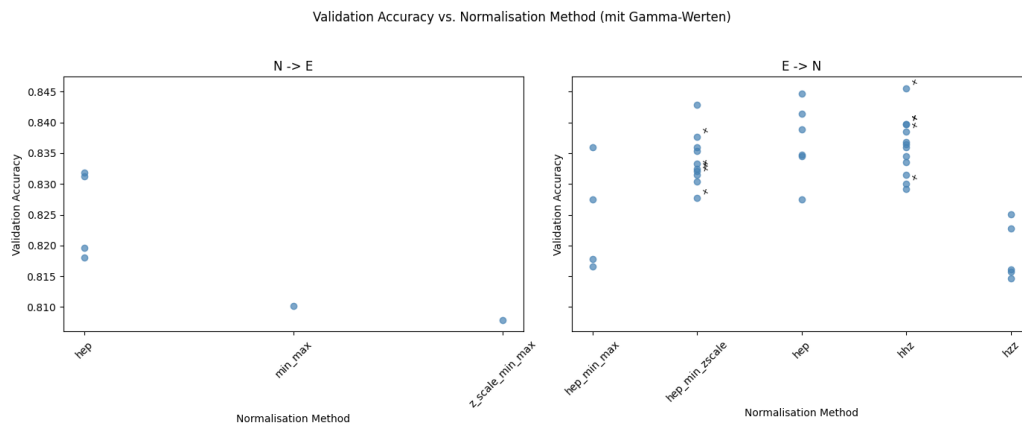


Abbildung 27: Validation Accuracies. Runs ohne Z-Score sind mit x gekennzeichnet

In Abbildung 27 zeigt sich einerseits, dass wenn auf allen Kanälen die gleiche Transformation angewendet wird, die HEP-Normalisierung deutlich besser funktioniert als Z-Score-Intervall- oder Min-Max-Normalisierung. Zudem ist, wie in Abbildung 28 erkennbar, dass kleinere γ -Werte (mit $\gamma > 1$) tendenziell zu besseren Ergebnissen führen als größere.

Zum anderen zeigt sich, dass Versuche, bei denen unterschiedliche Normalisierungen auf den verschiedenen Kanälen angewendet wurden, bessere Ergebnisse erzielen als Versuche, bei denen alle drei Kanäle gleich normalisiert wurden.

Eine Ausnahme bilden die Versuche mit der Normalisierungsmethode ,hhz', also die Runs, bei denen auf zwei Kanälen die Z-Scale-Intervall- und auf einem Kanal HEP-Normalisierung angewandt wurde.

Eine mögliche Erklärung ist, dass die Z-Scale-Intervall-Normalisierung auf zwei von drei Kanälen durchgeführt wurde und diese dadurch gegenüber dem HEP-Kanal dominieren. Da, wie in Abbildung 27 gezeigt, die Z-Scale-Intervall-Normalisierung schlechter abschneidet als die HEP-Normalisierung, könnte dies der Grund der schlechteren Ergebnisse sein.

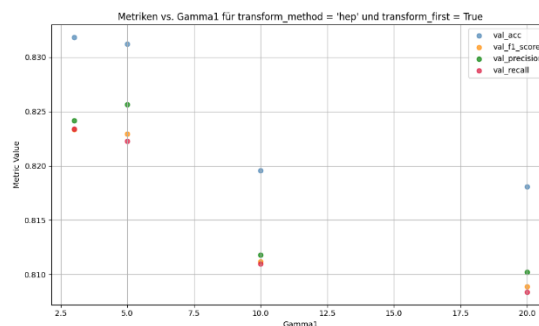


Abbildung 28: Metriken für HEP-Normalisierung auf mit uniformen Kanälen

Die besten Ergebnisse wurden Von hhz, hep und hep_min_zscale erzielt.

Dabei ist es schwer zu sagen, ob das Skalieren mit Z-Score nützlich ist. Für hzz wirkt sich das Skalieren eher nachteilig aus, allerdings ohne deutliche Verschlechterung. Bei hep_min_zscale hingegen scheint der Einfluss des Skalierens unbedeutend.

Weitergehende Forschung war in dem Umfang dieser Arbeit nicht abzubilden. Das Thema bietet Potenzial für weitere Analysen.

Außerdem wäre es spannend zu untersuchen, welche Abstände der γ -Werte bei der HEP-Normalisierung am besten funktionieren und ob sich daraus eine hilfreiche Heuristik ableiten lässt.

Im Zuge dieser Arbeit war es jedoch nicht möglich genügend Experimente mit unterschiedlichen γ -Kombinationen über alle drei Kanäle hinweg durchzuführen, um fundierte Aussagen in diesem Zusammenhang treffen zu können.

Name	Accuracy	F1-Score	Precision	Recall	Loss	Normalisierung	tags	γ_1	γ_2	γ_3
hardy-sweep-4	0.8455	0.8402	0.8410	0.8400	0.7226	hhz	'No-Z-Score'	5	10	0
earthy-sweep-6	0.8447	0.8371	0.8379	0.8370	0.6344	hep		2	5	5
glorious-sweep-6	0.8429	0.8380	0.8389	0.8376	0.7459	hep_min_zscale		0	0	10
stoic-sweep-2	0.8415	0.8328	0.8337	0.8331	0.6755	hep		2	2	5
kind-sweep-1	0.8397	0.8348	0.8367	0.8338	0.7284	hhz	'No-Z-Score'	5	30	0

Tabelle 24: Finale Metriken auf dem Validierungsset, sowie relevante Hyperparameter der besten 5 Runs

Die obige Liste zeigt die fünf besten Runs.

Der Verlauf der Accuracies und Losses in Abbildung 30 verdeutlicht, dass auch verschiedene Preprocessing-Methoden das Overfitting nicht reduzieren konnten. Abbildung 29 zeigt, dass Precision, Recall und F1-Score der Accuracy sehr genau folgen. Dies deutet darauf hin, dass sich False Positives und False Negatives im Wesentlichen die Waage halten.

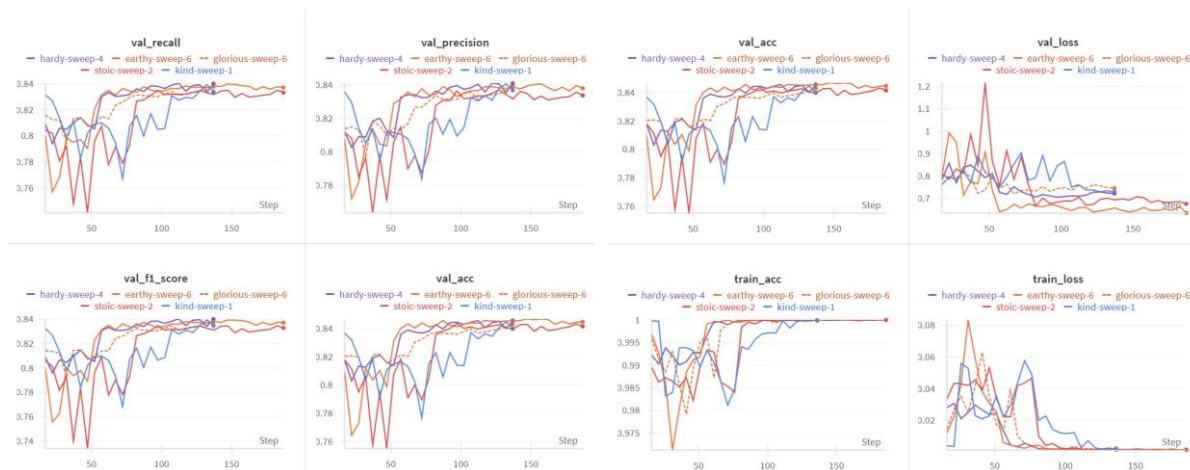


Abbildung 29: Metriken der besten 5 Modelle

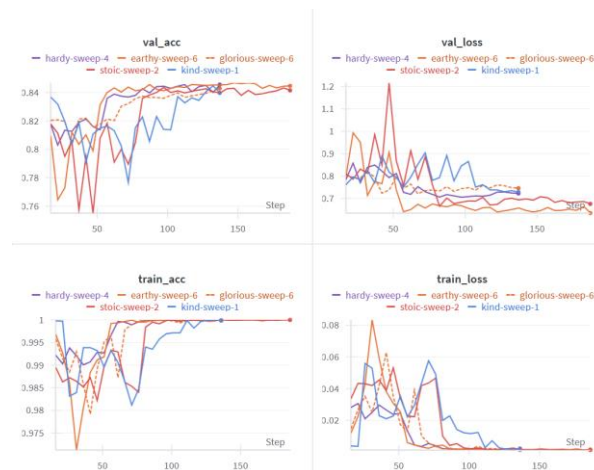


Abbildung 30: Trainings- und Validation-Loss und Accuracy der besten 5 Modelle

Die Konfusionsmatrix in Abbildung 31 zeigt, dass Klassen häufig paarweise verwechselt werden. Die Nummern in der Abbildung entsprechen dabei den folgenden Galaxienklassen:

0 = "1_1"	1 = 2_2	2 = 2_3	3 = 1_3	4 = 3_3	5 = 1_2
-----------	---------	---------	---------	---------	---------

Tabelle 25: Klassentabelle

Die Klasse 1_1 wird häufig mit 2_3 und 1_2 verwechselt. Auch tritt eine Verwechslung zwischen 2_2 und 3_3 auf.

Eine Tabelle aller häufigen Verwechslungen zeigt systematisch:

Galaxie	Häufigste Verwechslung	Weitere häufige Verwechslungen
1_1	1_2	2_3
2_2	3_3	1_3
2_3	1_2	1_1
1_3	2_2	3_3
3_3	2_2	2_3
1_2	2_3	1_1

Tabelle 26: Häufige Verwechslungen

Obwohl der Datensatz nach Peaks und Komponenten und nicht nach klassischen Galaxienklassen unterteilt ist, entsprechen 1_2 und 1_3 meist sogenannten FR-1-Galaxien, während 2_2 und 2_3 häufig FR-2-Galaxien darstellen. Dies könnte die Fehler zwischen 1_1 und 1_2 erklären. Eine Hypothese für die Verwechslungen von 1_1 und 2_3 aufzustellen gestaltet sich schwieriger.

Die Verwechslungen zwischen 3_3 und 2_2 lassen sich möglicherweise dadurch erklären, dass Galaxien mit mehreren Komponenten häufig eine geringere Zuverlässigkeitswert aufweisen, da sie für Freiwillige schwerer zu klassifizieren sind.

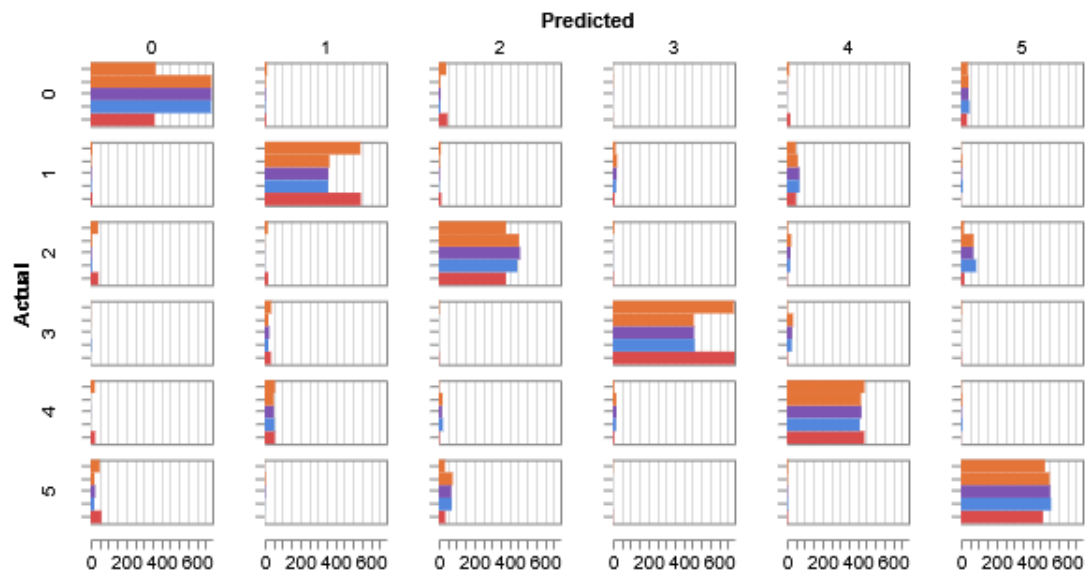


Abbildung 31: Konfusionsmatrix der besten 5 Modelle. Runs von Oben nach Unten: earthy-sweep-1, glorious-sweep-6, hardy-sweep-4, kind-sweep-1, stoic-sweep-2

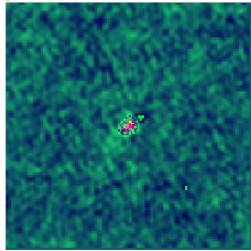
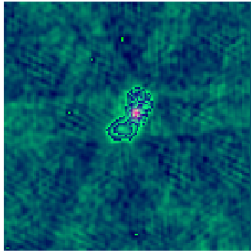
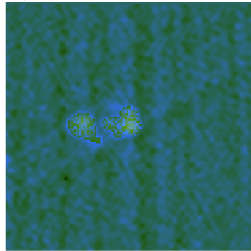
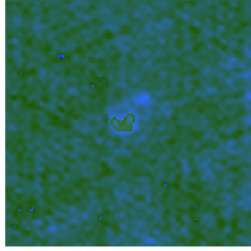
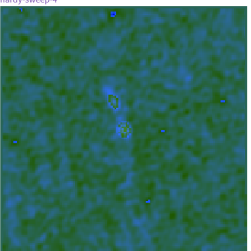
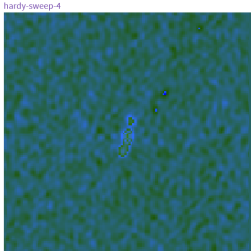
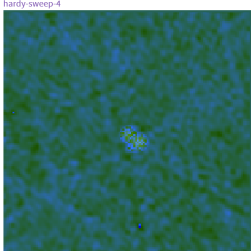
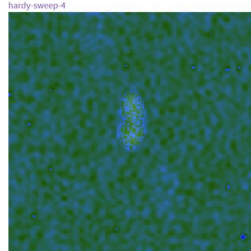
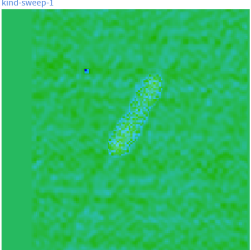
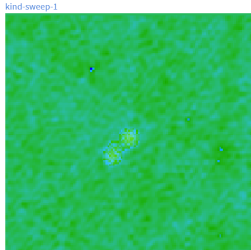
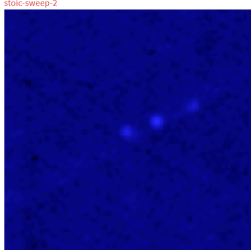
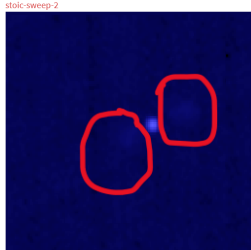
1	 True: [0], Pred: [0]	 True: [0], Pred: [2]	4	 True: [4], Pred: [4]	 True: [4], Pred: [1]
2	 True: [1], Pred: [1]	 True: [1], Pred: [4]	5	 True: [5], Pred: [5]	 True: [5], Pred: [2]
3	 True: [2], Pred: [2]	 True: [2], Pred: [5]	6	 True: [5], Pred: [5]	 True: [5], Pred: [0]

Abbildung 32: Tabelle von Falsch zugeordneten Galaxien (Rechts) und richtigen Vergleichsbildern (Links)

Die Betrachtung von Beispielbildern fehlerhafter Klassifikationen in Abbildung 32 (Eine grössere Abbildung befindet sich im Anhang) lässt keine eindeutige, allgemeingültige Ursache erkennen:

Die in Beispiel 1 der obigen Tabelle fehlklassifizierte Galaxie wirkt bereits visuell so, als könnte sie mehr als eine Komponente enthalten. In den Beispielen 2, 3 und 4 ähneln die falsch klassifizierten Galaxien intuitiv eher der jeweils vorhergesagten Klasse. In Beispiel 5 lässt sich der Fehlklassifikation schwer eine klare Ursache zuordnen, visuell erscheint die Galaxie durchaus als möglicher Vertreter der Klasse 1_2. In Beispiel 6 wiederum sind in den markierten Bereichen Strukturen erkennbar, die möglicherweise mit einem alternativen Preprocessing deutlicher hervorgetreten wären.

Die Fehler des Modelles sind, für das Laienauge eher schwierig nachzuvollziehen und könnte vielleicht auch genau daraus entstanden sein, dass diese Bilder schwer zu klassifizieren waren.

4.2 EfficientNet

4.2.1 Erstes EfficientNet Experiment (Kapitel 3.3.2.1)

Wie in dem ersten ResNet Experiment war es das Ziel das performanteste Modell zu identifizieren. Dazu zeigen Abbildung 33 und Abbildung 34 Validierungs- und Trainings-Accuracy sowie Precision, Recall und F1-Score. Runs des gleichen Sweeps wurden durch eine gestrichelte Linie verbunden.

Zusätzlich wurde eine Tabelle mit den fünf besten Runs aller Sweeps erstellt.

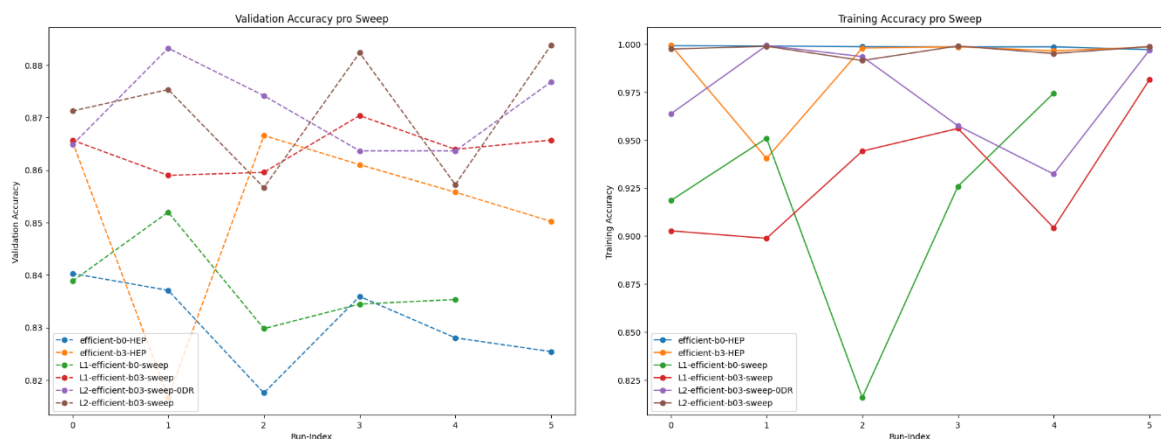


Abbildung 33: Validation und Test Accuracy der EfficientNet Sweeps

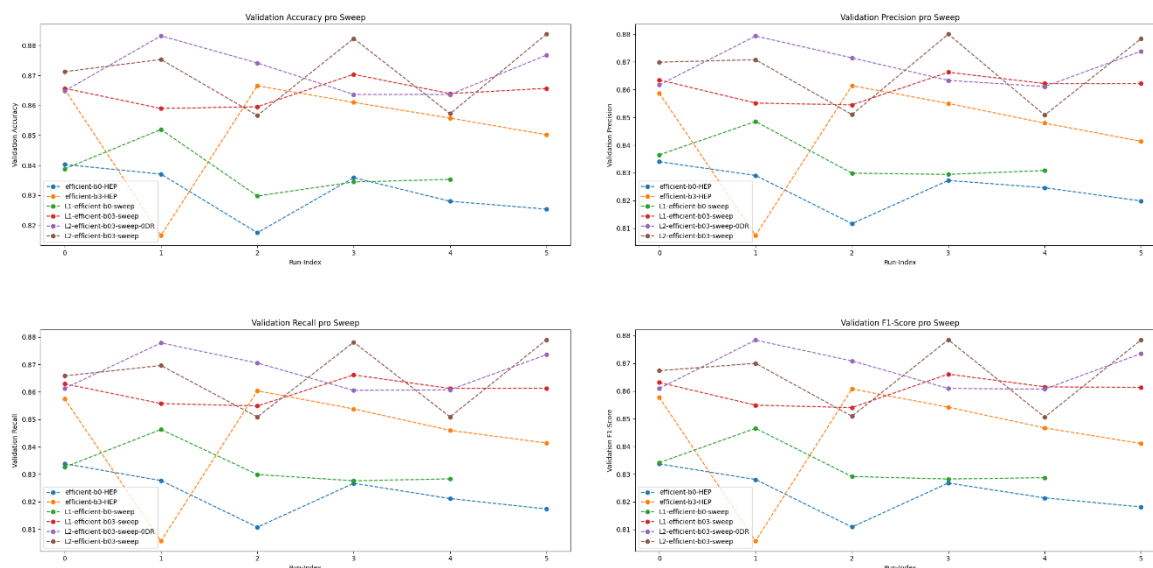


Abbildung 34: Sweep-Metriken des Validation-Sets

Abbildung 34 zeigt das EfficientNet-b3 generell besser performt als EfficientNet-b0. Dabei erzielen Versuche die Weight-Decay anwenden die besten Ergebnisse.

Die folgende Tabelle zeigt auf, dass eine kleine Lernrate von ca. 0.00025 relativ gut zu funktionieren scheint. Der Weight-Decay variiert etwas zwischen den Top 5 Runs, doch

teilen die besten 2 Runs ein Weight-Decay von ca. 0.000. Ob Dropout eingesetzt wird oder nicht scheint keinen grossen Effekt zu haben.

Run	Model	Accuracy	F1-Score	Precision	Recall	Loss	Lern-rate	DR-Rate	$l1$ λ	Weight-Decay
leafy-sweep-6	b3	0.88379	0.87843	0.87837	0.87901	0.62712	0.00027	0.5	0	0.00015084
exalted-sweep-2	b3	0.88321	0.87840	0.87935	0.87787	0.64504	0.00024	0	0	0.0001024
mild-sweep-4	b3	0.88233	0.87853	0.88005	0.87812	0.71569	0.00045	0.4	0	6.99E-05
wobbly-sweep-6	b3	0.87678	0.87353	0.87388	0.87357	0.49899	0.00036	0	0	0.00117564
charmed-sweep-2	b3	0.87532	0.87002	0.87082	0.86962	0.76065	0.00015	0.5	0	7.14E-05

Tabelle 27: Finale Metriken auf dem Validierungsset, sowie relevante Hyperparameter der besten 5 Runs

Abbildung 35 zeigt Trainings- und Validation-Loss und Accuracy der 5 Besten Runs aller EfficientNet Sweeps. Es zeigen sich erneut deutliche Anzeichen von Overfitting, die jedoch geringer als bei ResNet ausfallen.

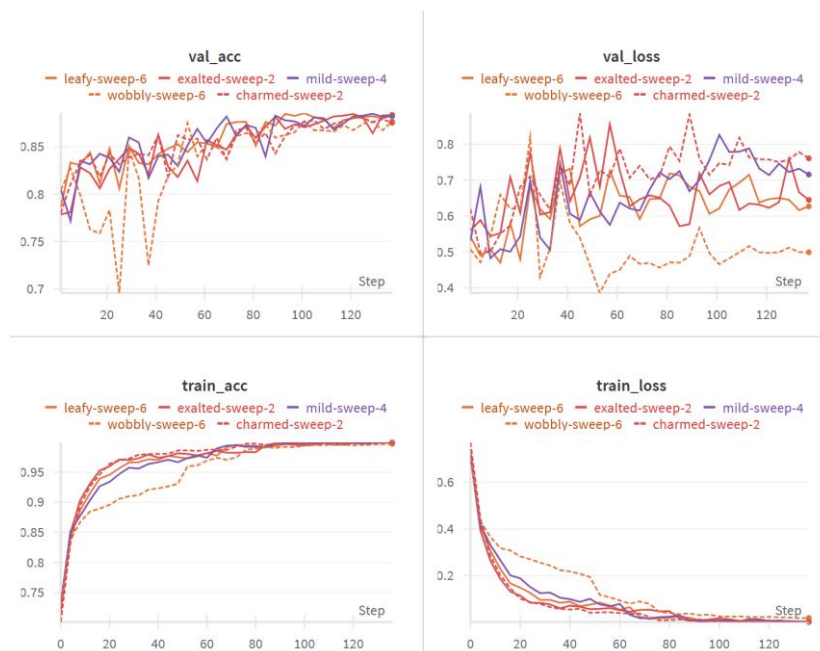


Abbildung 35: Trainings- und Validation-Loss und Accuracy

4.2.2 Test der Hep-Normalisierung (Kapitel 3.3.2.2)

Für jeden Sweep und jeden γ -Wert sind in der untenstehenden Tabelle die Validation Accuracies dargestellt. Diese wurden platzsparend in der Tabelle mit „Acc“ abgekürzt.

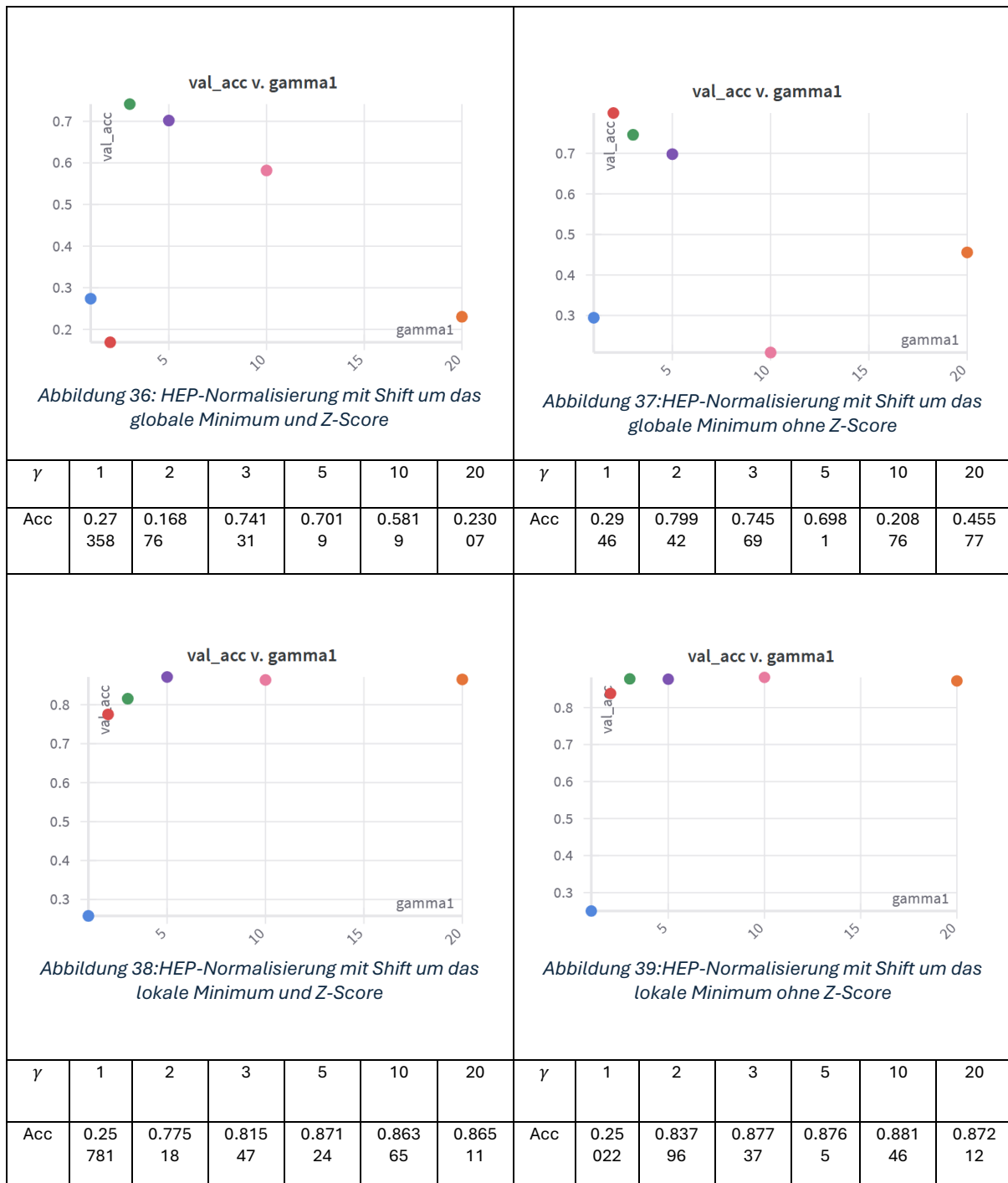


Tabelle 28: Ergebnisse des HEP-Experiments

Die obenstehende Tabelle zeigt, dass, wie erwartet, ein Shift um das lokale Minimum deutlich bessere Resultate liefert als ein Shift um das globale Minimum. In beiden Fällen schneiden die Sweeps ohne zusätzliches Z-Score-Scaling besser ab.

4.2.3 Zweites EfficientNet Experiment (Kapitel 3.3.2.2)

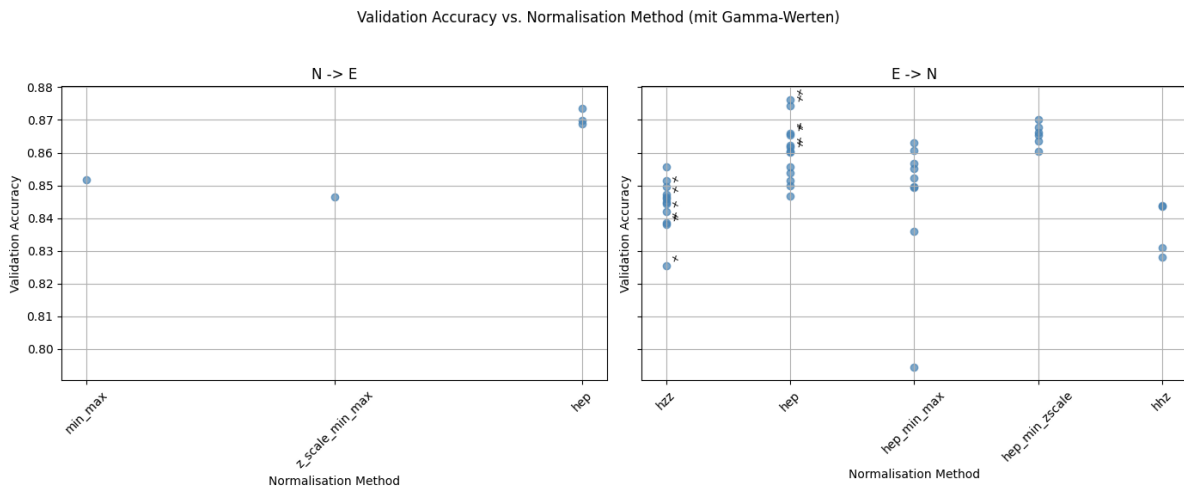


Abbildung 40: Validierungs Accuracies aller durchgeführten Runs

Abbildung 40 zeigt die Validierungs-Accuracy jedes einzelnen Versuchs. In dieser Darstellung sind alle Versuche, bei denen ein zusätzliches Z-Score-Scaling durchgeführt wurde, mit einem „o“ markiert.

Insgesamt schnitten Versuche, bei denen auf allen drei Kanälen dasselbe Preprocessing angewendet wurde, etwas besser ab als bei den ResNet-Experimenten. Innerhalb der getesteten $N \rightarrow E$ -Preprocessing-Methoden lieferte auch hier die HEP-Normalisierung die besten Ergebnisse.

Wie bereits in bei ResNet, gilt auch in diesem Fall, dass kleine γ -Werte (größer als 1) bessere Resultate erzielen als große.

Die besten Ergebnisse wurden mit der HEP-Normalisierung bei kanalweise unterschiedlichen γ -Werten erreicht. Bemerkenswert ist jedoch, dass selbst die HEP-Normalisierung mit einheitlichem γ noch alle anderen getesteten Normalisierungsverfahren, auch in Kombination mit kanalindividuellem Preprocessing, übertrifft. Dies stellt eine interessante Abweichung im Vergleich zu den ResNet-Ergebnissen dar.

Ebenfalls auffällig ist, dass beim hzz-Preprocessing die zusätzliche Z-Score-Skalierung zu keiner erkennbaren Verbesserung zu führen scheint, während sie bei der HEP-Normalisierung klar zu besseren Ergebnissen führt.

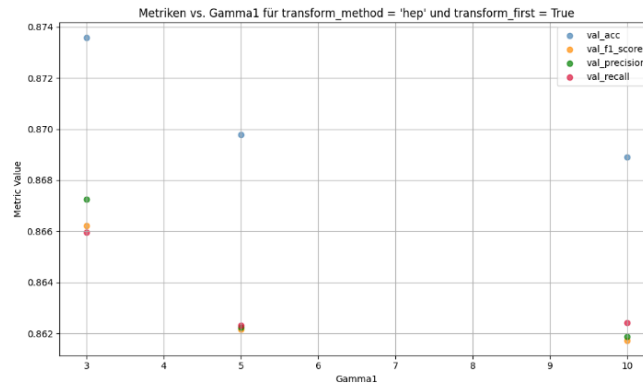


Abbildung 41: Metriken für HEP-Normalisierung auf mit uniformen Kanälen

Die Runs mit HEP-Scale-MinMax erzielten ebenfalls sehr gute Ergebnisse, was mit den Resultaten aus den ResNet-Experimenten übereinstimmt. Das hhz-Preprocessing hingegen schnitt deutlich schlechter ab.

Es lässt sich klar feststellen, dass bei allen Normalisierungsmethoden, die die drei Kanäle unterschiedlich behandeln, die Ergebnisse stark variieren – je nach Wahl von γ .

Betrachtet man die fünf besten Runs, zeigt sich, dass in den meisten Fällen überwiegend kleine γ -Werte verwendet wurden.

Name	Accuracy	F1-Score	Precision	Recall	Loss	Reihenfolge	Normalisierung	Tag	γ_1	γ_2	γ_3
hardy-sweep-4	0.87620	0.86700	0.86783	0.86677	0.76078	E -> N	hep	Z-Score	5	10	5
icy-sweep-1	0.87445	0.86584	0.86671	0.86574	0.84125	E -> N	hep	Z-Score	5	3	5
olive-sweep-1	0.87358	0.86620	0.86726	0.86597	0.81810	N -> E	hep		3	0	0
fearless-sweep-1	0.87007	0.86334	0.86334	0.86358	0.91893	E -> N	hep_min_z scale		0	0	5
olive-sweep-2	0.86978	0.86216	0.86223	0.86230	0.80610	N -> E	hep		5	0	0

Tabelle 29: Finale Metriken auf dem Validierungsset, sowie relevante Hyperparameter der besten 5 Runs

Precision, Recall, F1-Score und Accuracy lagen erneut nahe beieinander, was auf ein ausgeglichenes Verhältnis von False Positives zu True Negatives hinweist. Wie in Abbildung 42 erkennbar, bleibt Overfitting auch in diesen Experimenten ein relevantes Problem.

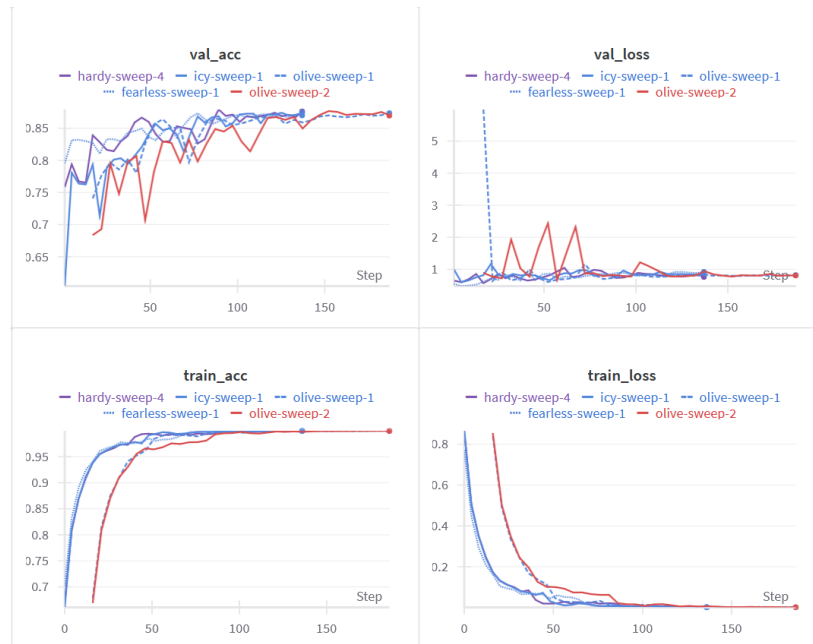


Abbildung 42: Trainings- und Validation-Loss und Accuracy der besten 5 Modelle

Ein Blick auf die Konfusionsmatrix in Abbildung 43 zeigt, dass hardy-sweep-4 und icy-sweep-1 andere Fehlermuster aufweisen als die übrigen drei Sweeps. Bei diesen beiden handelt es sich um die am besten performenden Sweeps, in denen mit HEP-Normalisierung gearbeitet wurde und auf allen drei Kanälen unterschiedliche γ -Werte angewendet wurden.

Häufige Verwechslungen in der Klassifikation traten insbesondere bei folgenden Klassenpaaren auf:

Verwechselt wurden:

Galaxie	Häufigste Verwechslung	Weitere häufige Verwechslungen
1_1	2_3	1_2, 2_2
2_2	3_3	2_3
2_3	1_1	2_2,
1_3	3_3	2_2
3_3	2_2	1_3
1_2	1_1	2_3

Tabelle 30: Häufige Verwechslungen

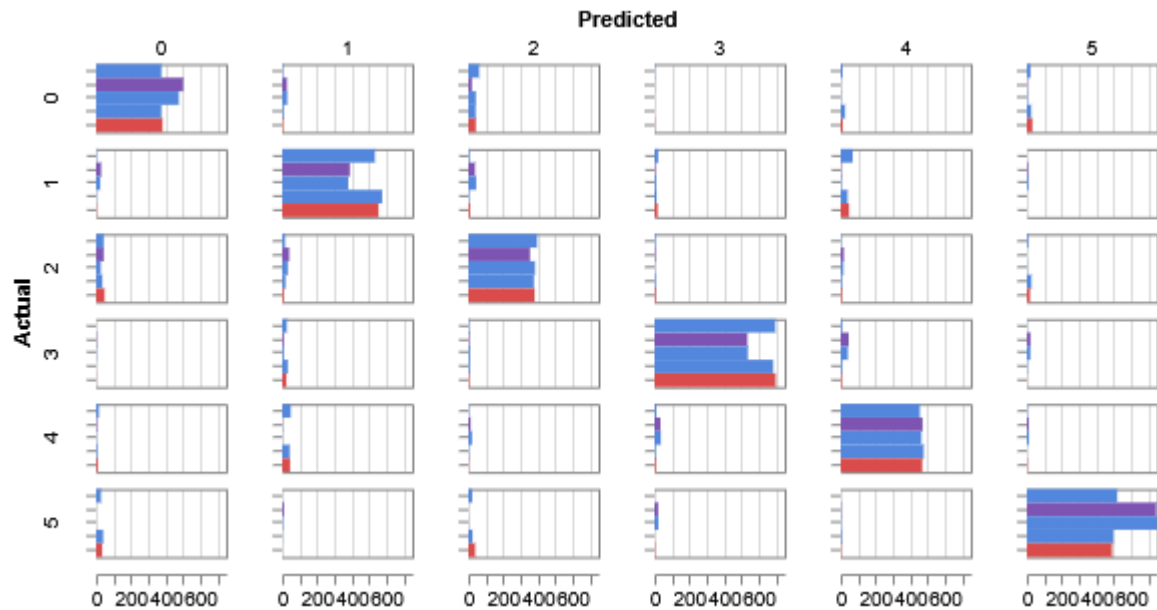


Abbildung 43: Konfusionsmatrix der besten 5 Modelle. Runs von Oben nach Unten: fearless-sweep-1, hardy-sweep-4, icy-sweep-1, olive-sweep-1, olive-sweep-2

4.3 Swin-Transformer

4.3.1 Erstes Swin-Transformer Experiment (3.3.3.1)

Abbildung 44 und Abbildung 45 Validierungs- und Trainings-Accuracy sowie Precision, Recall und F1-Score, um performante Konfigurationen besser identifizieren zu können.

Zusätzlich wurde eine Tabelle mit den fünf besten Runs aller Sweeps erstellt.

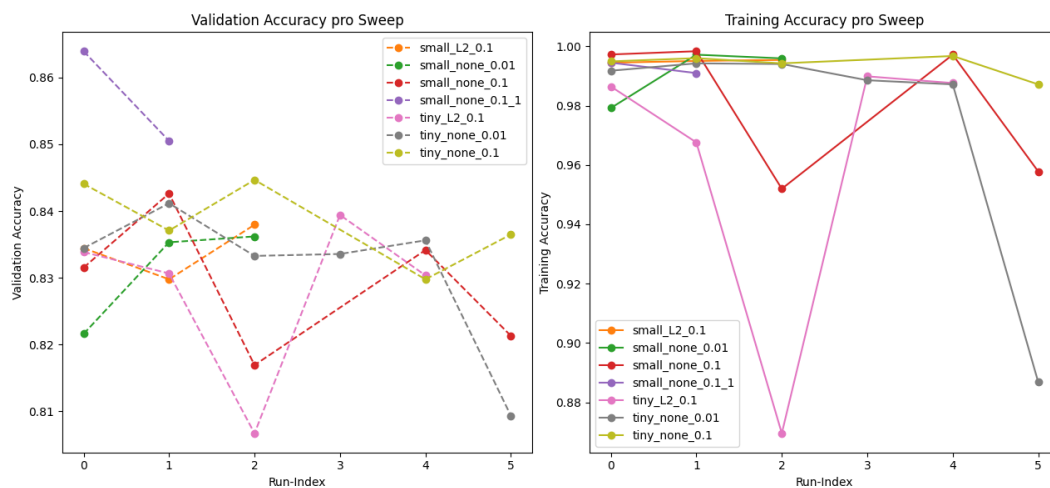


Abbildung 44: Validation und Test Accuracy der Swin-Transformer Sweeps

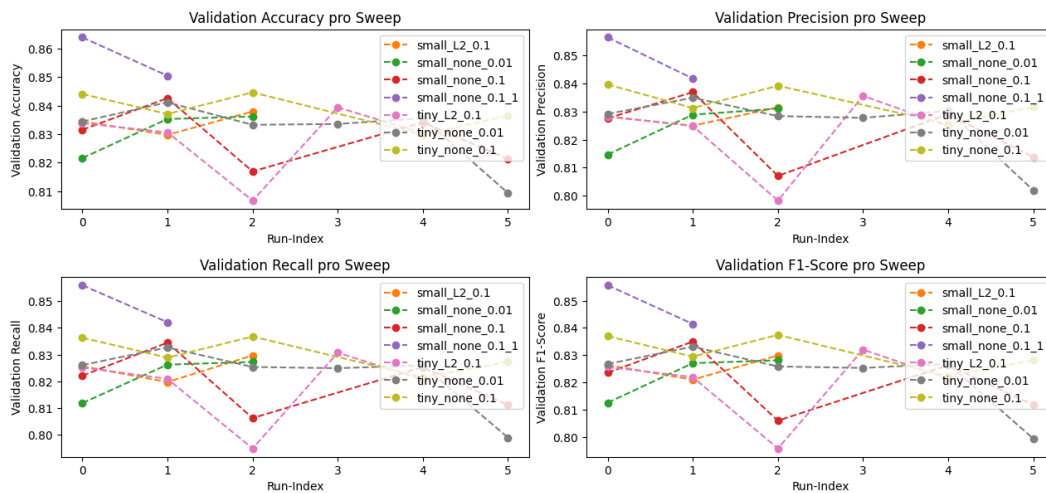


Abbildung 45: Metriken des Validation-Sets

Name	Modell	Accu- racy	F1- score	Preci- sion	Recall	Loss	Lern- rate	Regular- isierung	DR- Rate	LR- Div	Epoc hen
twilight-sweep-1	small	0.8639 4161	0.8556 5771	0.8565 0393	0.8559 5383	0.9454 8325	0.0027 8773	none	0	0.1	25
true-sweep-2	small	0.8505 1095	0.8414 4504	0.8418 484	0.8421 1812	0.9035 4276	0.0031 2858	none	0	0.1	20
firm-sweep-3	tiny	0.8446 7153	0.8374 2964	0.8392 487	0.8367 7012	1.1736 3258	0.0034 7958	none	0	0.1	35
northern-sweep-1	tiny	0.8440 8759	0.8369 7649	0.8397 0389	0.8363 2132	1.1542 6283	0.0041 1656	none	0	0.1	35
vibrant-sweep-2	small	0.8426 2774	0.8349 5451	0.8369 5802	0.8344 8297	1.2426 4358	0.0041 8832	none	0	0.1	35

Tabelle 31: Finale Metriken auf dem Validierungsset, sowie relevante Hyperparameter der besten 5 Runs

Die Obige Tabelle Sowie Abbildungen 44 und 45 zeigen das bei den Swin Modellen Keine Regularisierung am besten funktioniert. Spannend ist dabei die Tatsache, das die besten Runs nur 25-Epochen lang trainiert wurden. True-Sweep-2 und folgende wurden dabei frühzeitig bei Epoche 20 abgebrochen. LR-Div steht dabei für Lernratendivisor.

Der Vergleich der besten Sweeps der Grösse Small ohne Regularisierung und mit Lernratendivisor=0.1 in Abbildung 46 zeigt, dass es sich nicht um einen Fall handelte in welchem die Accuracy ab einem gewissen Zeitpunkt wegen Overfitting sinkt, sondern vermutlich einfach eine bessere Lernrate. Die für dieses Modle anscheinend bei etwas unter 0.003 liegt. Obwohl in Abbildung 49, vor allem in der Validation Loss-funktion sich doch klare Zeichen von Overfitting zeigen

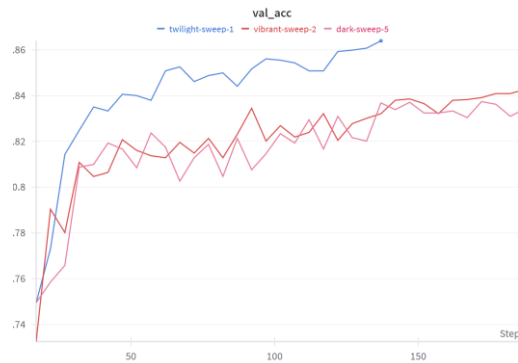


Abbildung 46: Validation Accuracy des besten unregularisierten Swin-Small Sweeps über 25-Epochen und der beiden besten Swin-Small Sweeps über 35-Epochen.

4.3.2 Zweites Swin Experiment (3.3.3.2)

Abbildung 47 zeigt, die Validierungsgenauigkeiten der einzelnen Experimente. Versuche, bei denen kein zusätzliches Z-Score-Scaling durchgeführt wurde, sind in beiden Abbildungen mit einem „x“ markiert.

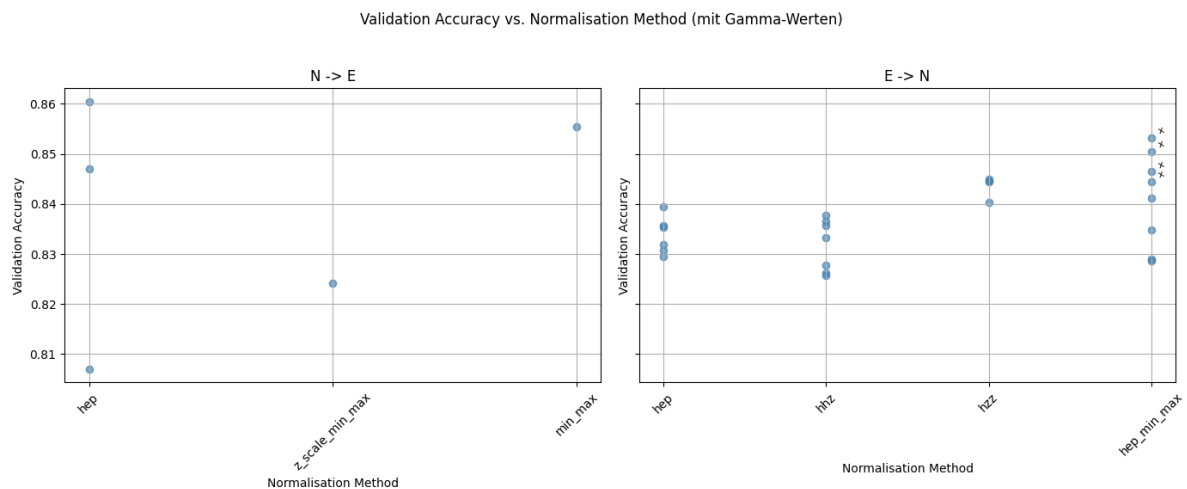


Abbildung 47: Validation Accuracies aller Versuche

Es zeigt sich, dass – anders als bei EfficientNet und ResNet – uniforme Kanalnormalisierungen beim Swin-Transformer gleich gut oder sogar besser funktionieren als Preprocessing-Methoden, bei denen für jeden Kanal eine unterschiedliche Normalisierungsmethode angewendet wurde.

Eine mögliche Hypothese hierfür ist, dass im Self-Attention-Mechanismus die Patch-Repräsentationen aller drei Kanäle geflattet werden. Bei einer uniformen Normalisierung würden sich gewisse Merkmale aller drei Kanäle innerhalb des resultierenden Vektors vereinfacht gesagt wiederholen, was zu einer strukturierten Repräsentation führt. Im Gegensatz dazu könnten kanalweise unterschiedliche Normalisierungen zu komplexeren Merkmalskombinationen führen, die schwerer zu verarbeiten sind.

Auch zeigt sich das bei der uniformen Kanalnormalisierung die HEP-Normalisierung mit $\gamma = 10$ immer noch die Beste Accuracy liefert doch ist sie nicht wie bei ResNet und EfficientNet in allen Fällen wesentlich besser als die anderen Methoden und liefert, mit $\gamma = 2$ auch das schlechteste Ergebnis über alle Versuche.

Abbildung 48 zeigt die Metriken aller Runs mit HEP-Normalisierung, bei denen alle drei Kanäle mit demselben γ normalisiert wurden.

Anderts als bei ResNet und EfficientNet erzielen hier grössere γ -Werte bessere Ergebnisse.

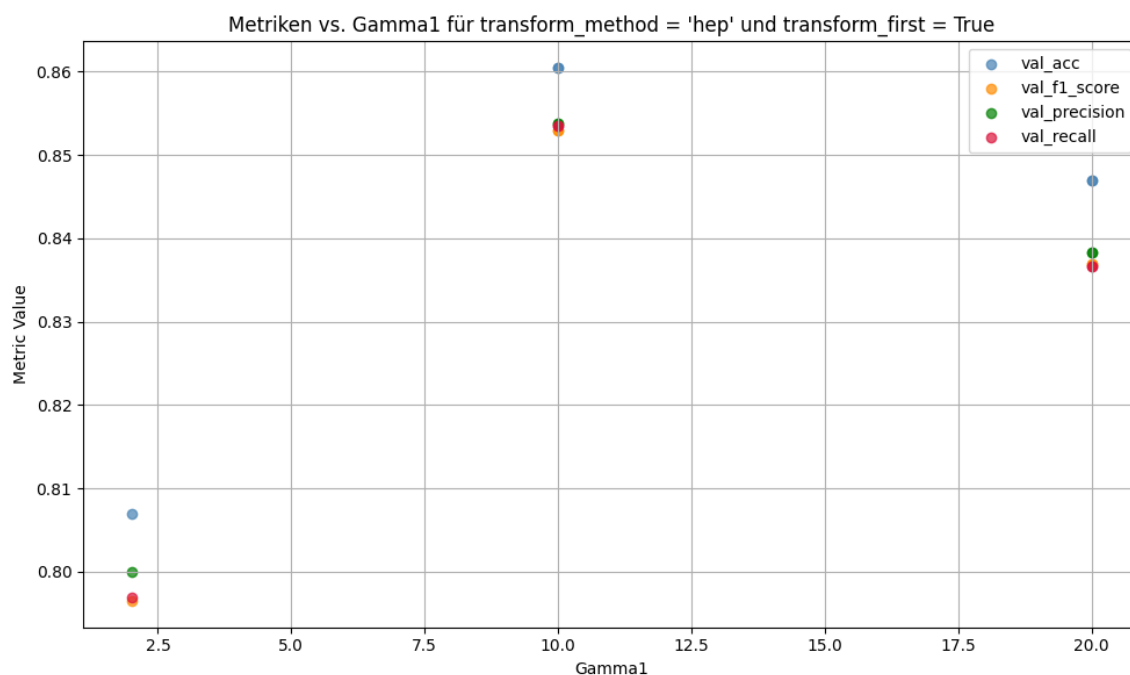


Abbildung 48: Metriken für HEP-Normalisierung mit gleichem γ auf allen 3 Kanälen

Zudem zeigt sich, dass bei der Funktion hep_min_max Runs mit zusätzlicher Z-Score-Normalisierung konstant bessere Ergebnisse erzielen als solche ohne.

In den Top 5 Runs Liegen, wie in allen andern Runs Alle Metriken vergleichbar nahe beieinander. Wie in Abbildung 49 ebenfalls erkennbar ist, stellt Overfitting auch hier ein Problem dar. Besonders deutlich wird dies im Verlauf des Validierungsverlustes.

Name	Modell	Accu-racy	F1-Score	Preci-sion	Recall	Loss	Reihen-folge	Normal-isierung	Scaling	γ_1	γ_2	γ_3
solar-sweep-2	small	0.86043	0.85294	0.85379	0.85352	0.93082	N -> E	hep	Z-Score	10	0	0
grateful-sweep-1	small	0.85547	0.84659	0.84796	0.84687	0.93067	N -> E	min_max	Z-Score	0	0	0
genial-sweep-2	small	0.85313	0.84362	0.84397	0.84397	0.91445	E -> N	hep_min_max	No-Z-Score	0	5	5
avid-sweep-1	small	0.85051	0.84066	0.84249	0.84011	0.97289	E -> N	hep_min_max	No-Z-Score	0	5	2
cool-sweep-3	small	0.84700	0.83691	0.83828	0.83668	0.98200	N -> E	hep	Z-Score	20	0	0

Tabelle 32: Finale Metriken auf dem Validierungsset, sowie relevante Hyperparameter der besten 5 Runs

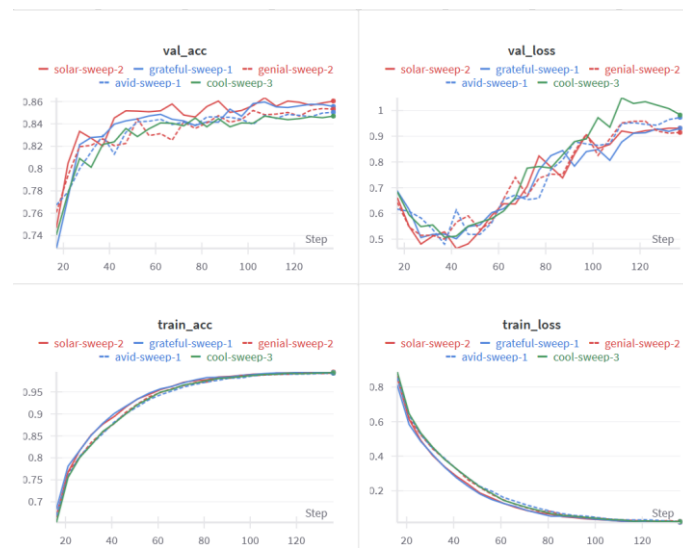


Abbildung 49: Trainings- und Validation-Loss und Accuracy der besten 5 Modelle

Betrachtet man die Konfusionsmatrix, so fällt auf, dass alle fünf Runs weitgehend ähnliche Fehlermuster aufweisen. In einigen Fällen stimmen diese mit den bei ResNet oder EfficientNet beobachteten Verwechslungen überein.

Häufig verwechselte Klassen waren:

Galaxie	Häufigste Verwechslung	Weitere häufige Verwechslungen
1_1	2_3	3_3, 1_2
2_2	3_3	1_3
2_3	1_1	2_2, 1_2
1_3	2_2	
3_3	2_2	1_1
1_2	1_1	2_3

Tabelle 33: Häufigste Verwechslungen

Es scheint also auch hier so, als würden die Verwechslungen in der Regel in beide Richtungen gehen. Also Galaxien der Klasse 1_1 werden zum Beispiel oft für Galaxien der Klasse 2_3 gehalten und umgekehrt

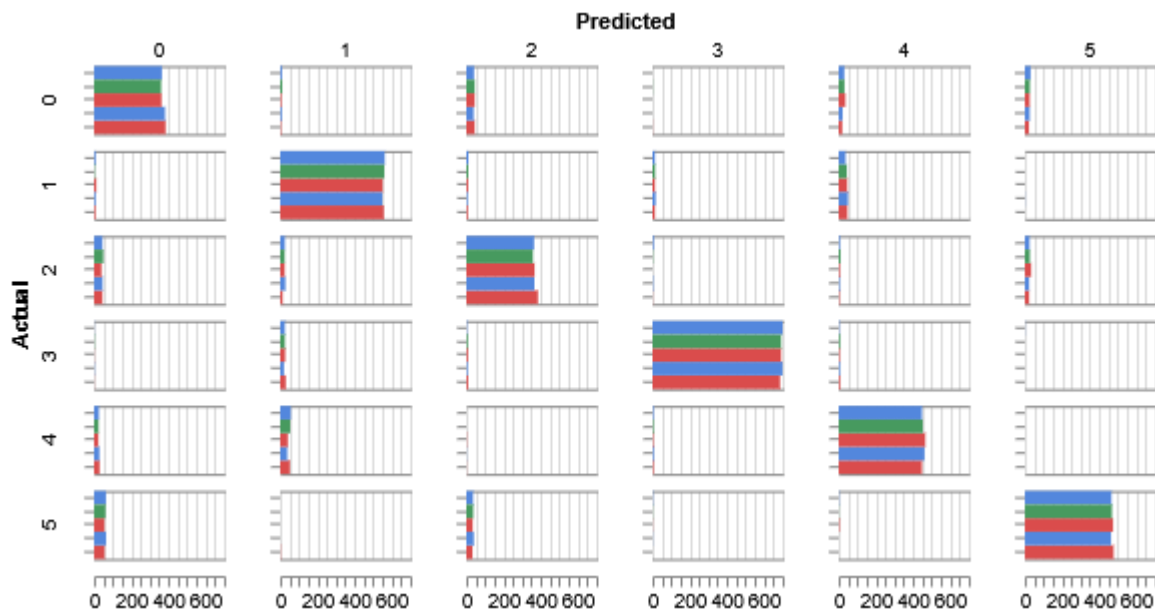


Abbildung 50: Konfusionsmatrix der besten 5 Modelle. Runs von Oben nach Unten: avid-sweep-1, cool-sweep-3, genial-sweep-2, grateful-sweep-1, solar-sweep-2

5 Diskussion und Ausblick

Ein Überblick über alle durchgeführten Experimente zeigt, dass EfficientNet und der Swin Transformer besser abschneiden als ResNet, wobei EfficientNet auch den Swin Transformer übertrifft.

Dieses Ergebnis steht im Einklang mit den auf Imagenet-1k bekannten Leistungsunterschieden der Architekturen auf ImageNet, wo ebenfalls sowohl EfficientNet als auch Swin-Transformer bessere Resultate erzielen als ResNet.

Dass EfficientNet in den vorliegenden Experimenten besser abschneidet als der Swin Transformer, könnte auf die Größe des verwendeten Datensatzes zurückzuführen sein. In der Arbeit von Dosovitskiy et al. wird gezeigt, dass Vision Transformer moderne CNNs vor allem bei sehr großen Datensätzen übertreffen.[10]

Modelle mittlerer Größe scheinen generell bessere Ergebnisse zu liefern als kleinere Modelle. Dabei war Overfitting in allen Versuchsreihen und architekturübergreifend ein wiederkehrendes Problem. Dieses Phänomen wird auch in der Arbeit von Becker et al. thematisiert.[4]

Das Overfitting konnte in den hier durchgeführten Experimenten ebenfalls nachvollzogen werden, wenn auch in abgeschwächter Form. Dies liegt an dem Unterschied der Arbeiten.

Diese Arbeit fokussiert sich nicht primär auf die Generalisierbarkeit auf neue, leicht abweichende Datensätze, sondern auf den Vergleich unterschiedlicher Modelle sowie die Untersuchung verschiedener Preprocessing-Methoden.

Dabei kommt ein deutlich größerer Trainingsdatensatz zum Einsatz, während der Validierungsdatensatz klein gehalten ist.

Im Gegensatz dazu verwenden Becker et al. zur Evaluierung einen Testdatensatz, der sich deutlich vom Trainingsdatensatz unterscheidet und diesen in Grösse überschreitet, was eine stärkere Aussage über die Generalisierungsfähigkeit ihrer Modelle erlaubt.

Ein Vergleich der Preprocessing-Methoden zeigt, dass unterschiedlich normalisierte Kanäle bei den beiden CNN-Architekturen besser funktionieren als uniform normalisierte Kanäle. Dies war bei ResNet besonders deutlich. Bei beiden CNNs war die HEP-Normalisierung mit unterschiedlichen γ -Werten sehr erfolgreich, wobei bei ResNet eine Kombination aus HEP auf zwei Kanälen und Z-Scale-Intervall-Normalisierung auf einem Kanal (ohne anschließendes Z-Score-Scaling) die besten Ergebnisse erzielte.

Für uniform normalisierte Kanäle war bei beiden CNNs die HEP-Normalisierung ebenfalls die beste Methode. Am besten funktionierten hier kleinere γ -Werte, die jedoch größer als 1 waren.

Beim Swin-Transformer haben sich die uniform normierten Kanäle mit schwachem Vorsprung durchgesetzt, wobei die Performance der HEP-Normalisierung auf einem Kanal von dem schlechtesten Overall-Wert bis zum Besten reicht.

Hier verhält es sich genau umgekehrt, nämlich dass grössere γ -Werte besser funktionieren.

Es wird, wie in den Resultaten angesprochen angenommen, dass diese Performance Unterschiede auf dem Unterschied der Architekturen beruht.

Ob das Scaling mit Z-Score einen tatsächlichen Nutzen bringt, lässt sich aus den vorliegenden Experimenten nicht eindeutig ableiten. Es sind hierfür weitere Untersuchungen notwendig.

Bei der HEP-Normalisierung zeigte sich jedoch klar, dass ein Shift um das lokale Minimum deutlich bessere Ergebnisse liefert als ein Shift um das globale Minimum.

Hierbei ergeben sich einige Ansätze für weiterführende Untersuchungen: Zum einen wäre es sinnvoll, weitere Experimente durchzuführen, die sich ausschließlich mit der Verteilung verschiedener γ -Werte auf den drei Kanälen befassen, um ein besonders geeignetes Set-up zu identifizieren.

Zum anderen ist der Datensatz für ein Preprocessing mit HEP-Normalisierung und einem Shift um das globale Minimum ungünstig verteilt. Es bietet sich jedoch an, den Datensatz durch gezieltes Cleanup in eine Form von $\mu \pm \sigma$ zu überführen, wobei μ den Mittelwert der lokalen Minima und σ ein sinnvolles Intervall beschreibt, innerhalb dessen

der Shift um das globale Minimum besser funktioniert als der Shift um das lokale Minimum ohne dabei zu viele Daten zu löschen.

Des Weiteren ist zu beachten, welche Architekturen welche Klassen verwechseln. Vergleicht man die Tabellen, so fallen gewisse Fehler auf, die alle Modelle wiederholen. Klassen 2_2 und 3_3 werden von allen Modellen am häufigsten miteinander, aber kaum mit anderen Klassen verwechselt. Klasse 1_1 wird in allen Fällen hauptsächlich mit 2_3 und 1_2 verwechselt und umgekehrt.

Das beste Modell war EfficientNet-B3 mit L2-Regularisierung und einer kleinen Lernrate. Das Beste in dieser Arbeit erzielte Ergebnis war eine Validierungsgenauigkeit von 88,4 %.

Dies reicht zwar nicht an die herausragenden 97 % von Alhassan et al. heran, was jedoch erwartbar war und sich in erster Linie durch Unterschiede in den verwendeten Datensätzen erklären lässt[3]

Der Datensatz von Alhassan et al. ist in lediglich vier Klassen unterteilt und folgt unter anderem der Fanaroff–Riley-Klassifikation [21]. Der in dieser Arbeit verwendete RGZ-OD-Datensatz unterscheidet hingegen sechs Klassen, wobei die Klassen 1_1 und 1_2 bzw. 2_2 und 2_3 zwar nicht exakt, aber häufig als Subklassen von FR-I bzw. FR-II interpretiert werden können.

Der verwendete Datensatz ist somit feiner unterteilt bzw. granularer, was eine direkte Vergleichbarkeit der Ergebnisse nur sehr eingeschränkt zulässt.

Nichtsdestotrotz könnten auch die Leistungen der in dieser Arbeit verwendeten Architekturen vermutlich noch weiter verbessert werden. Beispielsweise durch die Kombination des best performenden EfficientNet-Modells mit dem besten Preprocessing, durch das Experimentieren mit alternativen Modell-Heads oder durch die Variation von Hyperparametern, die in dieser Arbeit nicht systematisch untersucht wurden.

Auch wenn der „Sky“ hier vielleicht nicht das Limit ist, so ist doch immer noch ein bisschen Raum zu erkunden da.

6 Verzeichnisse

6.1 Quellenverzeichnis

- [1] «PDF». Zugegriffen: 6. Juni 2025. [Online]. Verfügbar unter:
<https://pos.sissa.it/215/021/pdf>
- [2] C. Wu u. a., «Radio Galaxy Zoo: ClaRAN - A Deep Learning Classifier for Radio Morphologies», *Mon. Not. R. Astron. Soc.*, Bd. 482, Nr. 1, S. 1211–1230, Jan. 2019, doi: 10.1093/mnras/sty2646.

- [3] W. Alhassan, A. R. Taylor, und M. Vaccari, «The FIRST Classifier: compact and extended radio galaxy classification using deep Convolutional Neural Networks», *Mon. Not. R. Astron. Soc.*, Bd. 480, Nr. 2, S. 2085–2093, Okt. 2018, doi: 10.1093/mnras/sty2038.
- [4] B. Becker, M. Vaccari, M. Prescott, und T. Grobler, «CNN architecture comparison for radio galaxy classification», *Mon. Not. R. Astron. Soc.*, Bd. 503, Nr. 2, S. 1828–1846, März 2021, doi: 10.1093/mnras/stab325.
- [5] «Papers with Code - ImageNet Benchmark (Image Classification)». Zugegriffen: 16. Mai 2025. [Online]. Verfügbar unter: <https://paperswithcode.com/sota/image-classification-on-imagenet>
- [6] A. Krizhevsky, I. Sutskever, und G. E. Hinton, «ImageNet classification with deep convolutional neural networks», *Commun. ACM*, Bd. 60, Nr. 6, S. 84–90, Mai 2017, doi: 10.1145/3065386.
- [7] «ImageNet Large Scale Visual Recognition Competition 2012 (ILSVRC2012)». Zugegriffen: 3. Juni 2025. [Online]. Verfügbar unter: <https://www.image-net.org/challenges/LSVRC/2012/results.html>
- [8] O. I. Wong u. a., «Radio Galaxy Zoo data release 1: 100185 radio source classifications from the FIRST and ATLAS surveys», *Mon. Not. R. Astron. Soc.*, Bd. 536, Nr. 4, S. 3488–3506, Feb. 2025, doi: 10.1093/mnras/stae2790.
- [9] S. J. D. Prince, *Understanding Deep Learning*. The MIT Press, 2023. [Online]. Verfügbar unter: <http://udlbook.com>
- [10] A. Dosovitskiy u. a., «An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale», 3. Juni 2021, *arXiv*: arXiv:2010.11929. doi: 10.48550/arXiv.2010.11929.
- [11] K. He, X. Zhang, S. Ren, und J. Sun, «Deep Residual Learning for Image Recognition», 10. Dezember 2015, *arXiv*: arXiv:1512.03385. doi: 10.48550/arXiv.1512.03385.
- [12] M. Tan und Q. V. Le, «EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks», 11. September 2020, *arXiv*: arXiv:1905.11946. doi: 10.48550/arXiv.1905.11946.
- [13] Z. Liu u. a., «Swin Transformer: Hierarchical Vision Transformer using Shifted Windows», 17. August 2021, *arXiv*: arXiv:2103.14030. doi: 10.48550/arXiv.2103.14030.
- [14] D. Tody, «The Iraf Data Reduction And Analysis System», gehalten auf der 1986 Astronomy Conferences, D. L. Crawford, Hrsg., Tucson, Okt. 1986, S. 733. doi: 10.1117/12.968154.
- [15] M. Drozdova u. a., «Radio-astronomical image reconstruction with a conditional denoising diffusion model», *Astron. Astrophys.*, Bd. 683, S. A105, März 2024, doi: 10.1051/0004-6361/202347948.
- [16] «PyTorch documentation — PyTorch 2.6 documentation — pytorch.org». [Online]. Verfügbar unter: <https://pytorch.org/docs/stable/index.html>
- [17] «Regularization in Machine Learning», GeeksforGeeks. Zugegriffen: 28. Mai 2025. [Online]. Verfügbar unter: <https://www.geeksforgeeks.org/regularization-in-machine-learning/>
- [18] «scikit-learn: machine learning in Python — scikit-learn 1.7.0 documentation». Zugegriffen: 6. Juni 2025. [Online]. Verfügbar unter: <https://scikit-learn.org/stable/>

- [19] «Transfer learning and fine-tuning | TensorFlow Core», TensorFlow. Zugriffen: 31. Mai 2025. [Online]. Verfügbar unter:
https://www.tensorflow.org/tutorials/images/transfer_learning
- [20] L. Melas-Kyriazi, *lukemelas/EfficientNet-PyTorch*. (30. Mai 2025). Python. Zugriffen: 30. Mai 2025. [Online]. Verfügbar unter:
<https://github.com/lukemelas/EfficientNet-PyTorch>
- [21] B. L. Fanaroff und J. M. Riley, «The Morphology of Extragalactic Radio Sources of High and Low Luminosity», *Mon. Not. R. Astron. Soc.*, Bd. 167, Nr. 1, S. 31P-36P, Apr. 1974, doi: 10.1093/mnras/167.1.31P.

6.2 Tabellenverzeichnis

Tabelle 1: Verteilung der RGZ-OD Datensatzes.....	27
Tabelle 2: Verteilung des Augmentierten Datensatzes	28
Tabelle 3: Mittelwert und Standardabweichung der Z-Score-Normalisierung.....	30
Tabelle 4: Fixe Konstanten und Hyperparameter	42
Tabelle 5: Versuchsaufbau mit ResNet34 und ResNet50.....	44
Tabelle 6: Versuchsaufbau mit nur ResNet50.....	44
Tabelle 7: Model Setup für zweites ResNet-Experiment.....	45
Tabelle 8: Zu testende Preprocessingmethoden mit Z-Score	46
Tabelle 9: Zu Testende Preprocessingmethoden mit Z-Score	46
Tabelle 10: Hyperparameter Set-Up für EfficientNet	47
Tabelle 11: Nachträgliche Experimente	47
Tabelle 12: Model Set-Up für HEP-Experiment.....	48
Tabelle 13: Testaufbau Hep-Experiment	49
Tabelle 14: Optimales Set-up für Preprocessing Experiment mit EfficientNet	49
Tabelle 15: Verwendetes Set-Up	50
Tabelle 16: Untersuchte Preprocessingmethoden	51
Tabelle 17: Hyperparameter Set-Up für Swin-Transformer	53
Tabelle 18: Model Set-Up für Preprocessing-Experiment.....	53
Tabelle 19: Zu testende Preprocessingmethoden	54
Tabelle 20: Finale Metriken auf dem Validierungsset	55
Tabelle 21: Beste Runs der ResNet50 Sweeps.....	56
Tabelle 22: Beste Runs der ResNet34 Sweeps	56
Tabelle 23: Finale Metriken auf dem Validierungsset, sowie relevante Hyperparameter der besten 5 Runs	57
Tabelle 24: Finale Metriken auf dem Validierungsset, sowie relevante Hyperparameter der besten 5 Runs	61
Tabelle 25: Klassentabelle.....	62
Tabelle 26: Häufige Verwechslungen	62
Tabelle 27: Finale Metriken auf dem Validierungsset, sowie relevante Hyperparameter der besten 5 Runs	66

Tabelle 28: Ergebnisse des HEP-Experiments.....	67
Tabelle 29: Finale Metriken auf dem Validierungsset, sowie relevante Hyperparameter der besten 5 Runs	69
Tabelle 30: Häufige Verwechslungen	70
Tabelle 31: Finale Metriken auf dem Validierungsset, sowie relevante Hyperparameter der besten 5 Runs	72
Tabelle 32: Finale Metriken auf dem Validierungsset, sowie relevante Hyperparameter der besten 5 Runs	75
Tabelle 33: Häufigste Verwechslungen	75

6.3 Abbildungsverzeichnis

Abbildung 1: Schematische Darstellung eines seichten Neuronalen Netzes. Darstellungen aus [9] nachempfunden.....	12
Abbildung 2: Übernommen aus [9]. Gezeigt wird ein 1D-Convolutional-Layer mit Kernelgrösse 3, Stride 2 und Dilation 1. In (a) und (b) wird dargestellt, wie ein Filter mit drei trainierbaren Gewichten $\omega_1, \omega_2, \omega_3$ über die Eingabe x gleitet. Zero-Padding wird links angewendet, um auch den ersten Eingabebereich abzudecken. Die dritte Darstellung rechts zeigt den vollständigen Filter und dessen Gewichtsmatrix, wobei identische Gewichte farblich gleich dargestellt sind. Nicht verwendete Gewichtseinträge (Nullen) sind weiss. Bias-Terme sind nicht gezeigt.....	17
Abbildung 3: Übernommen aus [9], Die Abbildung zeigt eine 2D-Convolution auf ein Bild angewandt. Dabei wird das Bild als zweidimensionaler Input mit drei Kanälen interpretiert, die den Rot-, Grün- und Blau-Komponenten entsprechen. Bei Verwendung eines 3×3 -Kernels wird jede Pre-Activation z_{ij} des Hidden-Layers als punktweise Multiplikation der 3×3 Weights Matrix mit dem entsprechenden 3×3 -RGB-Ausschnitt des Bildes berechnet, aufsummiert und um einen Bias ergänzt.	18
Abbildung 4: Multi-Head Self-Attention. Übernommen aus[9]	20
Abbildung 5: Aufbau des Vision Transformer. Übernommen aus [10]	21
Abbildung 6: Residual-Learningblock. Übernommen aus[11]	22
Abbildung 7: Abbildung eines ResNet, mit 24 Residual-Block mit Batchnorm, ReLU, Convolutional-Layers und Skip-Connections	22
Abbildung 8: Performance-Vergleich von EfficientNet mit anderen Bildklassifikatoren auf Imagenet. Übernommen aus[12]	24
Abbildung 9: Architektur eines Swin-Transformers und aufbau zweier aufeinanderfolgender Transformer blocks mit Window-Shift. Übernommen aus[13]...	25
Abbildung 10: Im Layer l (links) wird eine reguläre Fensteraufteilung verwendet, und die Self-Attention wird innerhalb jedes Fensters berechnet. Im nächsten Layer $l + 1$ (rechts) wird die Fensteraufteilung verschoben , wodurch neue Fenster entstehen. Die Self-Attention-Berechnung in diesen neuen Fenstern überschreitet die Grenzen der	

vorherigen Fenster aus Layer l und ermöglicht dadurch Verbindungen zwischen den Fenstern. Übernommen aus [13].....	26
Abbildung 11: Beispiele von verschiedenen Radioquellen	27
Abbildung 12: Verteilung des RGZ-OD Datensatz	28
Abbildung 13: Verteilung des verwendeten Datensatzes.	29
Abbildung 14: Beispiel eines nicht normalisierten Bildes. Die meisten Werte liegen um das Minimum mit zwei helleren Stellen.....	30
Abbildung 15: Verteilung der maximalen und minimalen Werte in allen Bildern.	31
Abbildung 16: Mit Min-Max Normalisiertes Bild. Die Form der Verteilung bleibt erhalten ist aber auf einen Grösseren Wertebereich skaliert	31
Abbildung 17: Mit dem Z-Scale-Intervall Normalisiertes Bild.	32
Abbildung 18: Mit HEP normalisiertes Bild. Gamma = 5. Es wurde ein Shift um das Lokale Minimum vorgenommen	33
Abbildung 19: Implementation der L1-Regularisierung.....	36
Abbildung 20: Code-Snippet das aufzeigt, wie ein Model geladen wird.	40
Abbildung 21: Aus WandB - Beste Runs der ResNet50 Sweeps	56
Abbildung 22: Aus WandB - Beste Runs der ResNet34 Sweeps	56
Abbildung 23: Validation und Test Accuracy der ResNet50 Sweeps	56
Abbildung 24: Metriken des Validation-Sets	57
Abbildung 25: Trainings- und Validation-Loss und Accuracy.....	58
Abbildung 26: Validation Accuracy aller Runs mit allen Preprocess-methoden.....	59
Abbildung 27: Validation Accuracies. Runs ohne Z-Score sind mit x gekennzeichnet ..	60
Abbildung 28: Metriken für HEP-Normalisierung auf mit uniformen Kanälen.....	60
Abbildung 29: Metriken der besten 5 Modelle	62
Abbildung 30: Trainings- und Validation-Loss und Accuracy der besten 5 Modelle	62
Abbildung 31: Konfusionsmatrix der besten 5 Modelle. Runs von Oben nach Unten: earthy-sweep-1, glorious-sweep-6, hardy-sweep-4, kind-sweep-1, stoic-sweep-2	63
Abbildung 32: Tabelle von Falsch zugeordneten Galaxien (Rechts) und richtigen Vergleichsbildern (Links)	64
Abbildung 33: Validation und Test Accuracy der EfficientNet Sweeps	65
Abbildung 34: Sweep-Metriken des Validation-Sets	65
Abbildung 35: Trainings- und Validation-Loss und Accuracy.....	66
Abbildung 36: HEP-Normalisierung mit Shift um das globale Minimum und Z-Score ...	67
Abbildung 37:HEP-Normalisierung mit Shift um das globale Minimum ohne Z-Score ..	67
Abbildung 38:HEP-Normalisierung mit Shift um das lokale Minimum und Z-Score	67
Abbildung 39:HEP-Normalisierung mit Shift um das lokale Minimum ohne Z-Score	67
Abbildung 40: Validierungs Accuracies aller durchgeführten Runs.....	68
Abbildung 41:Metriken für HEP-Normalisierung auf mit uniformen Kanälen	69
Abbildung 42: Trainings- und Validation-Loss und Accuracy der besten 5 Modelle	70
Abbildung 43: Konfusionsmatrix der besten 5 Modelle. Runs von Oben nach Unten: fearless-sweep-1, hardy-sweep-4, icy-sweep-1, olive-sweep-1, olive-sweep-2	71

Abbildung 44: Validation und Test Accuracy der Swin-Transformer Sweeps	71
Abbildung 45: Metriken des Validation-Sets	72
Abbildung 46: Validation Accuracy des besten unregulisierten Swin-Small Sweeps über 25-Epochen und der beiden besten Swin-Small Sweeps über 35-Epochen.	73
Abbildung 47: Validation Accuracies aller Versuche	73
Abbildung 48: Metriken für HEP-Normalisierung mit gleichem γ auf allen 3 Kanälen	74
Abbildung 49: Trainings- und Validation-Loss und Accuracy der besten 5 Modelle	75
Abbildung 50: Konfusionsmatrix der besten 5 Modelle. Runs von Oben nach Unten: avid-sweep-1, cool-sweep-3, genial-sweep-2, grateful-sweep-1, solar-sweep-2	76

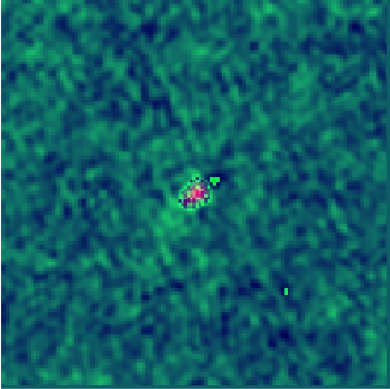
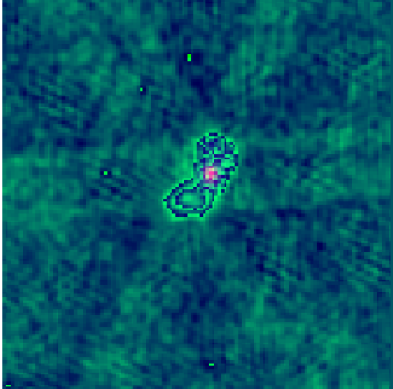
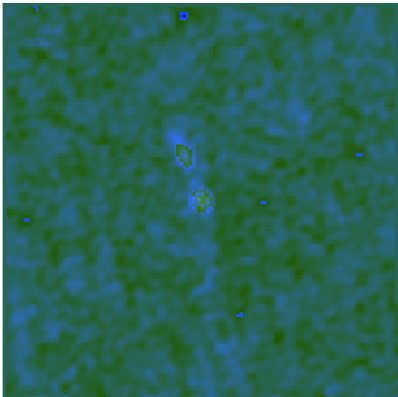
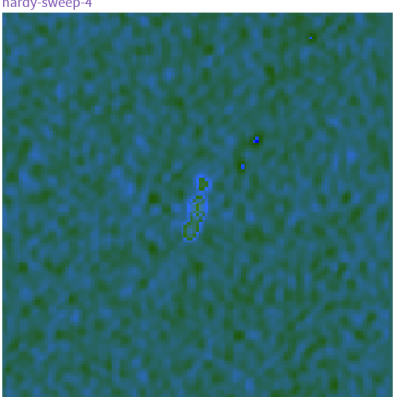
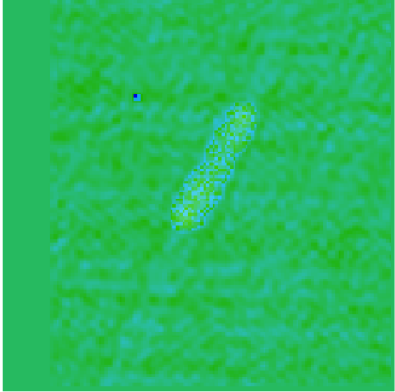
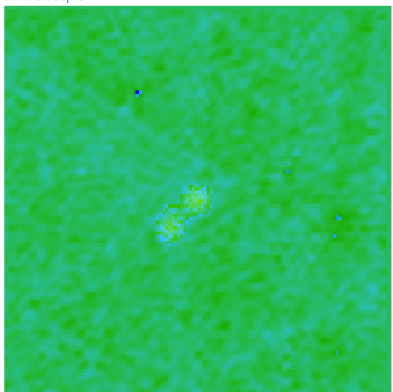
7 Anhang

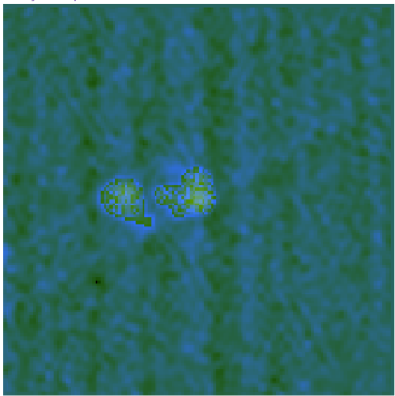
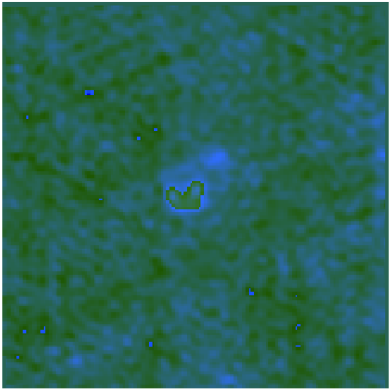
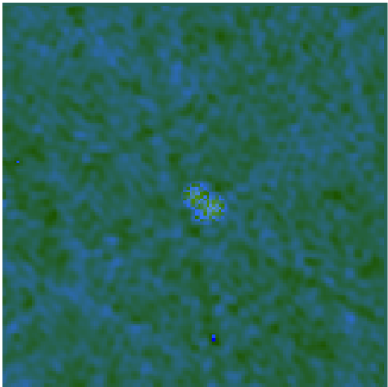
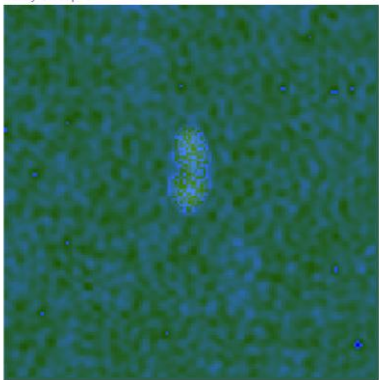
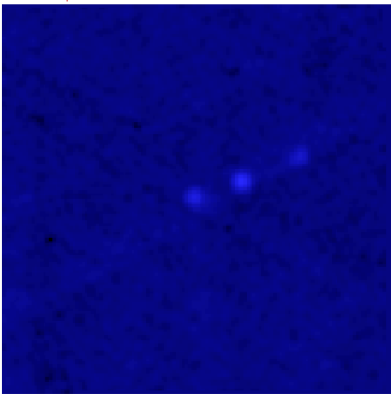
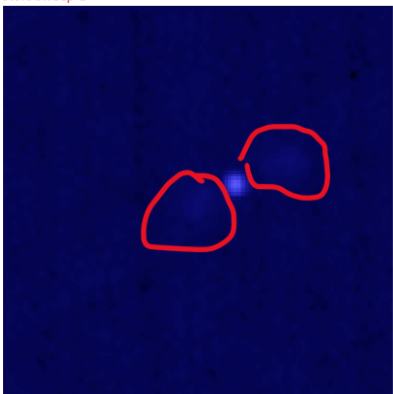
7.1 Aufgabenstellung aus Complexis

7.2 Weitere Anhänge

Morph	Count	Morph	Count	Morph	Count
1C-1P	49766	2C-5P	36	4C-6P	7
1C-2P	14242	2C-6P	7	4C-7P	5
1C-3P	1412	2C-7P	2	5C-5P	28
1C-4P	191	3C-3P	1347	5C-6P	11
1C-5P	28	3C-4P	163	5C-7P	1
1C-6P	12	3C-5P	20	6C-6P	2
1C-7P	3	3C-6P	13	6C-7P	1
2C-2P	9772	3C-7P	2	7C-7P	2
2C-3P	1220	4C-4P	99	7C-10P	1
2C-4P	181	4C-5P	18		

Anhang: 1: Übernommen aus [2]. Galaxientypen und deren Häufigkeit in RGZ DR1

1	glorious-sweep-6  True: [0], Pred: [0]	glorious-sweep-6  True: [0], Pred: [2]
2	hardy-sweep-4  True: [1], Pred: [1]	hardy-sweep-4  True: [1], Pred: [4]
3	kind-sweep-1  True: [2], Pred: [2]	kind-sweep-1  True: [2], Pred: [5]

4	hardy-sweep-4  True: [4], Pred: [4]	hardy-sweep-4  True: [4], Pred: [1]
5	hardy-sweep-4  True: [5], Pred: [5]	hardy-sweep-4  True: [5], Pred: [2]
6	stoic-sweep-2  True: [5], Pred: [5]	stoic-sweep-2  True: [5], Pred: [0]

Config.yaml

method: "bayes" # "grid" oder "bayes" sind auch möglich

metric:

goal: "maximize"

name: "score"

parameters:

transform_method:

values: ["hzz"] #hep,hzz, hhz, z_scale_min_max

transform_first:

value: False

gamma1:

values: [2]

gamma2:

values: [1]

gamma3:

values: [1]

model_architecture:

value: "EfficientNet"

model_name:

value: "efficientnet-b3"

activation_function:

value: "NotDefined"

in_features:

value: 1536

img_size:

value: 300

optimizer:

value: "ADAM"

scheduler:

value: "plateau"

learning_rate:

#value: 0.00025

#min: 0.0001

#max: 0.001

learning_rate_div:

value: 0.1

reg_type:

value: "L1"

l1lambda:

values: [0.0005]

#min: 0.0001

#max: 0.001

weight_decay:

values: [0]

distribution: "log_uniform_values"

min: 0.0001

max: 0.1

dropout:

values: [0.3]

num_epochs_lastlayer:

value: 1

num_epochs_finetune:

value: 1

momentum:

value: 0.91

betas:

value: [0.9, 0.999]

eps:

value: 1e-08

device:

value: "cuda"

enable_logging:

value: True # enable logging to WandB

run_on_cluster:

value: True # change configs to run on cluster

random_seed:

value: 42

num_classes:

value: 6

batch_size:

value: 32